



PUC Minas

BALANCEAMENTO, HASH E ESTRUTURA HÍBRIDA

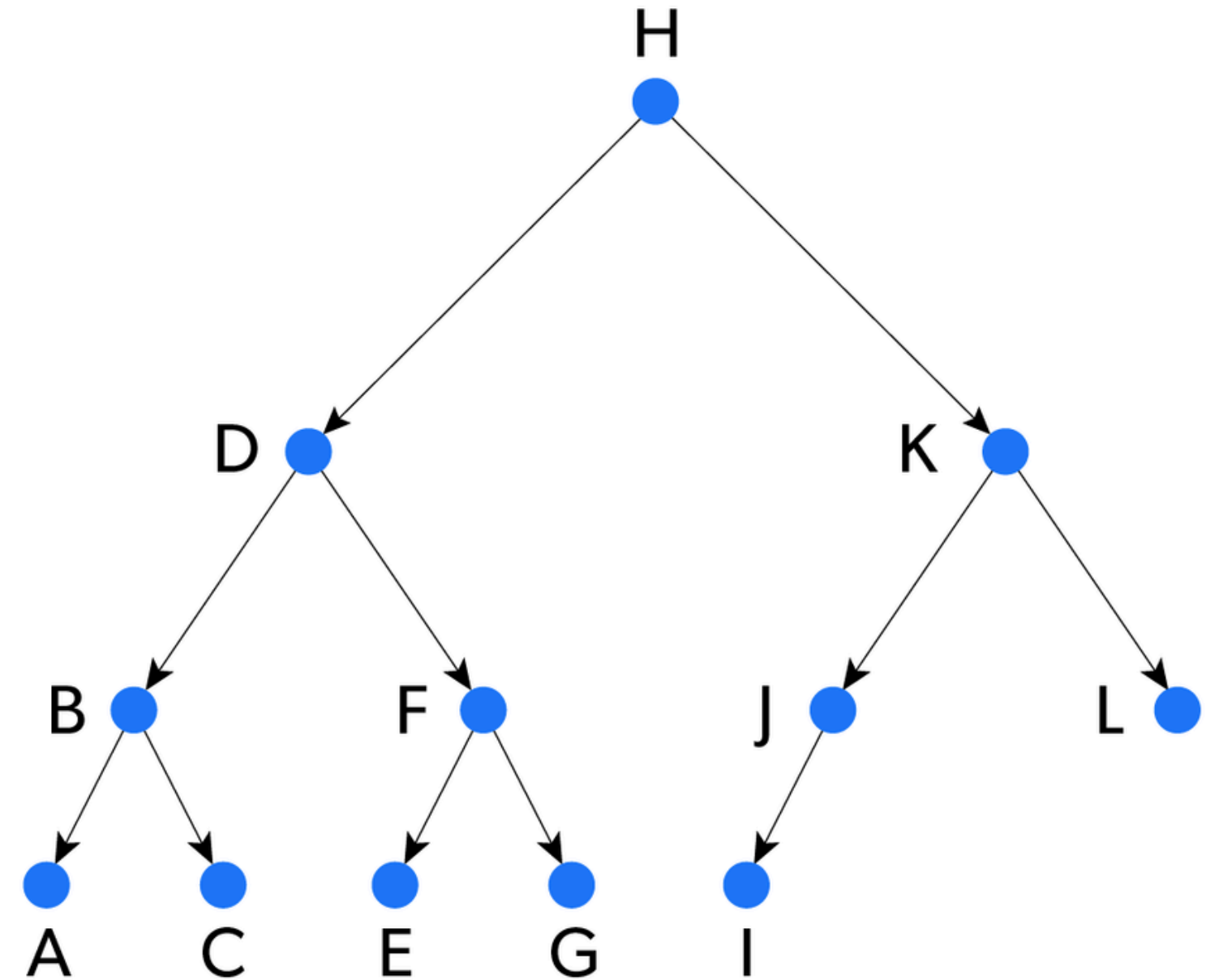
AULÃO DE AEDS 2



Tópicos

- Balanceamento de Árvore
- Árvore AVL
- Árvore 2-3-4
- Árvore Alvinegra (Rubronegra)
- Doidona (Híbrida) e Hash

Balanceamento Árvores



Balanceamento Árvores

Qual é a vantagem de uma árvore balanceada?

Balanceamento Árvores

Qual é a vantagem de uma árvore balanceada?

**Resposta: eficiência em termos de
pesquisa, inserção e remoção**

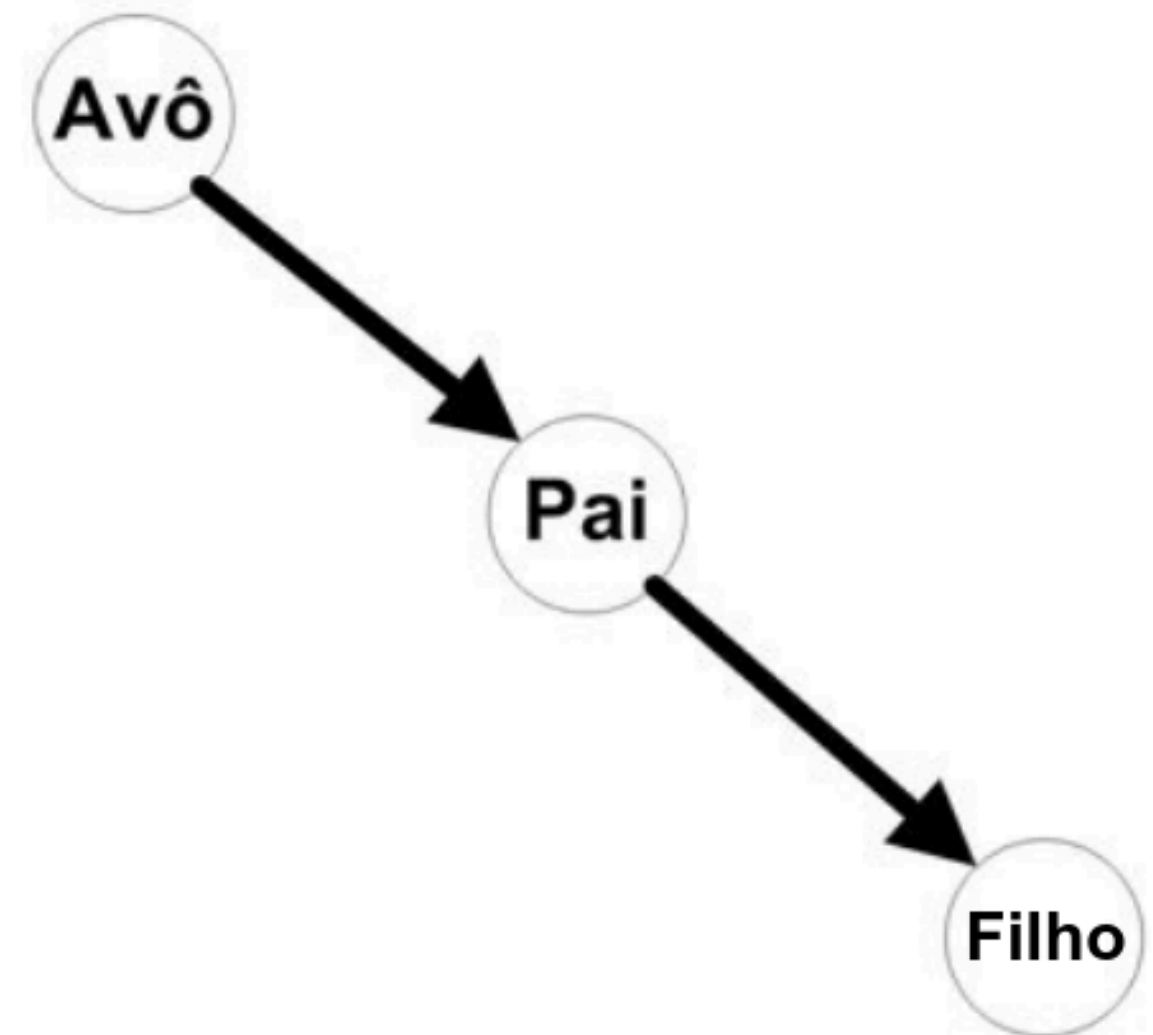
Balanceamento Árvores

Regras de balanceamento

**As árvores desbalanceadas para a esquerda
devem ser rotacionadas para
a direita e as para a direita, para a esquerda**

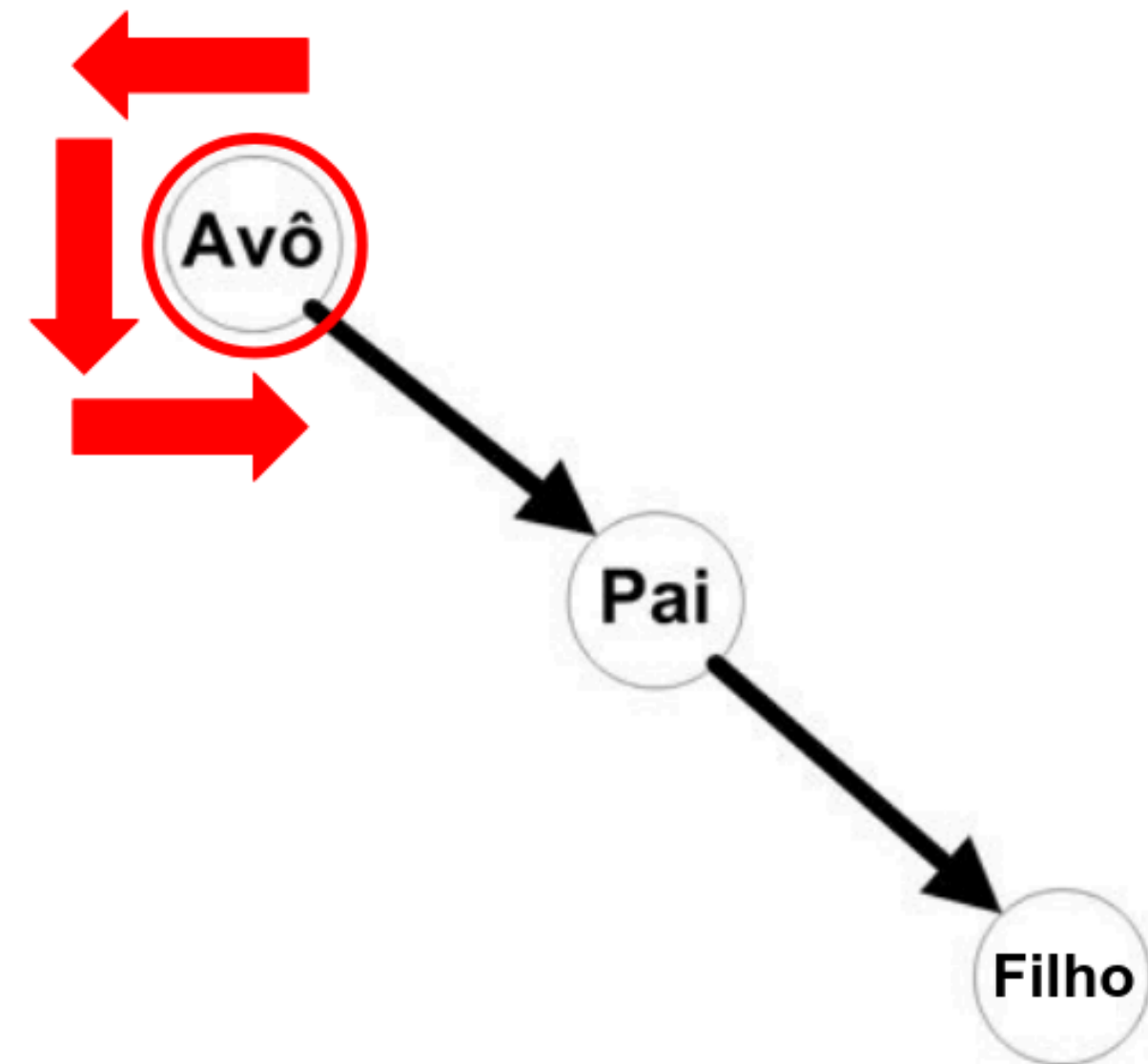
Balanceamento Árvores

Rotação Simples a Esquerda



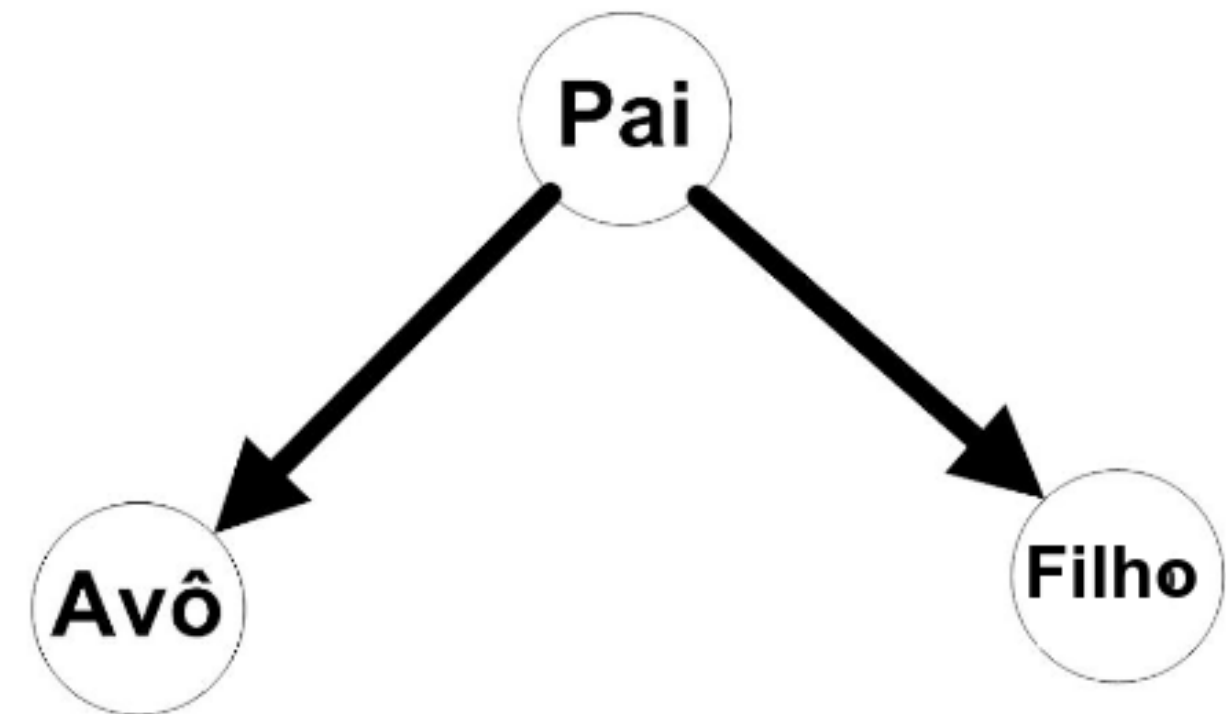
Balanceamento Árvores

Rotação Simples a Esquerda

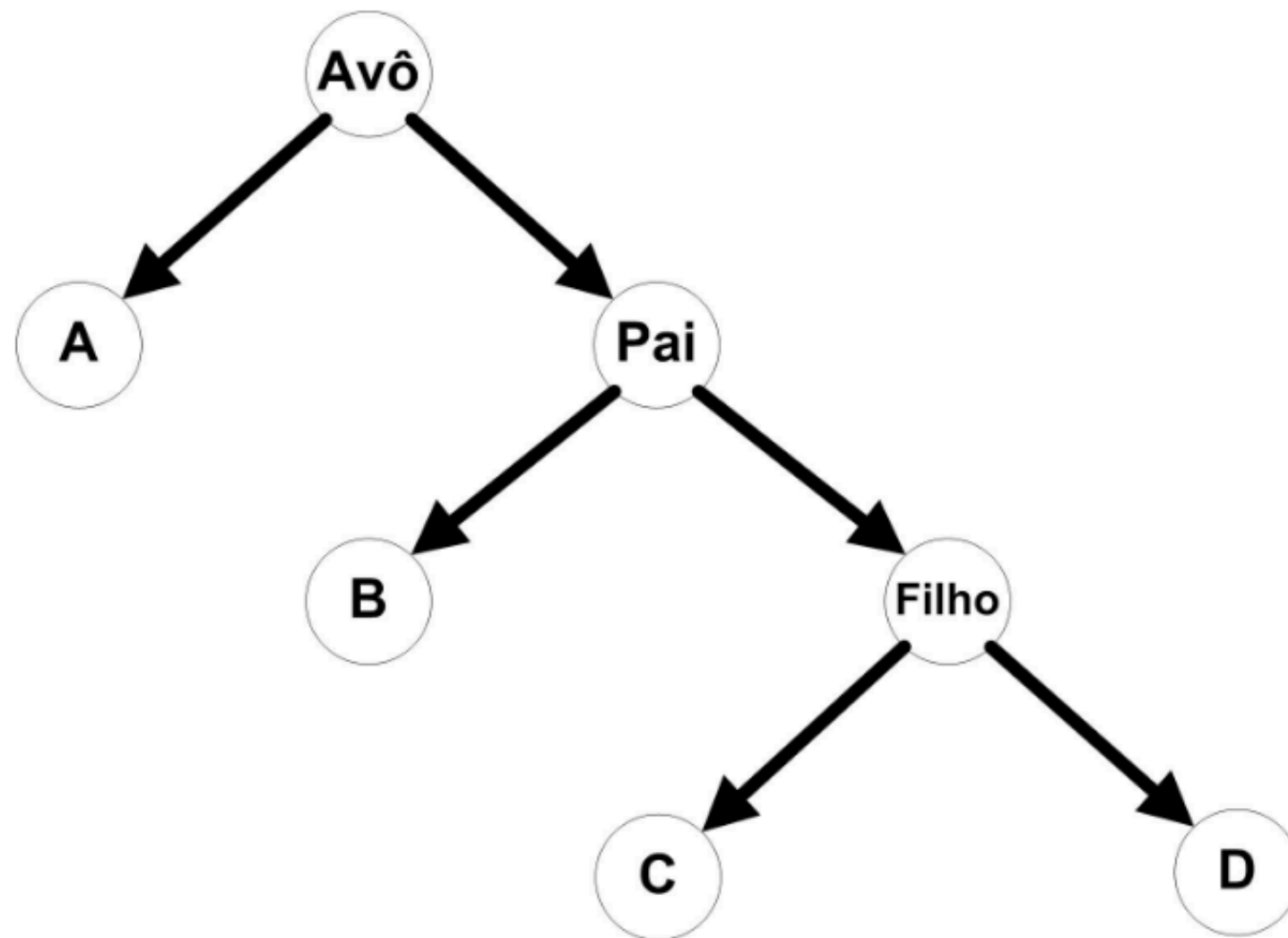


Balanceamento Árvores

Rotação Simples a Esquerda

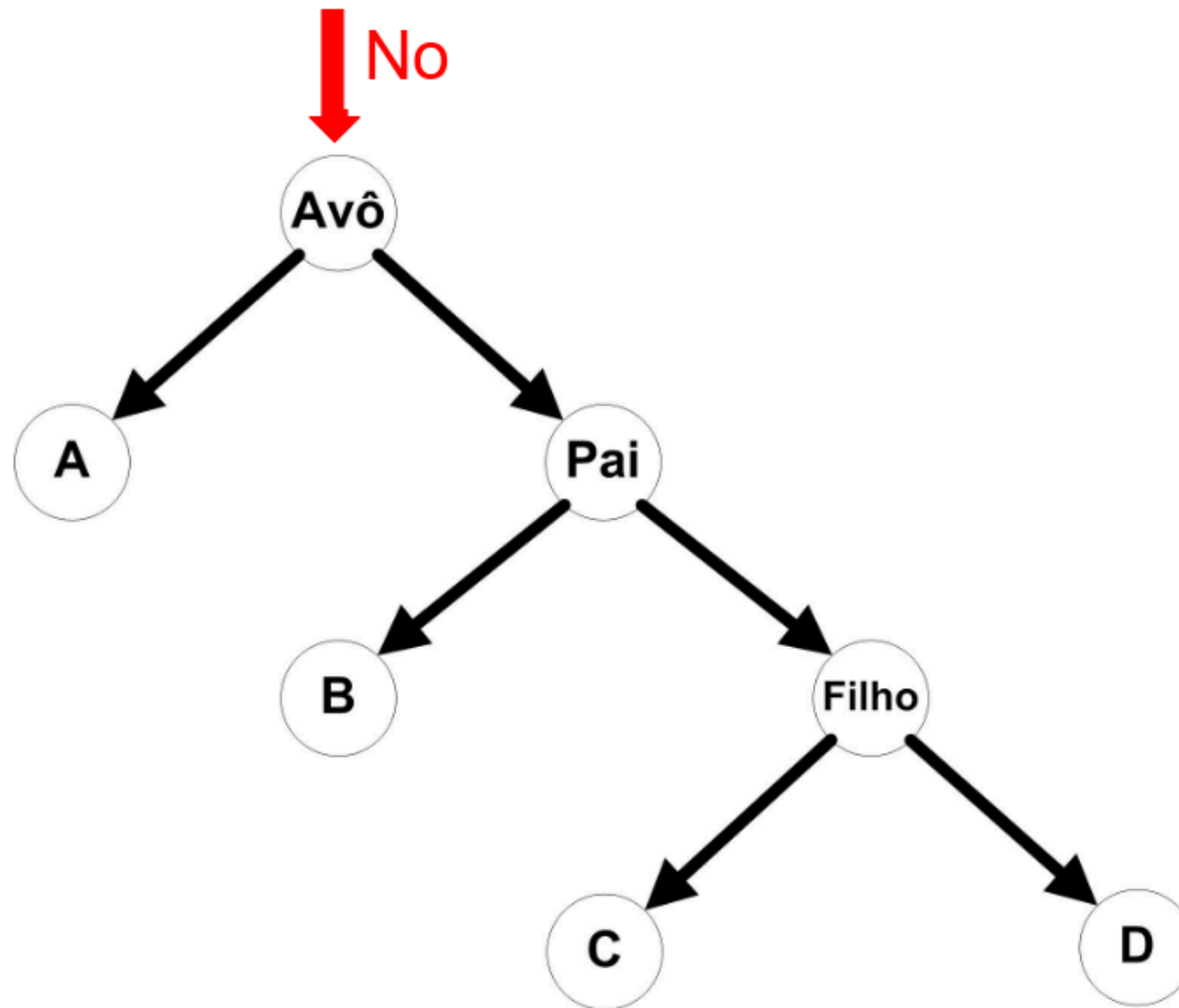


Hora do Código!



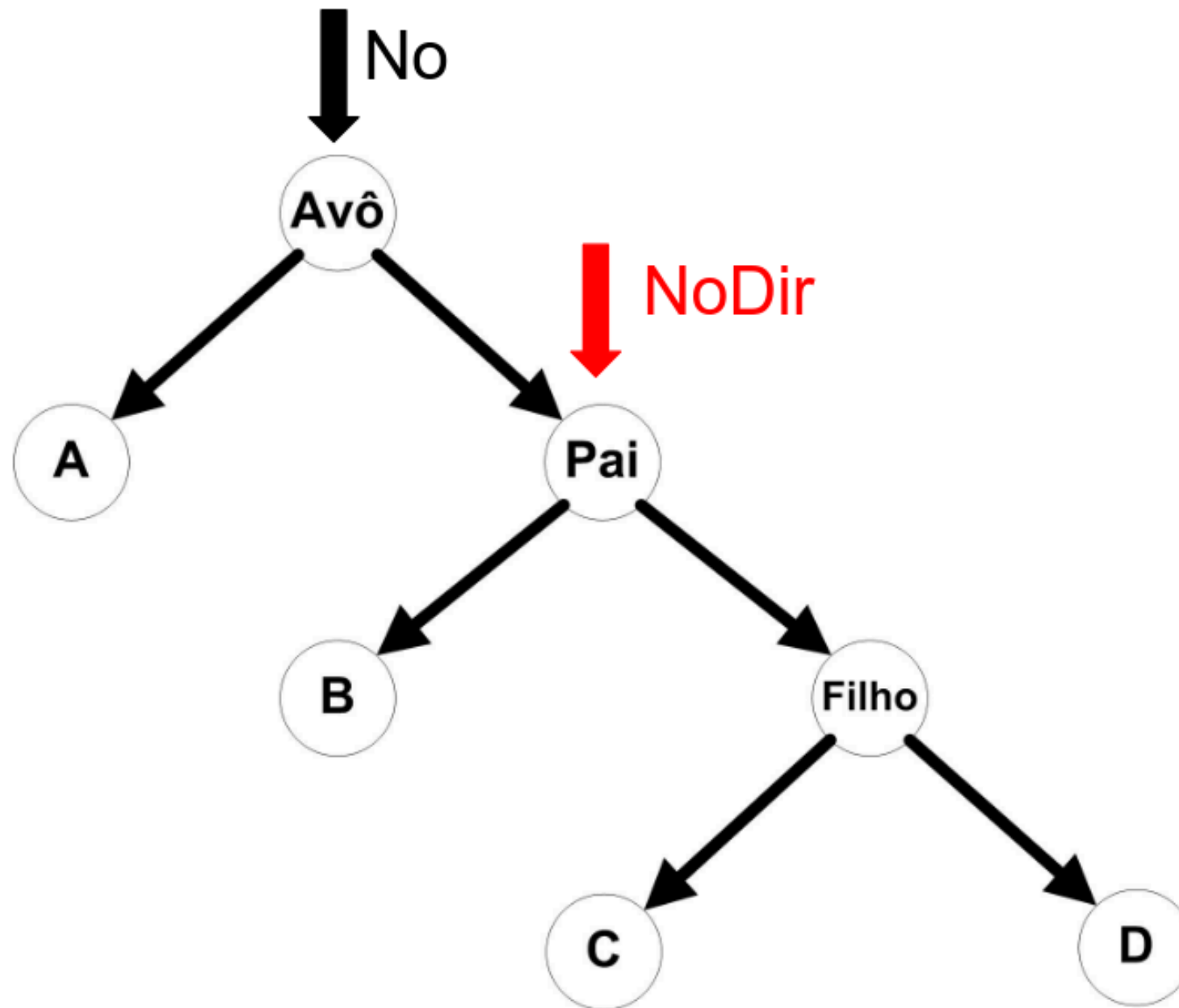
```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
No rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```


Hora do Código!



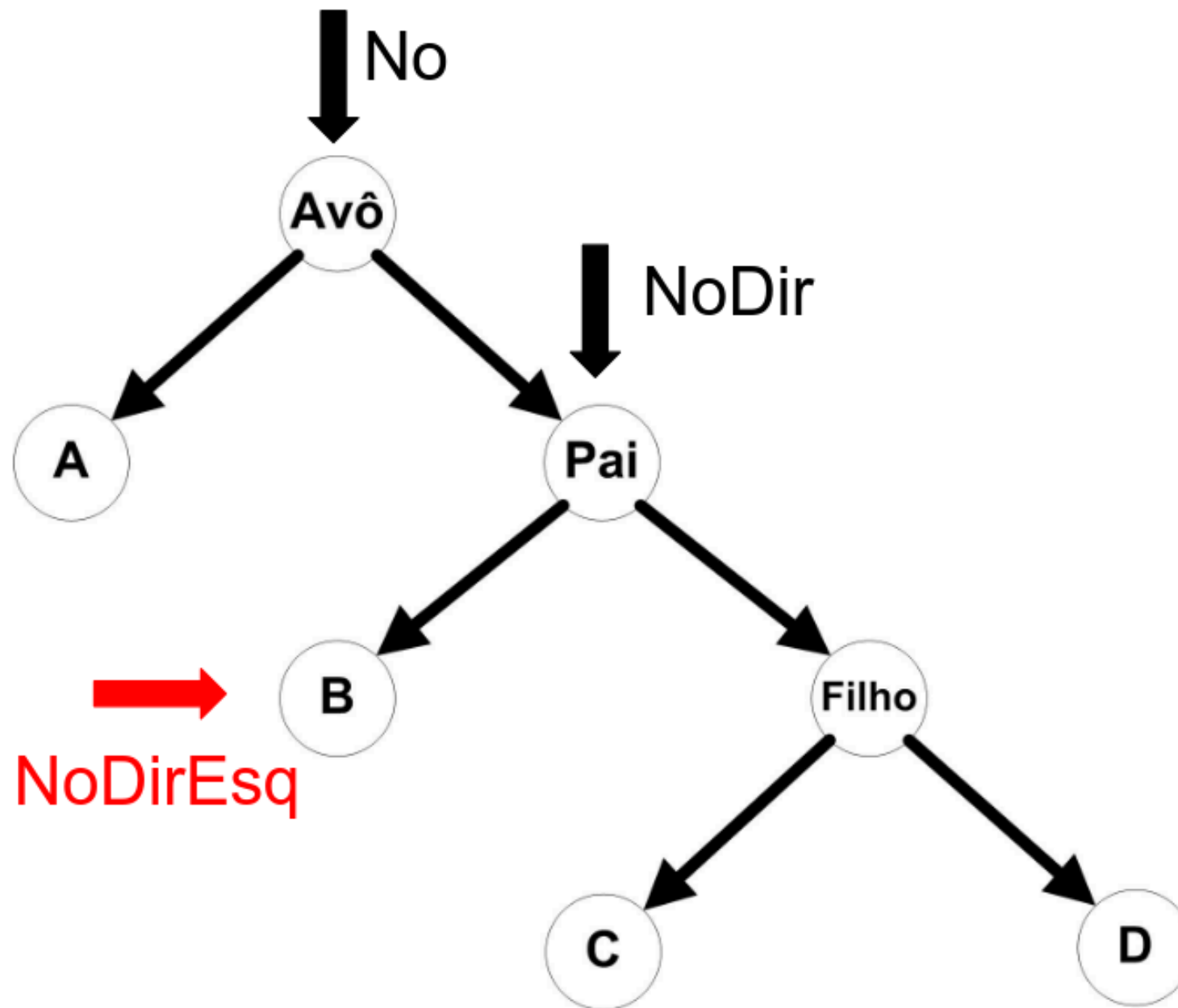
```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
No rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```

Hora do Código!



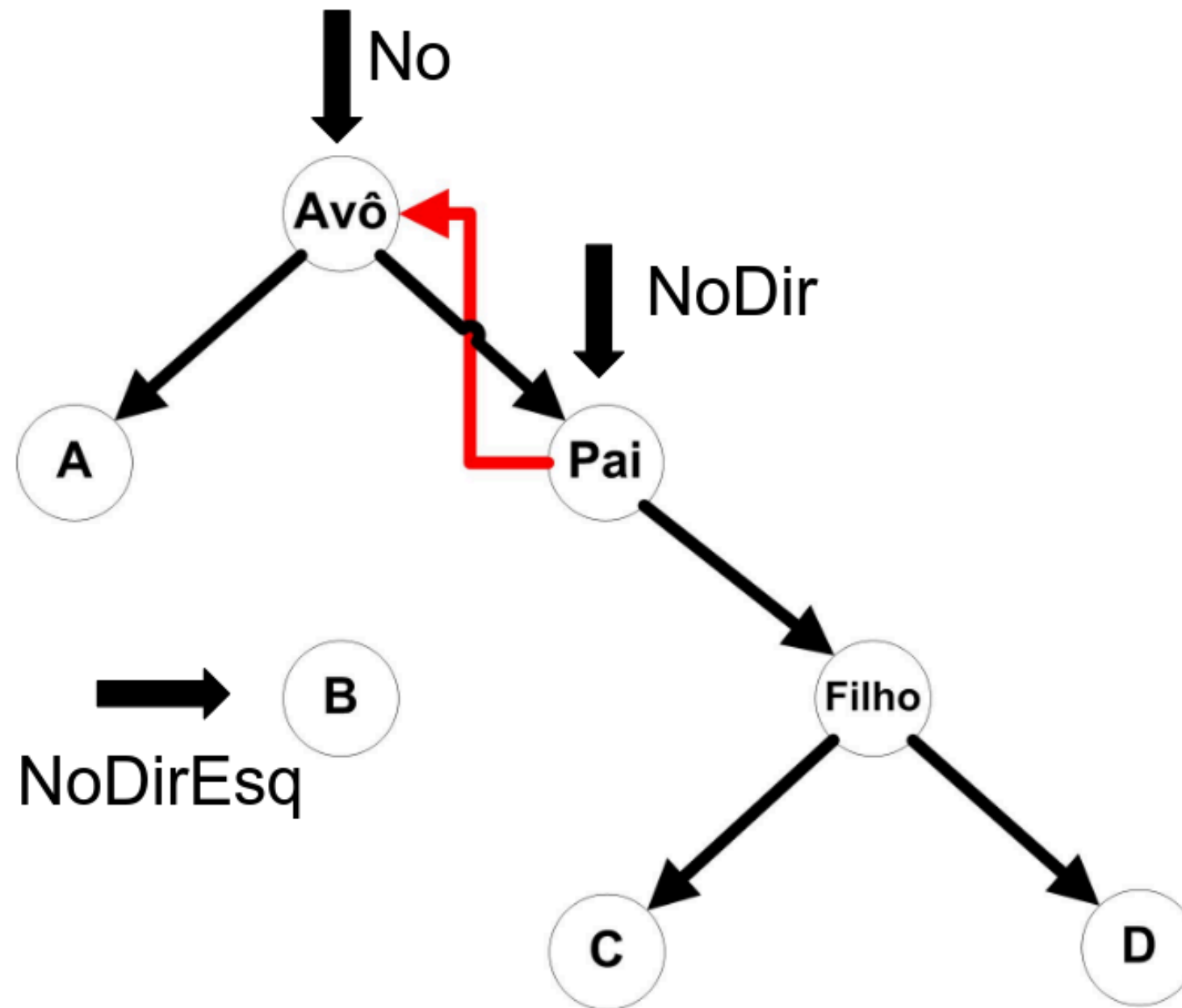
```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
NoNo rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```

Hora do Código!



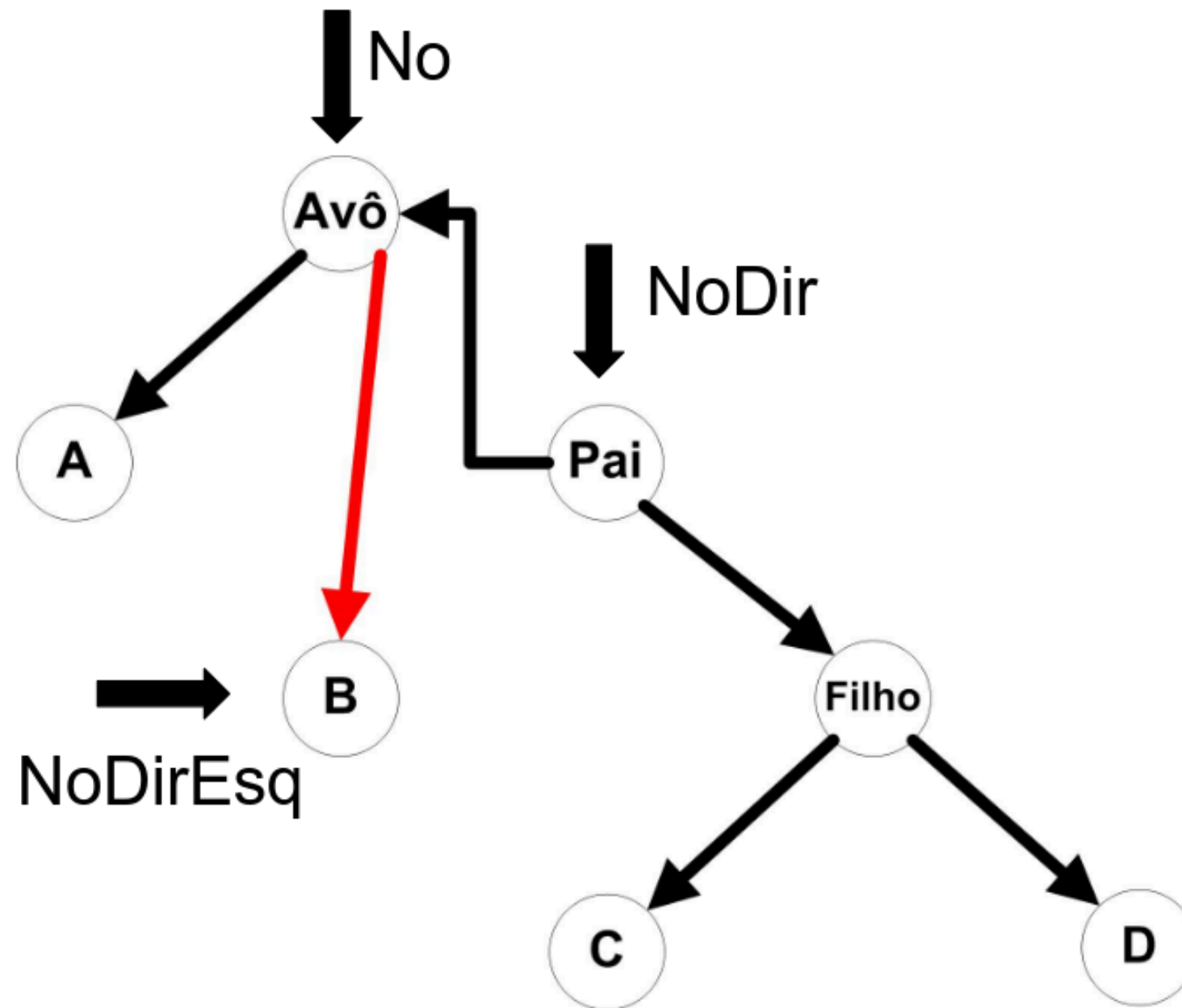
```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
NoNo rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```

Hora do Código!



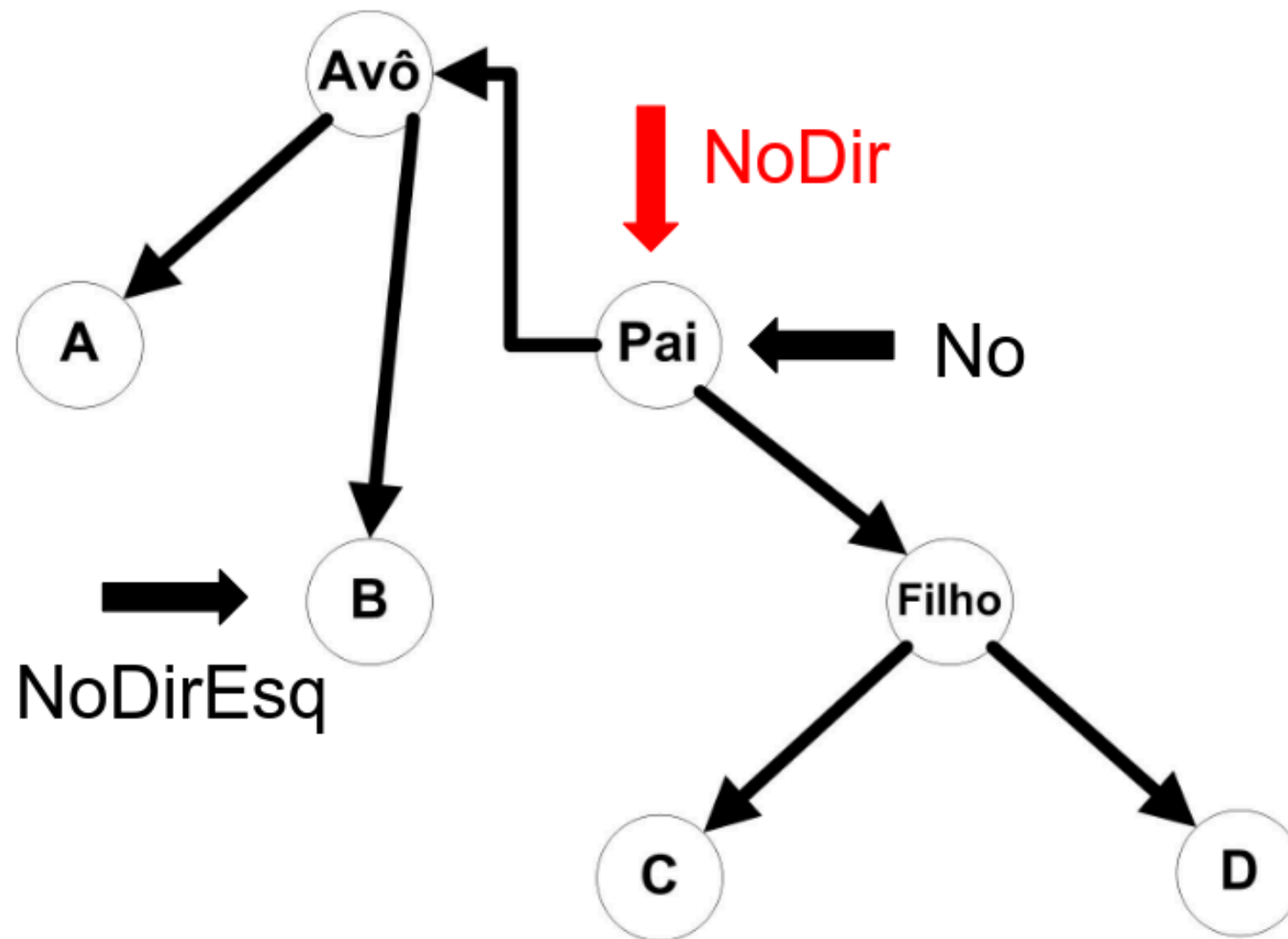
```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
No rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```

Hora do Código!



```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
No rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```

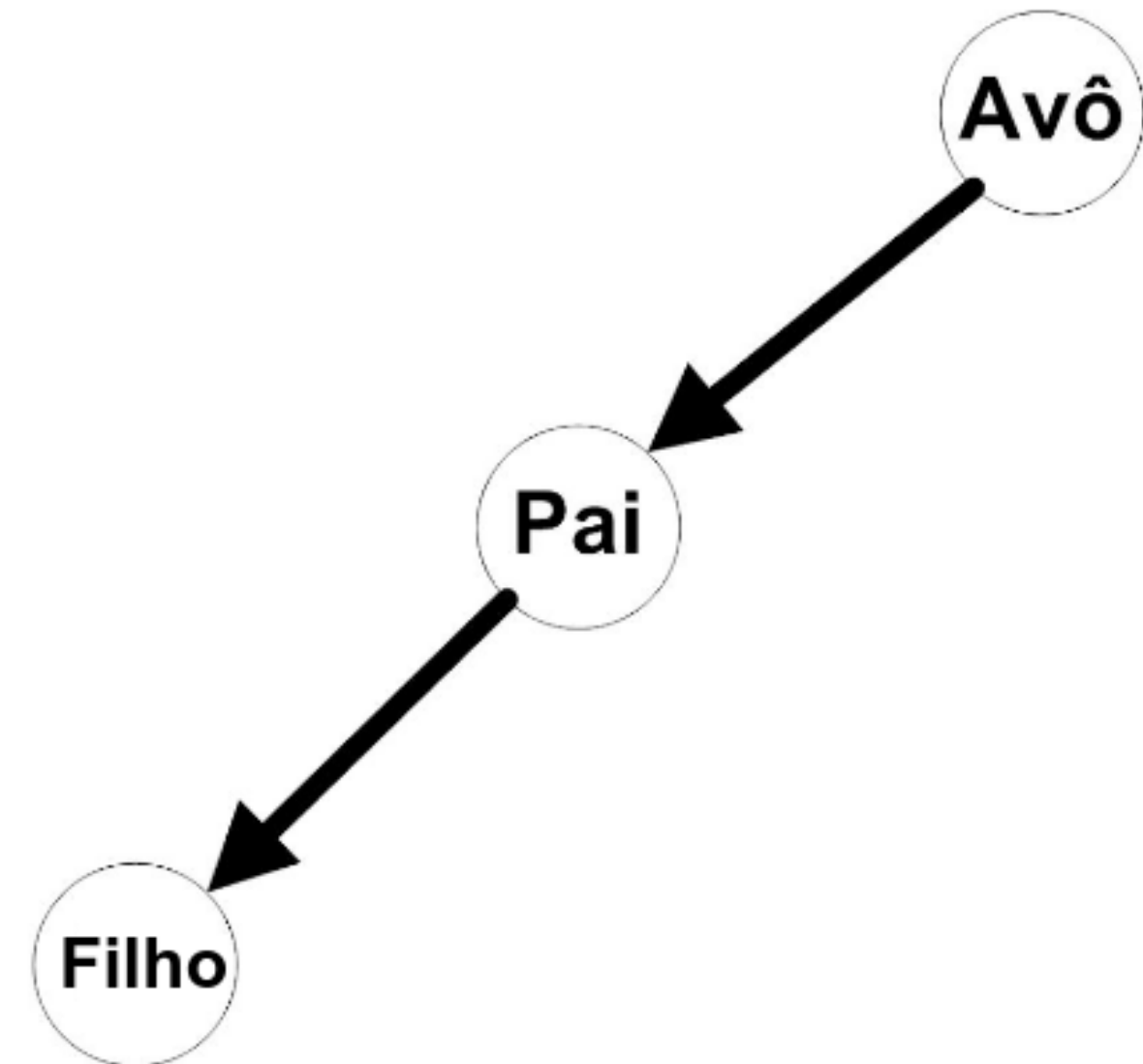

Hora do Código!



```
void metodo(){  
    ...  
    no = rotacionarEsq(no);  
    ...  
}  
No rotacionarEsq (No no) {  
    No noDir = no.dir;  
    No noDirEsq = noDir.esq;  
  
    noDir.esq = no;  
    no.dir = noDirEsq;  
  
    return noDir;  
}
```

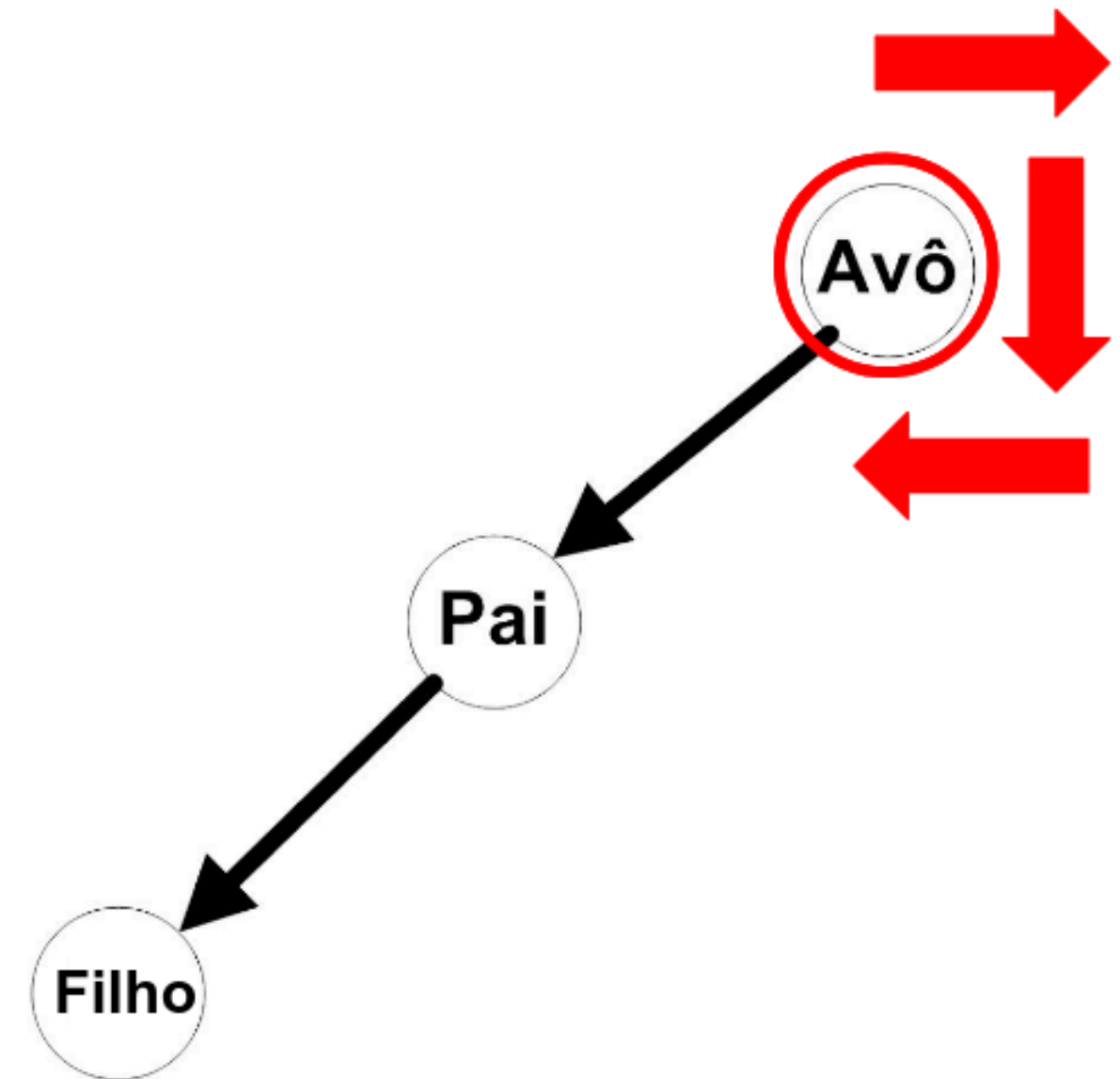
Balanceamento Árvores

Rotação Simples a Direita



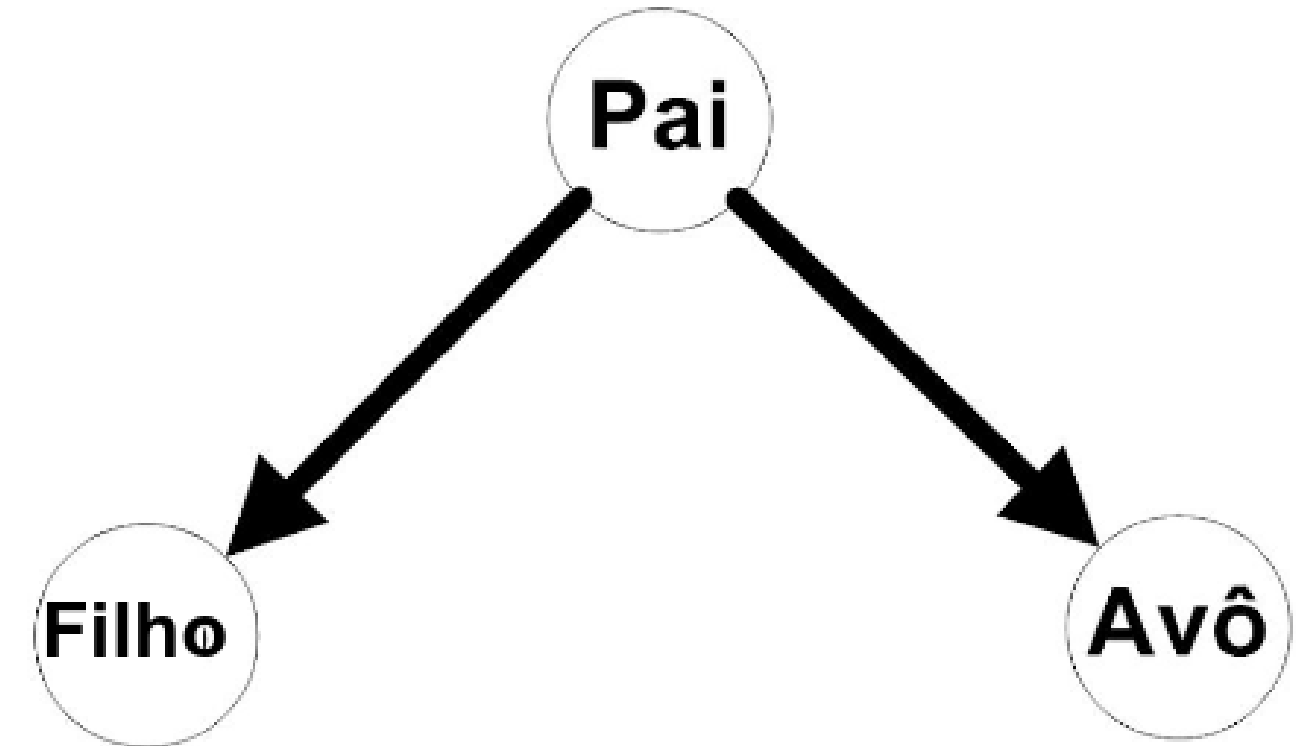
Balanceamento Árvores

Rotação Simples a Direita

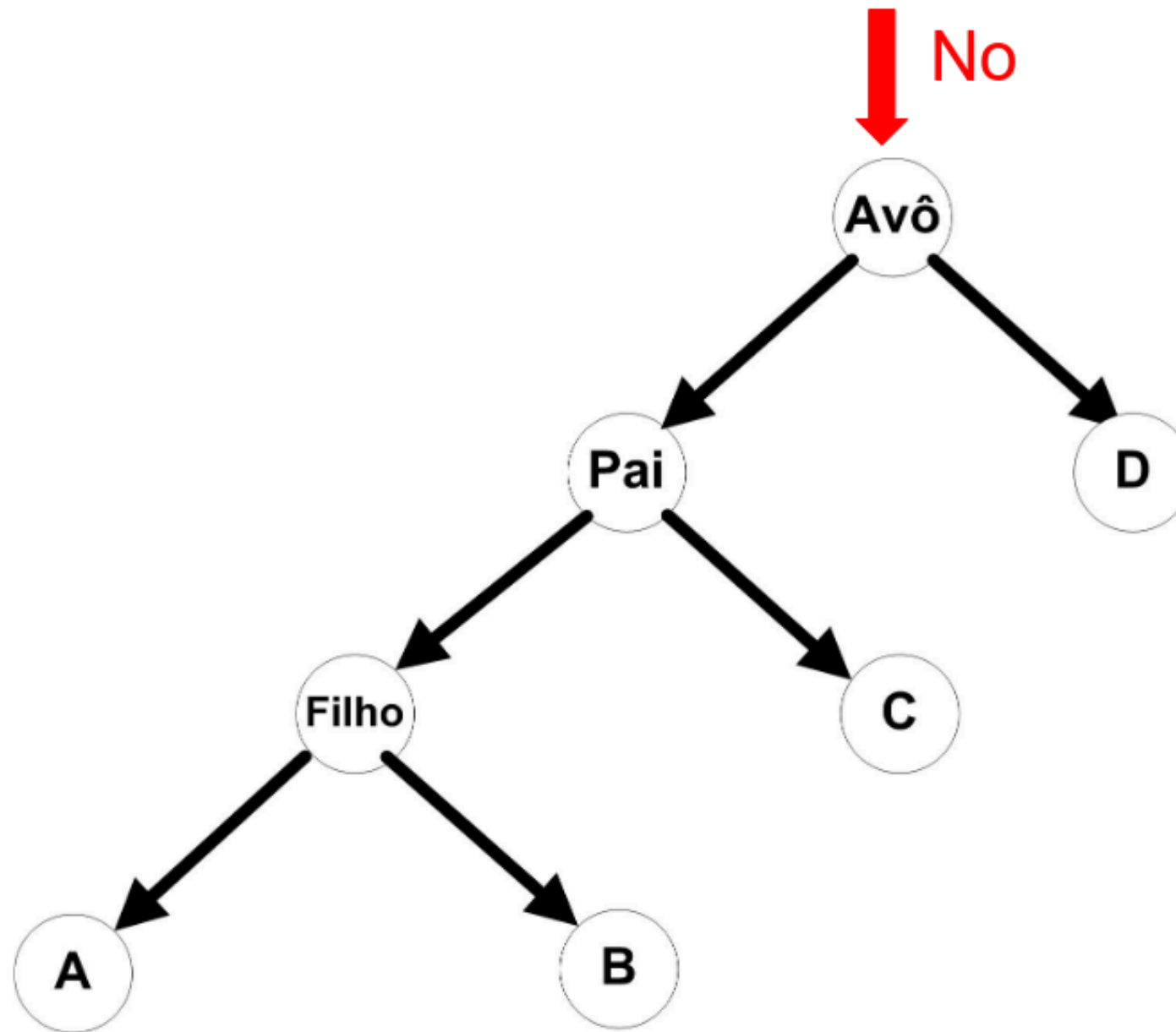


Balanceamento Árvores

Rotação Simples a Direita

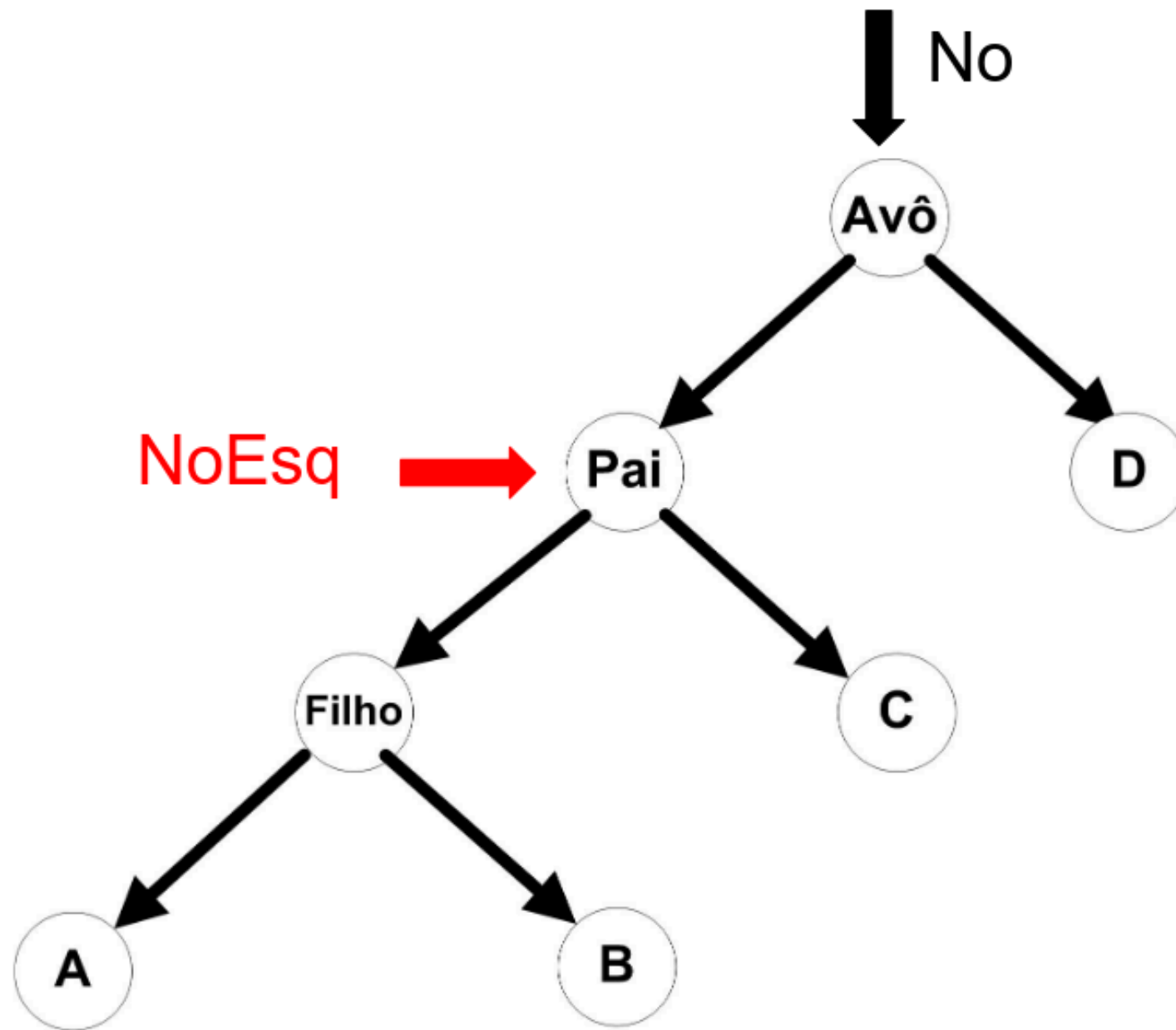


Hora do Código!



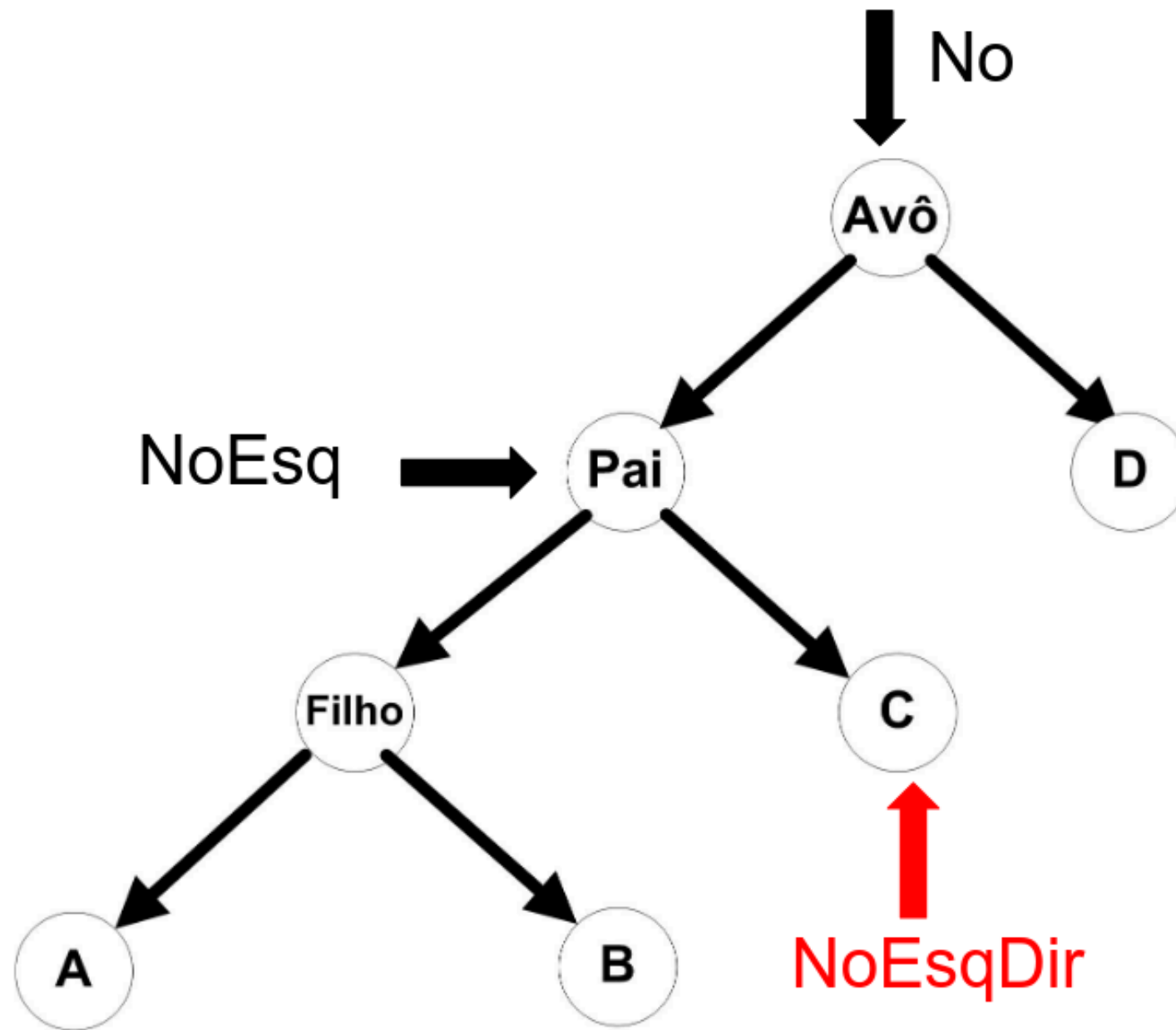
```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```

Hora do Código!



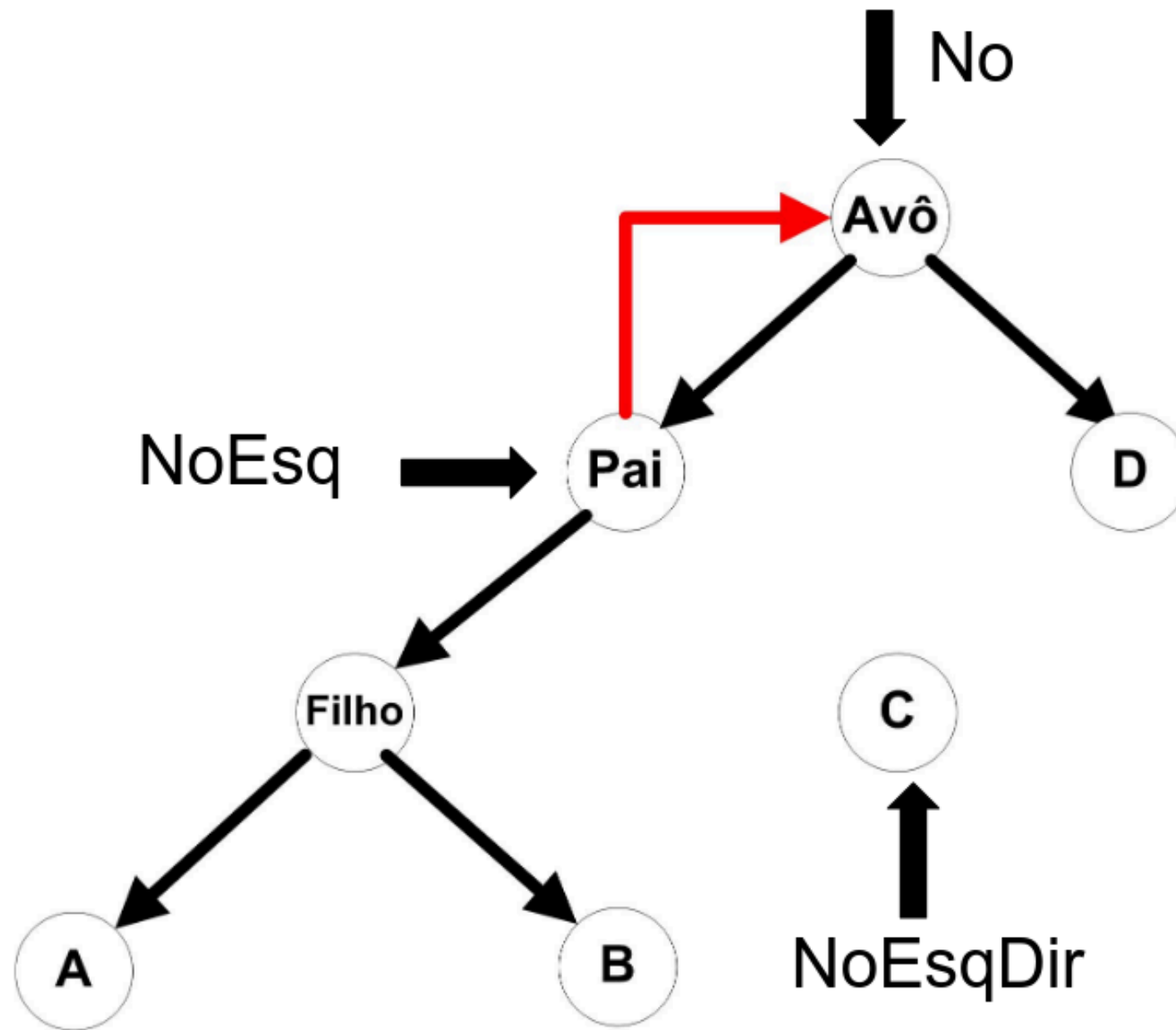
```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```

Hora do Código!



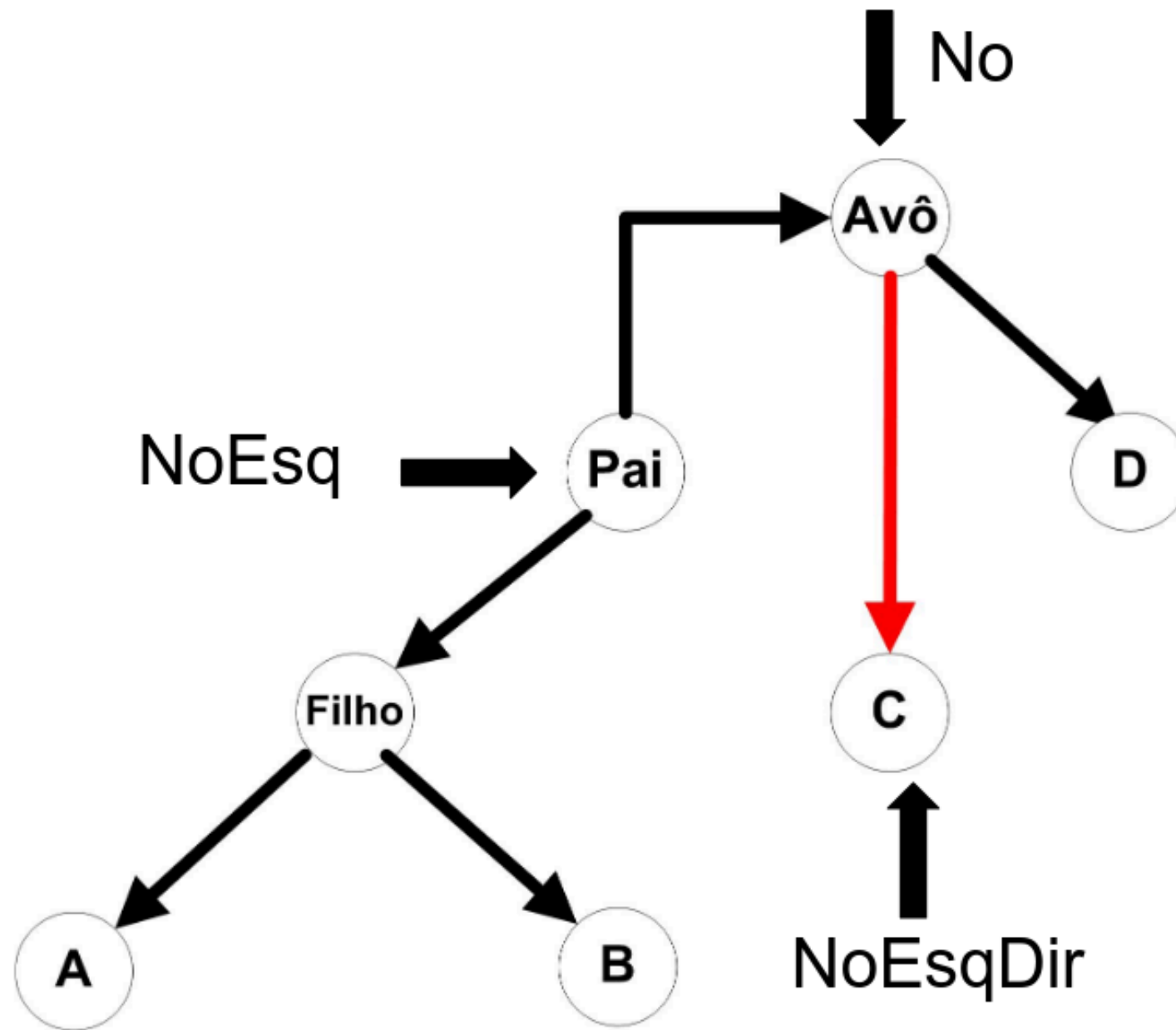
```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```

Hora do Código!



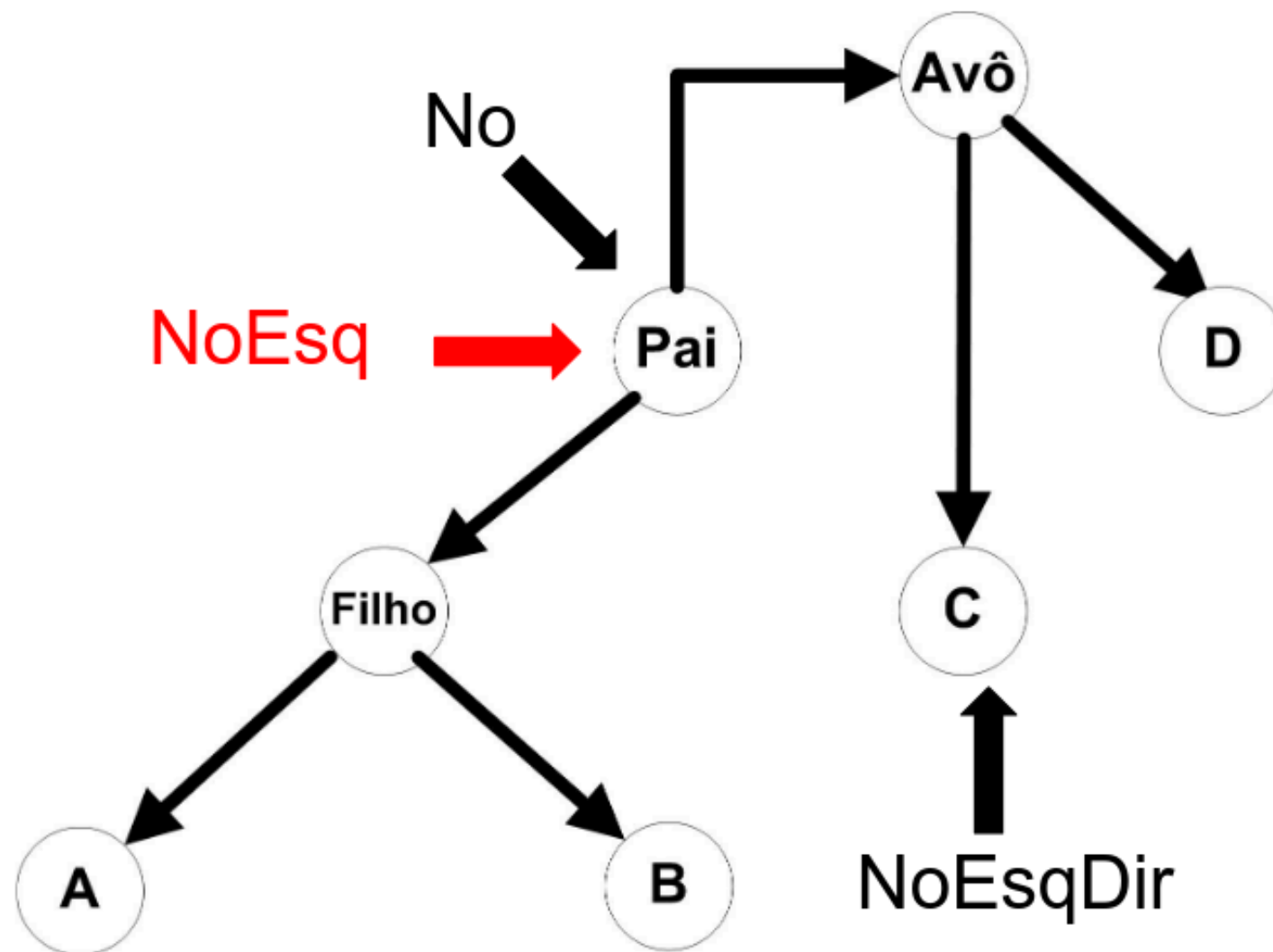
```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```


Hora do Código!



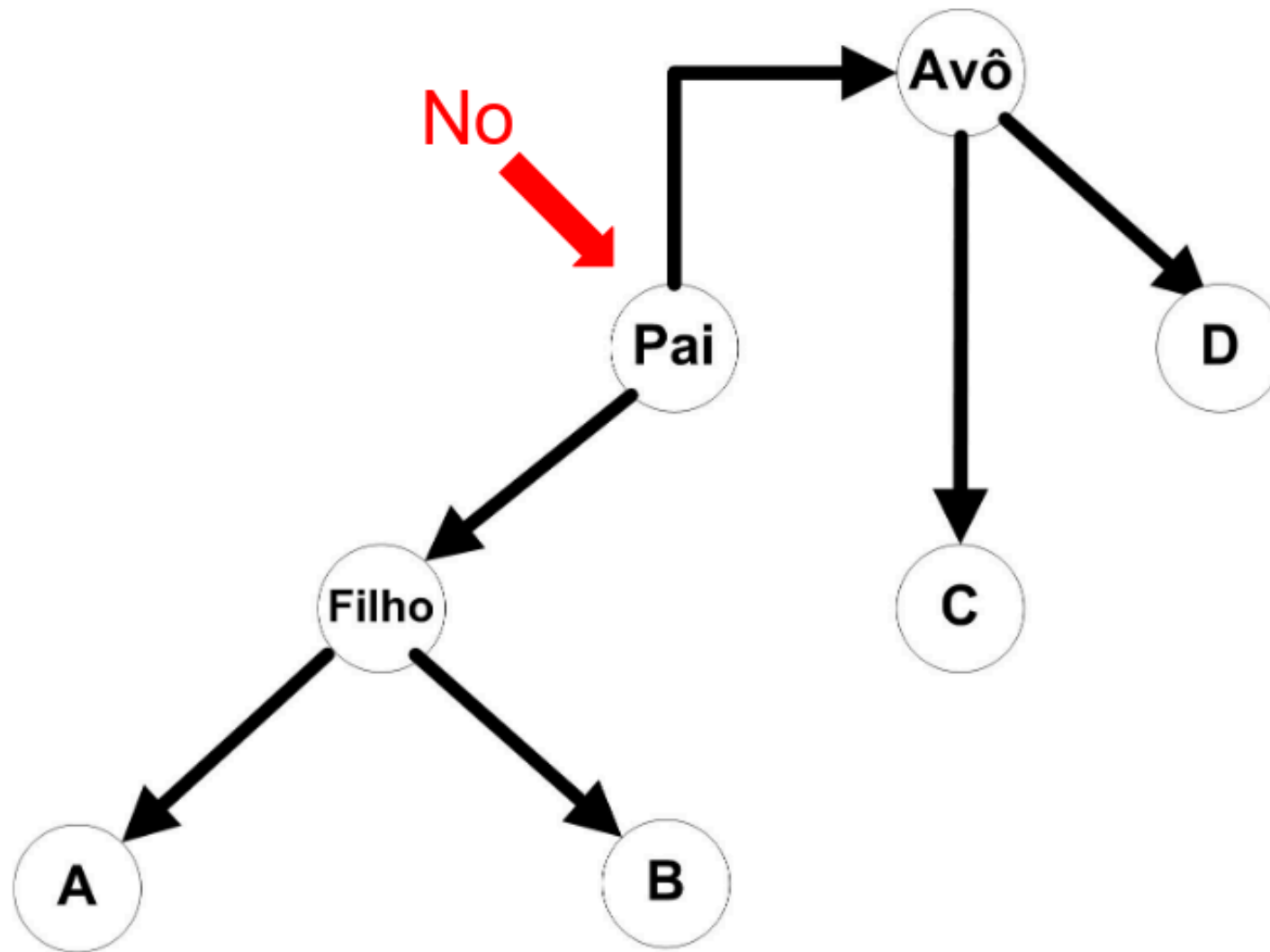
```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```

Hora do Código!



```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```

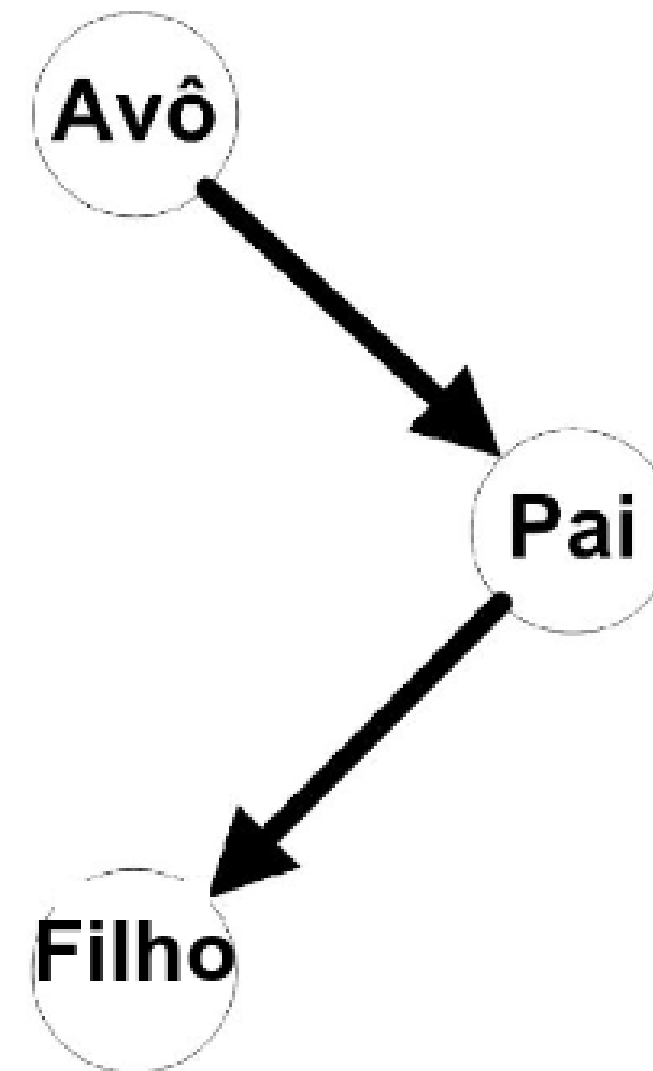
Hora do Código!



```
void metodo(){  
    ...  
    no = rotacionarDir(no);  
    ...  
}  
No rotacionarDir (No no) {  
    No noEsq = no.esq;  
    No noEsqDir = noEsq.dir;  
  
    noEsq.dir = no;  
    no.esq = noEsqDir;  
  
    return noEsq;  
}
```

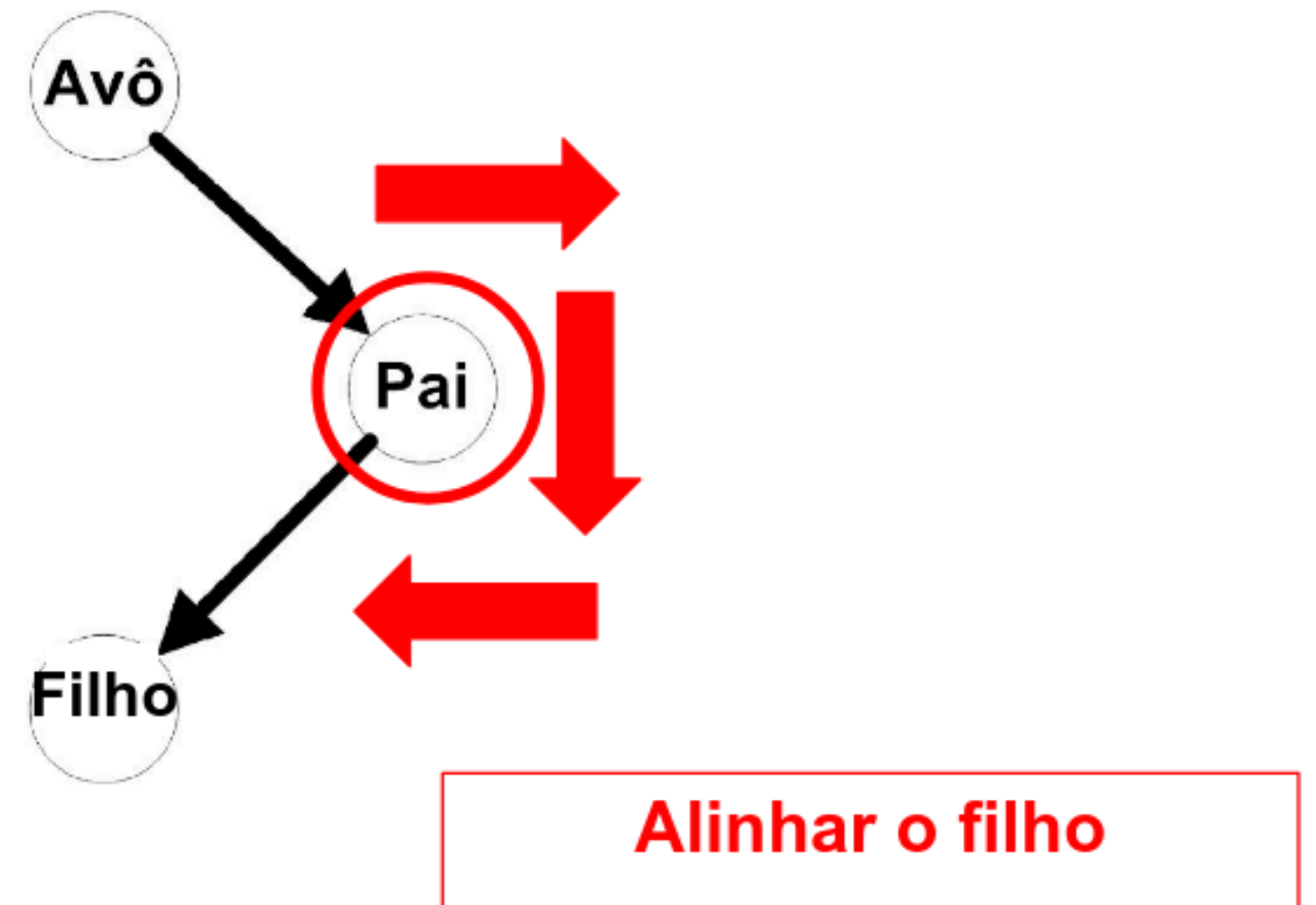

Balanceamento Árvores

Rotação Dupla Direita - Esquerda



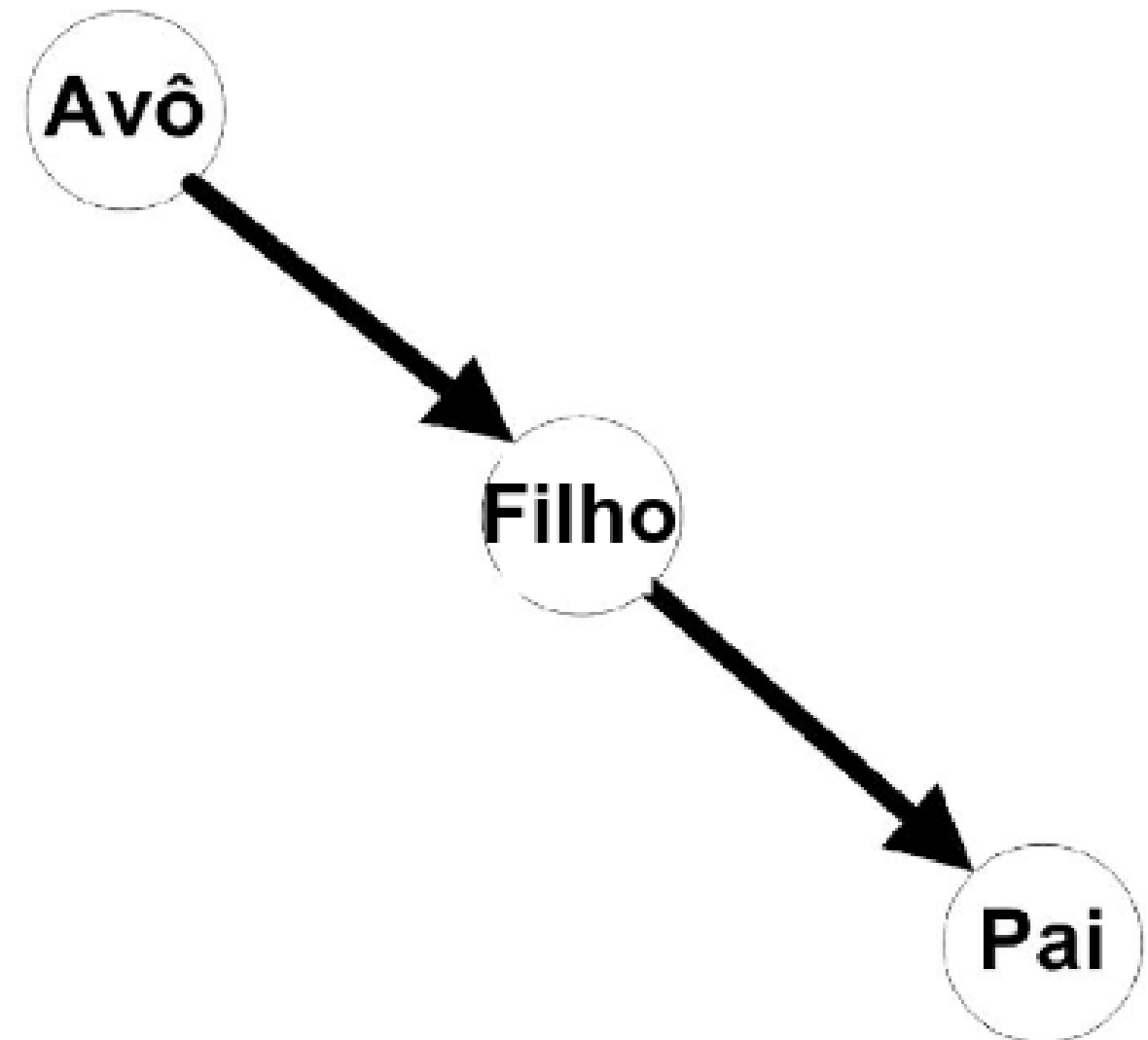
Balanceamento Árvores

Rotação Dupla Direita - Esquerda



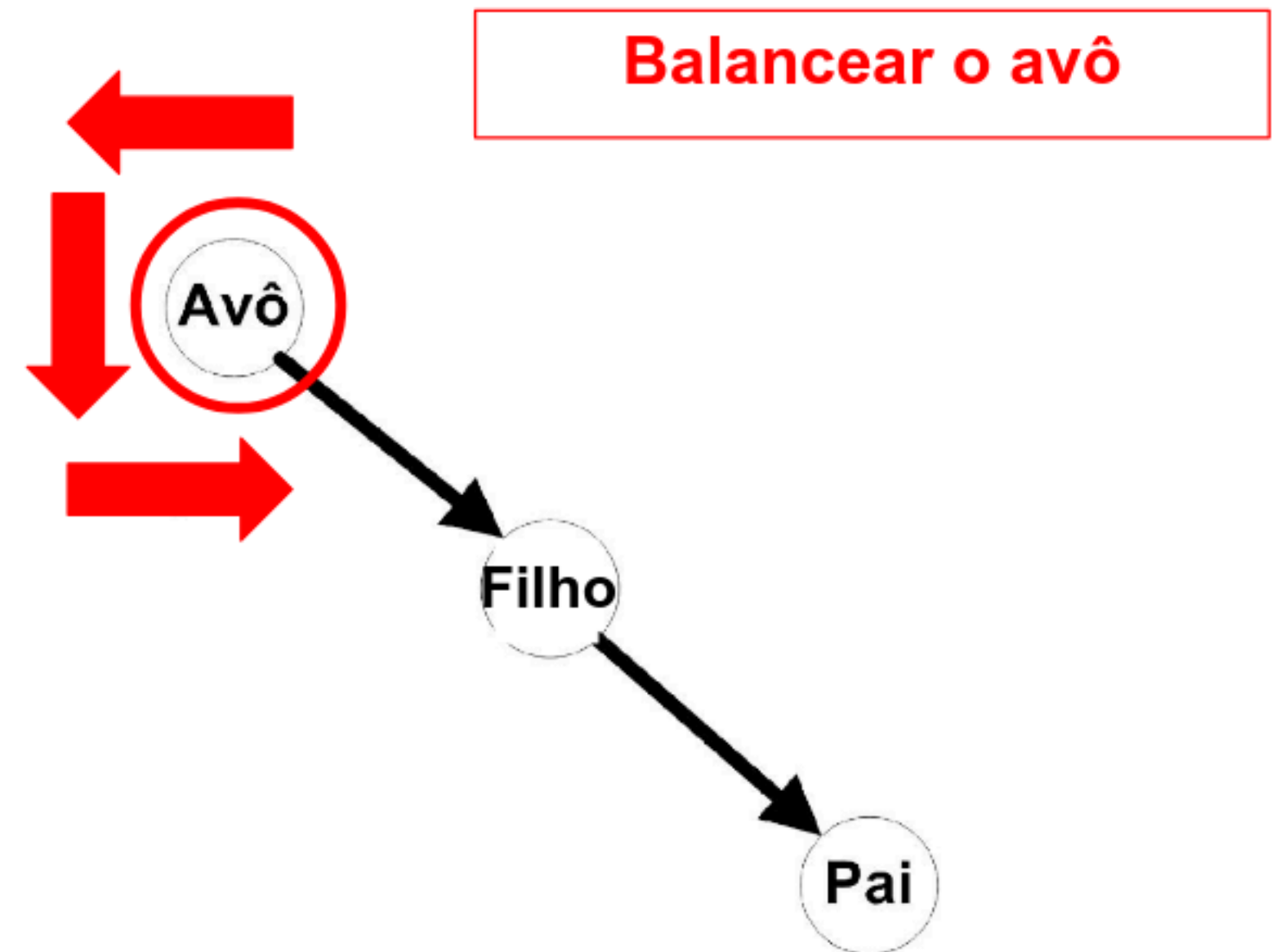
Balanceamento Árvores

Rotação Dupla Direita - Esquerda



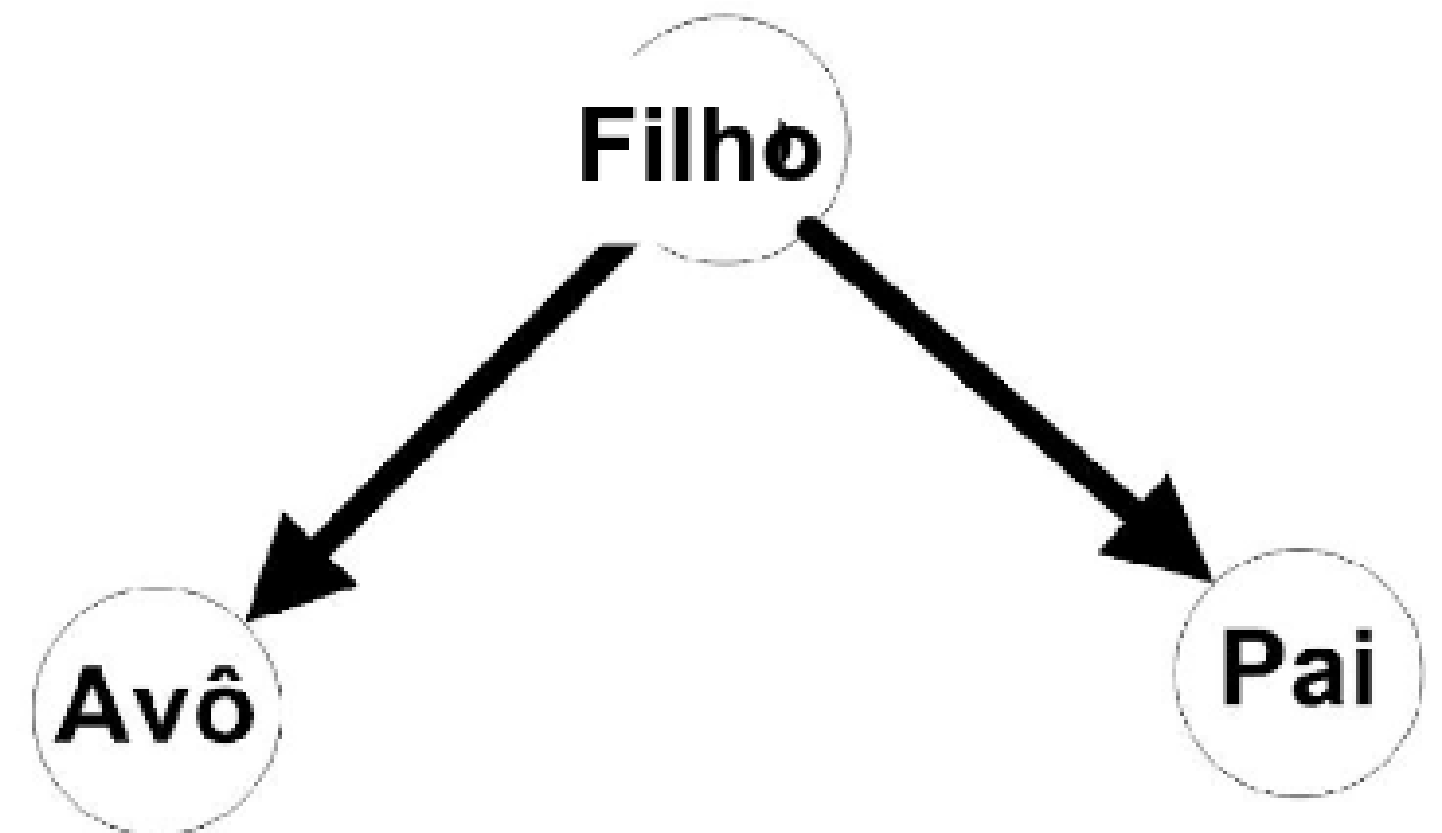
Balanceamento Árvores

Rotação Dupla Direita - Esquerda



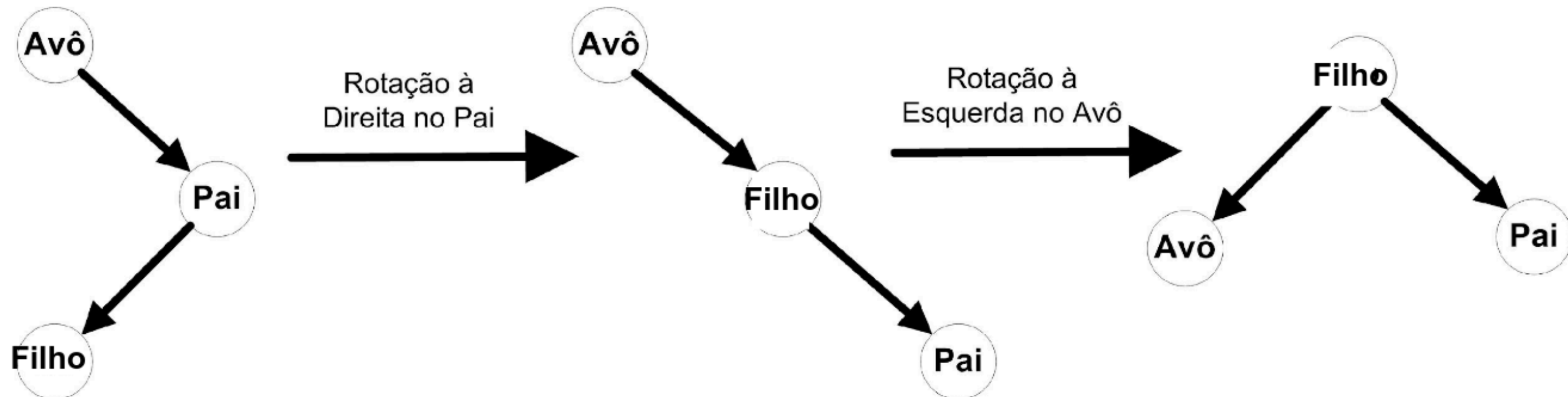
Balanceamento Árvores

Rotação Dupla Direita - Esquerda



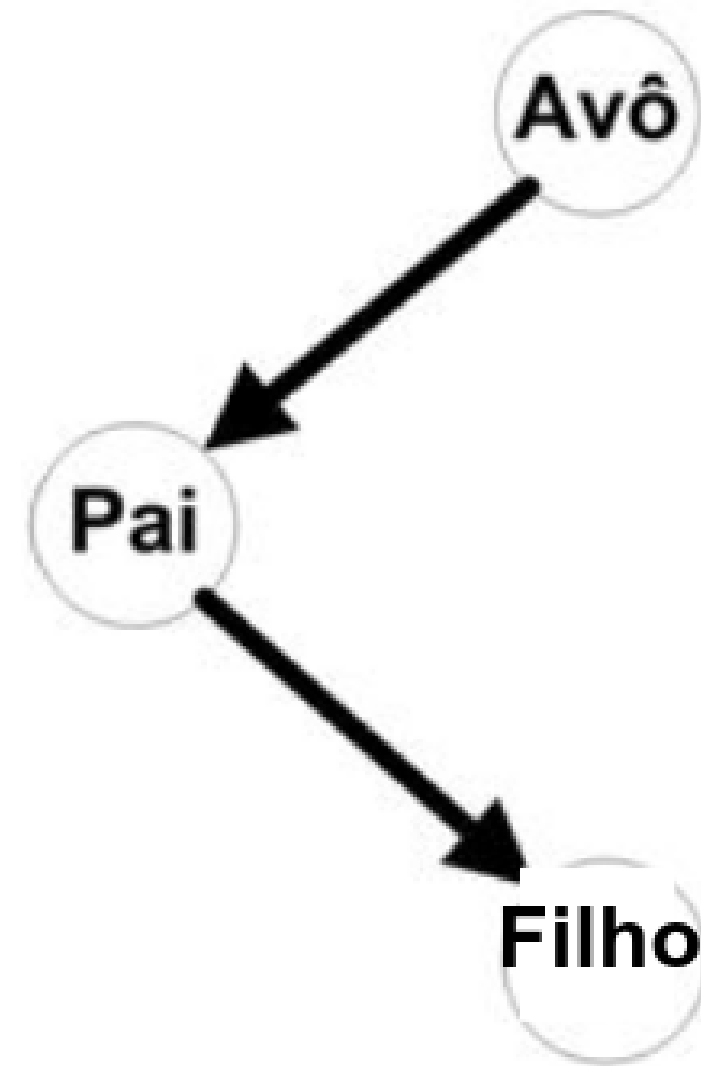
Hora do Código!

```
No rotacionarDirEsq(No no) {  
    no.dir = rotacionarDir(no.dir);  
    return rotacionarEsq(no);  
}
```



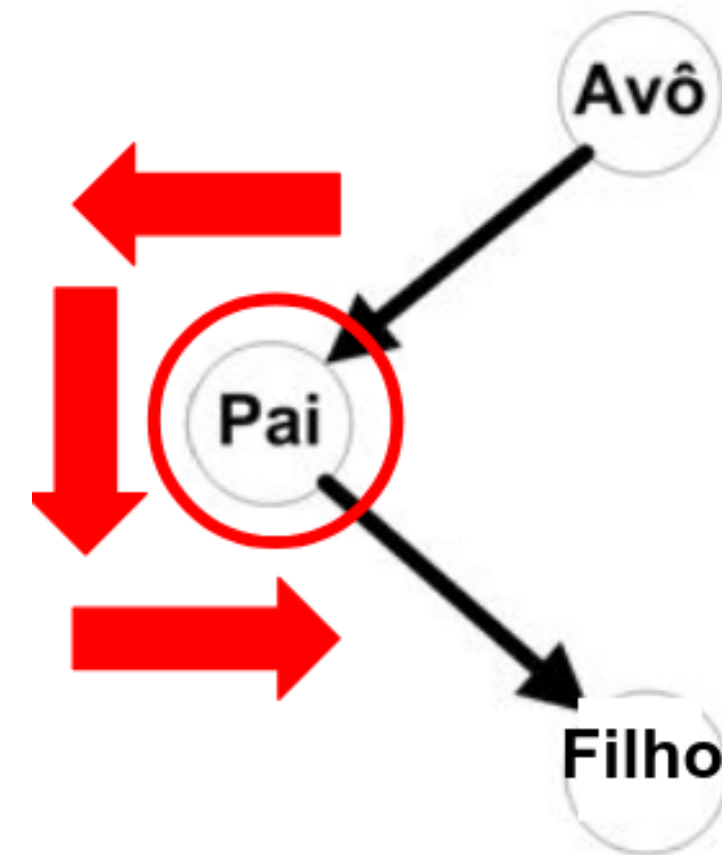
Balanceamento Árvores

Rotação Dupla Esquerda - Direita



Balanceamento Árvores

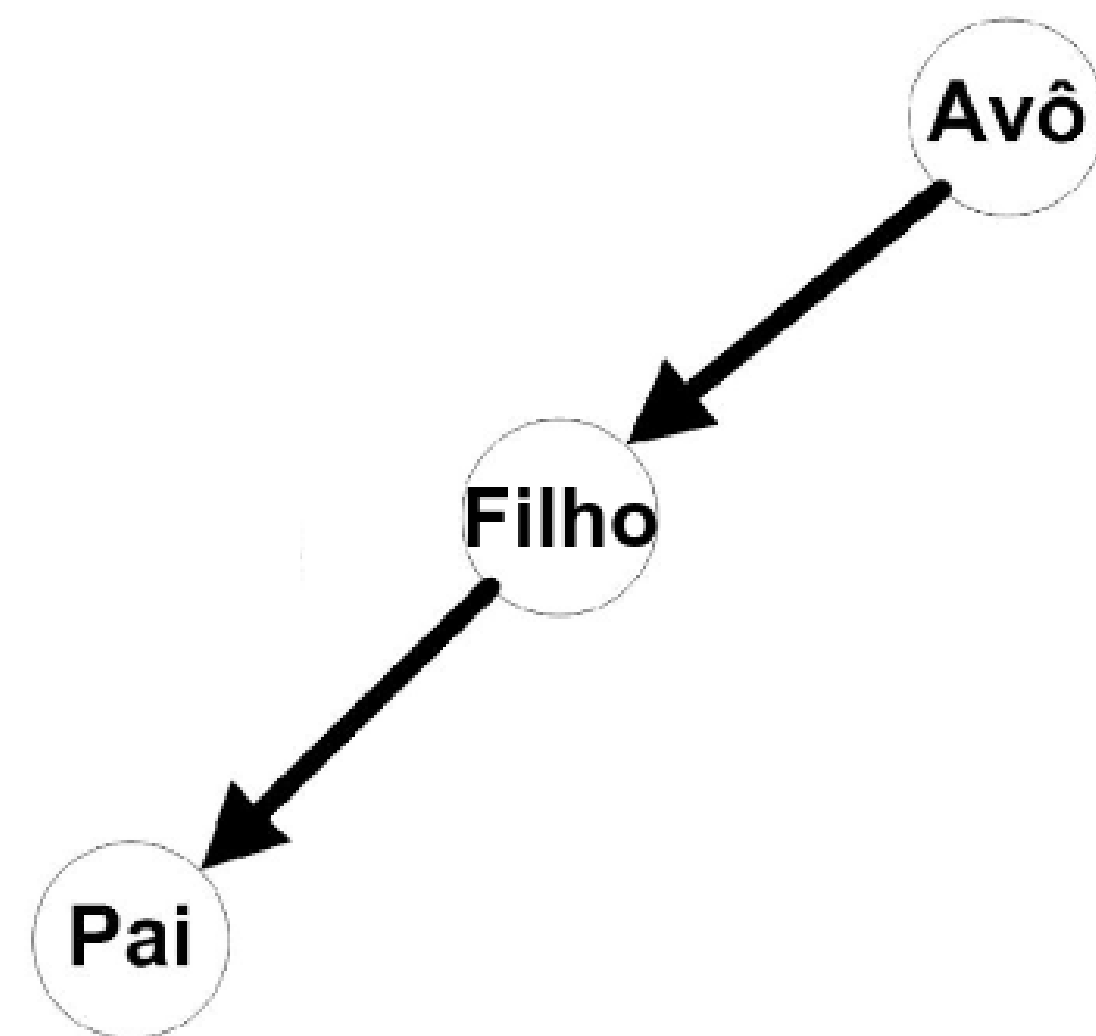
Rotação Dupla Esquerda - Direita



Alinhar o filho

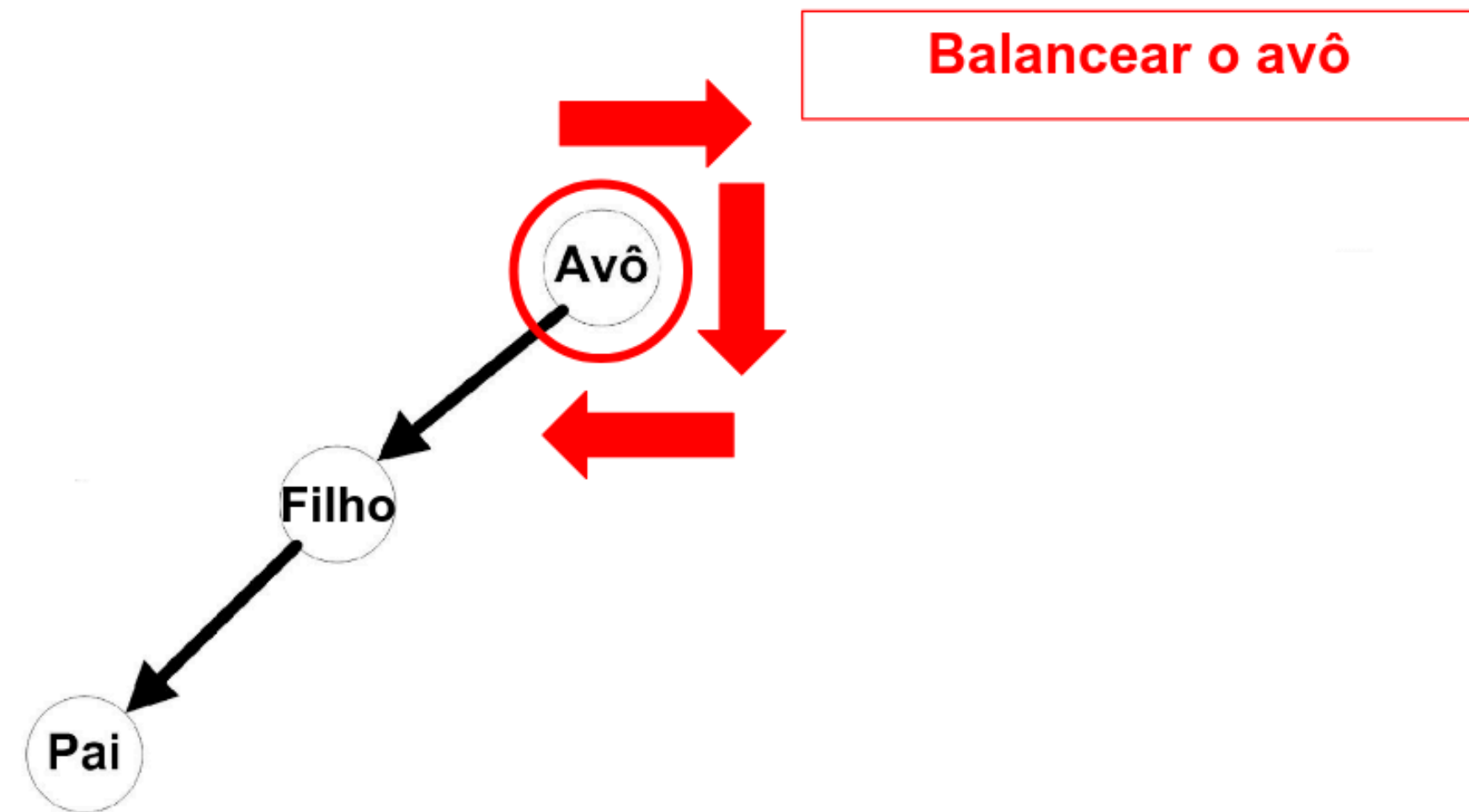
Balanceamento Árvores

Rotação Dupla Esquerda - Direita



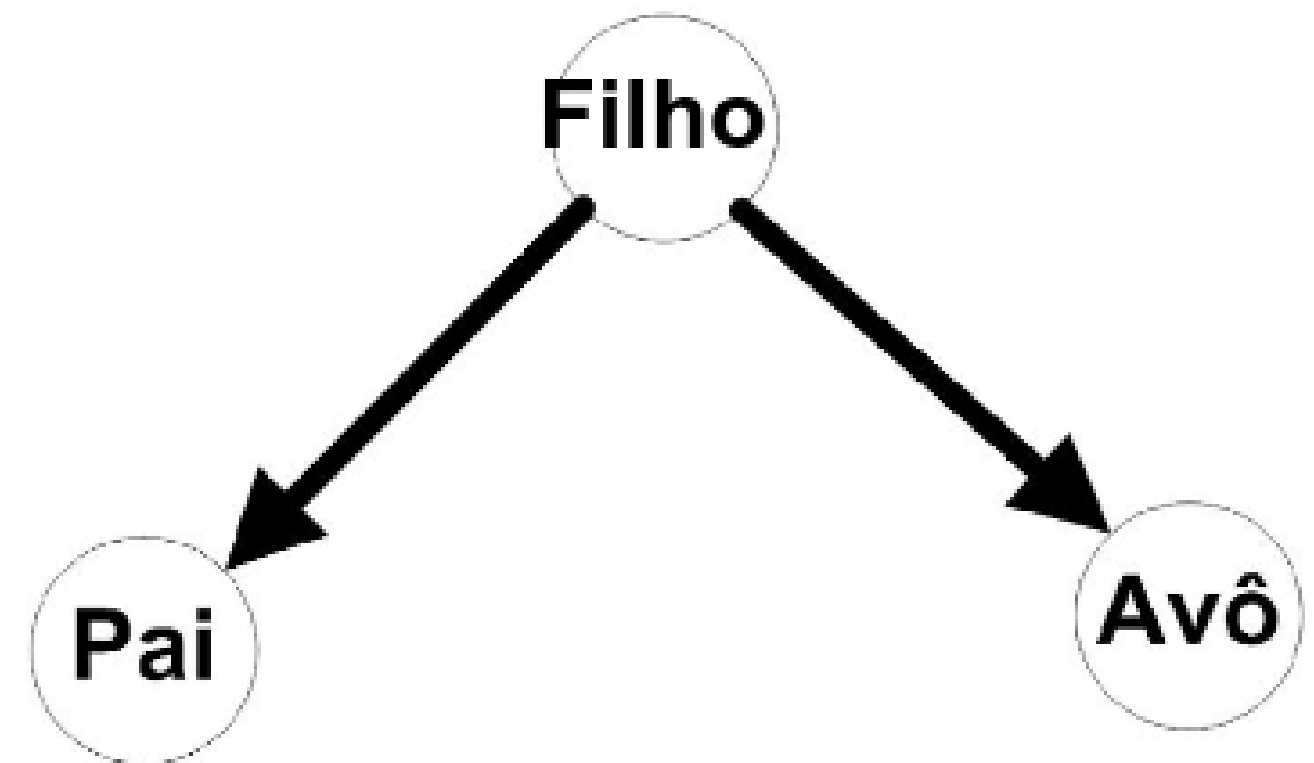
Balanceamento Árvores

Rotação Dupla Esquerda - Direita



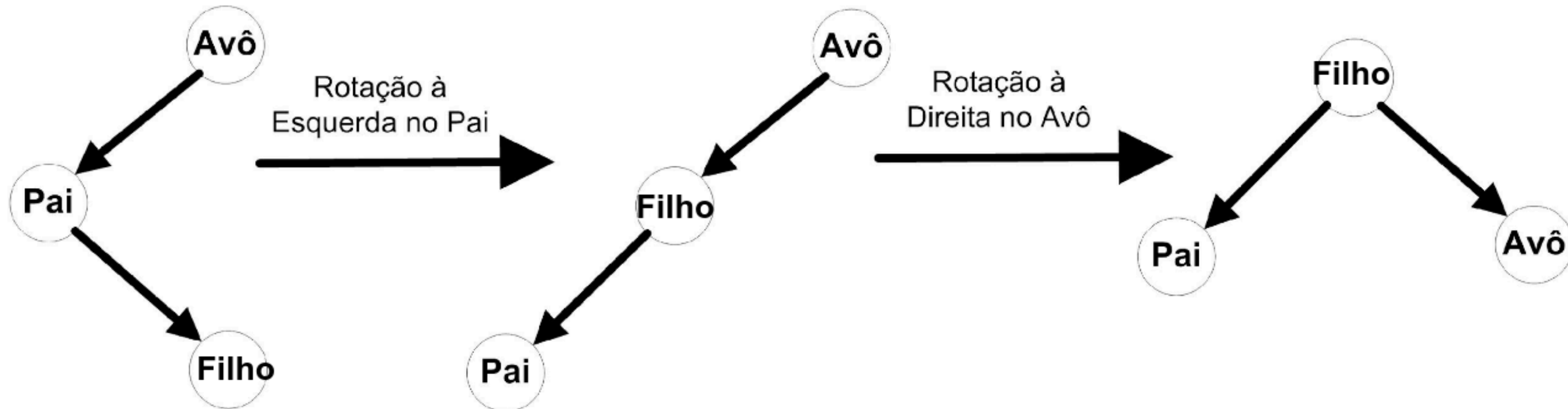
Balanceamento Árvores

Rotação Dupla Esquerda - Direita

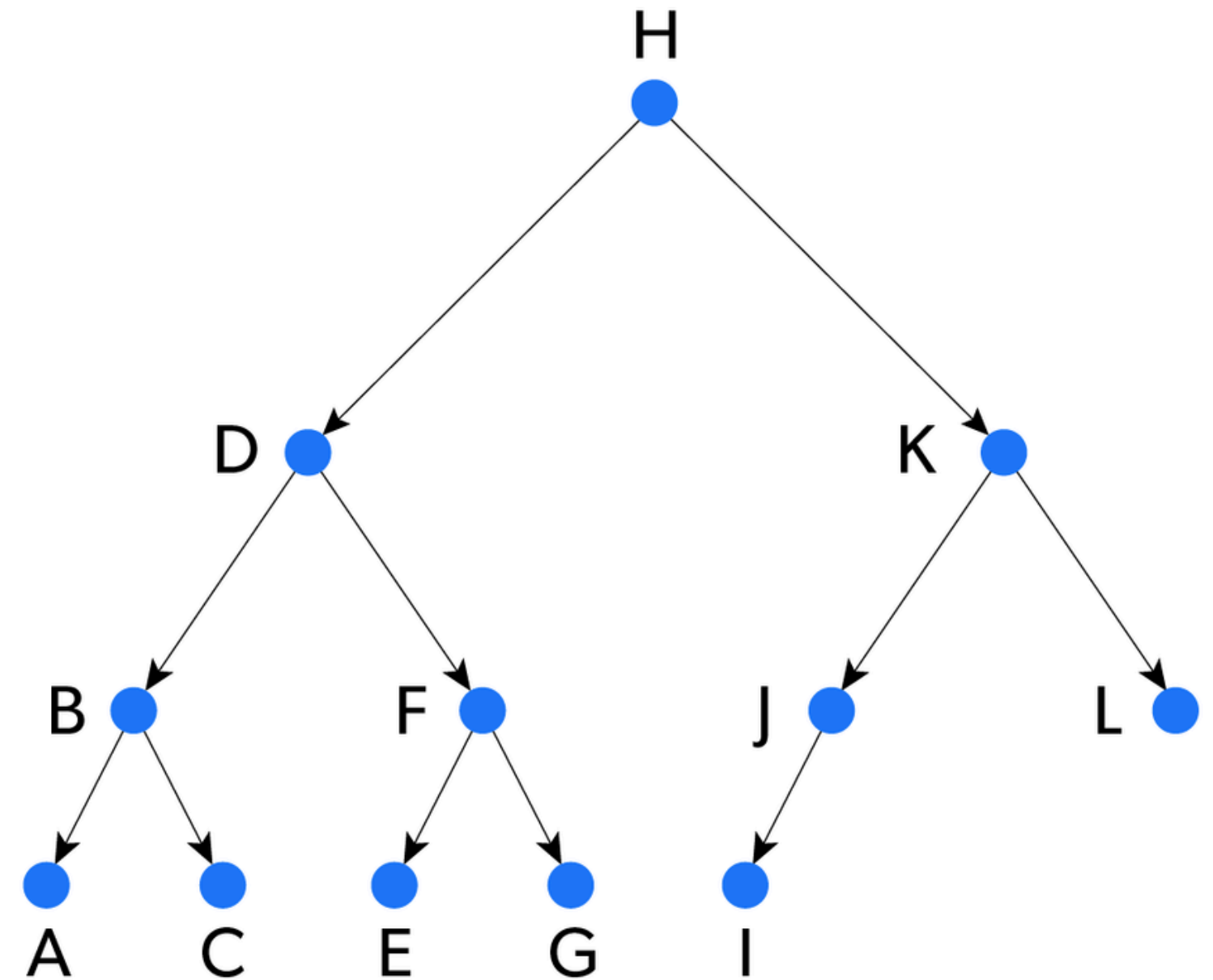


Hora do Código!

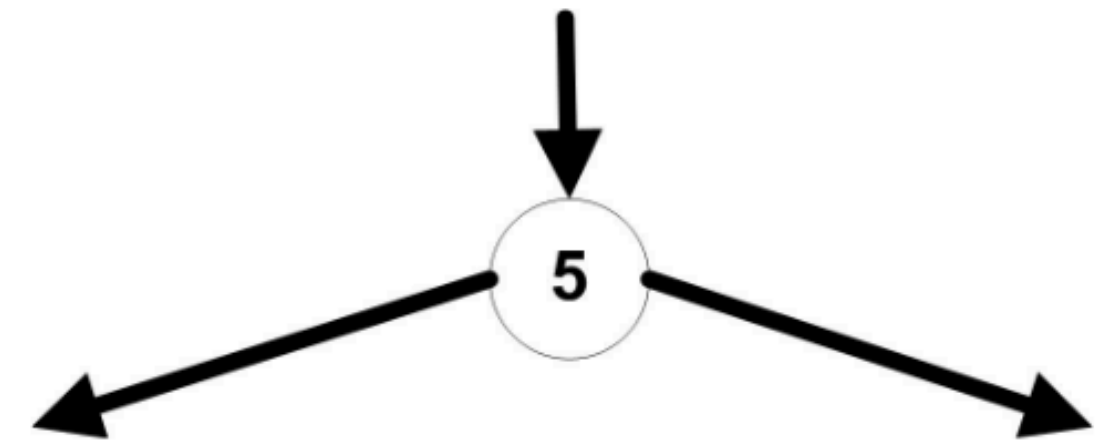
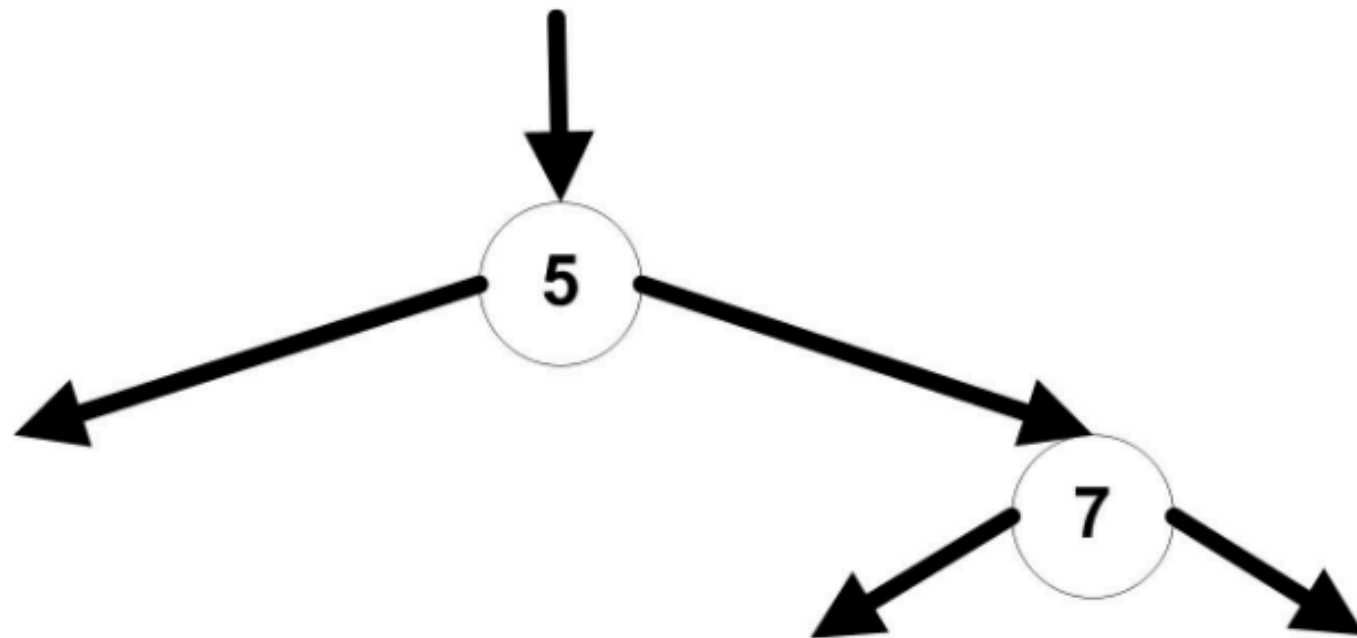
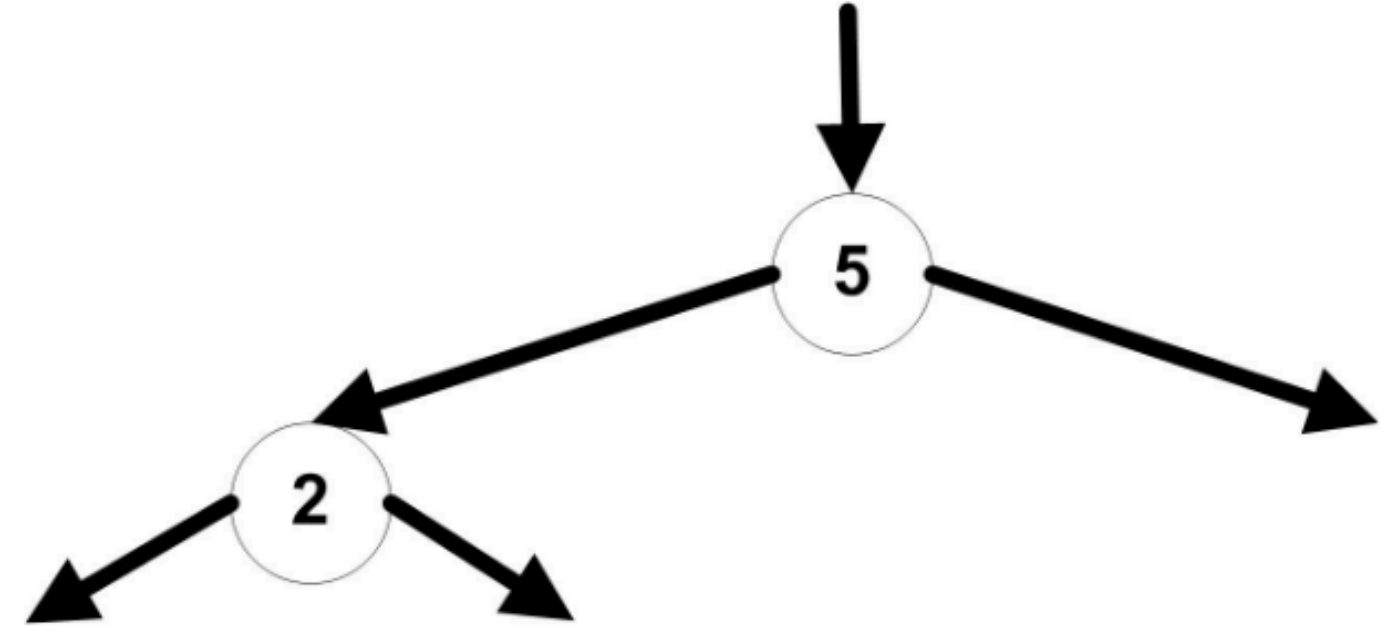
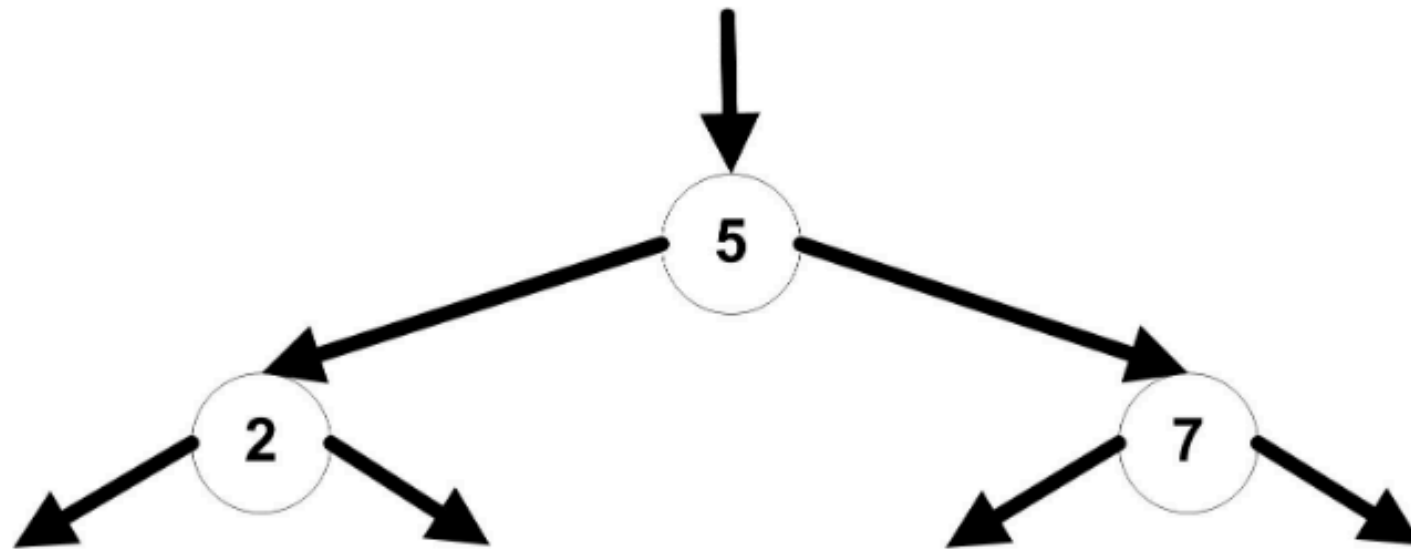
```
No rotacionarEsqDir(No no) {  
    no.esq = rotacionarEsq(no.esq);  
    return rotacionarDir(no);  
}
```



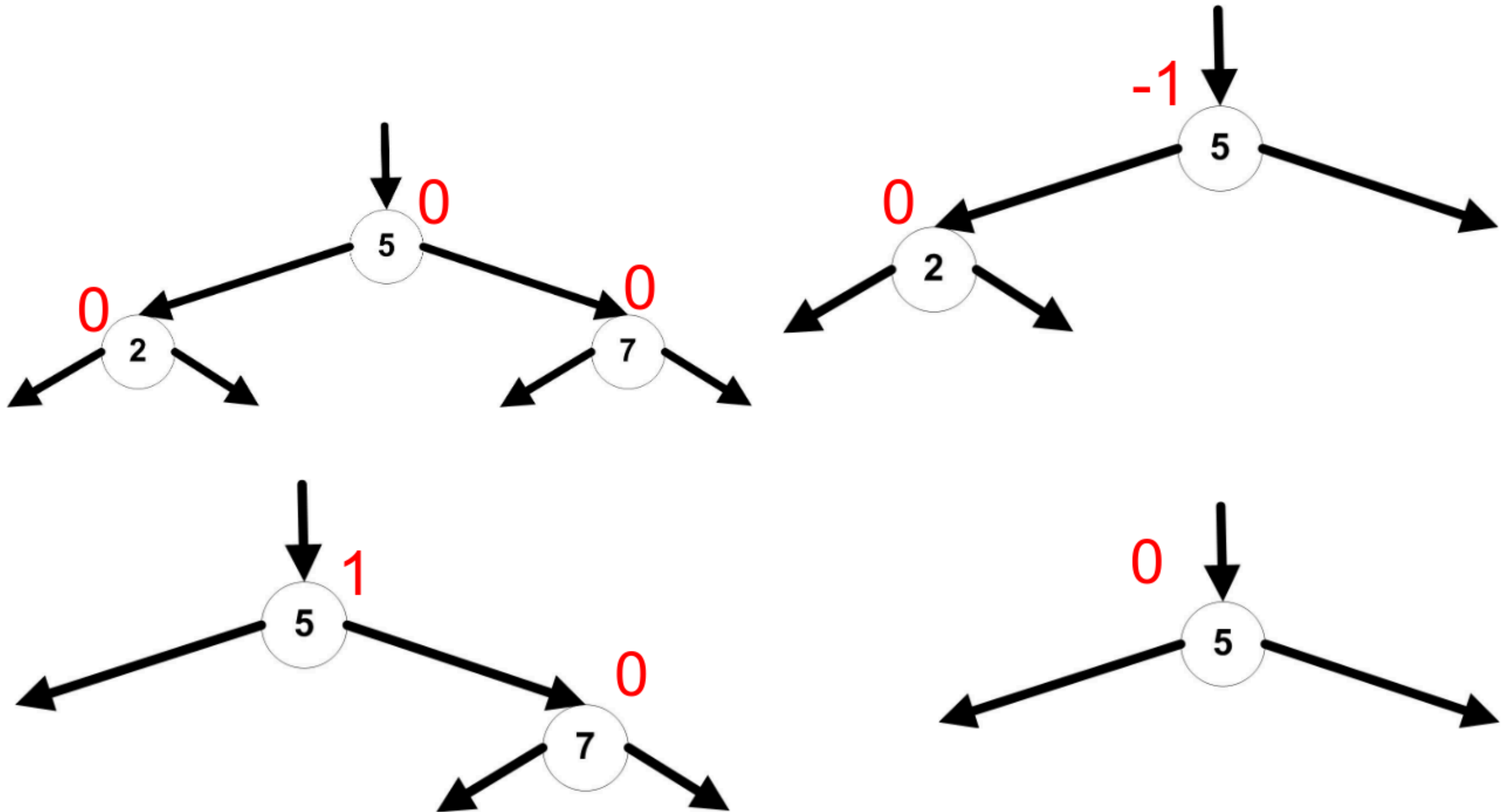
Árvores AVL



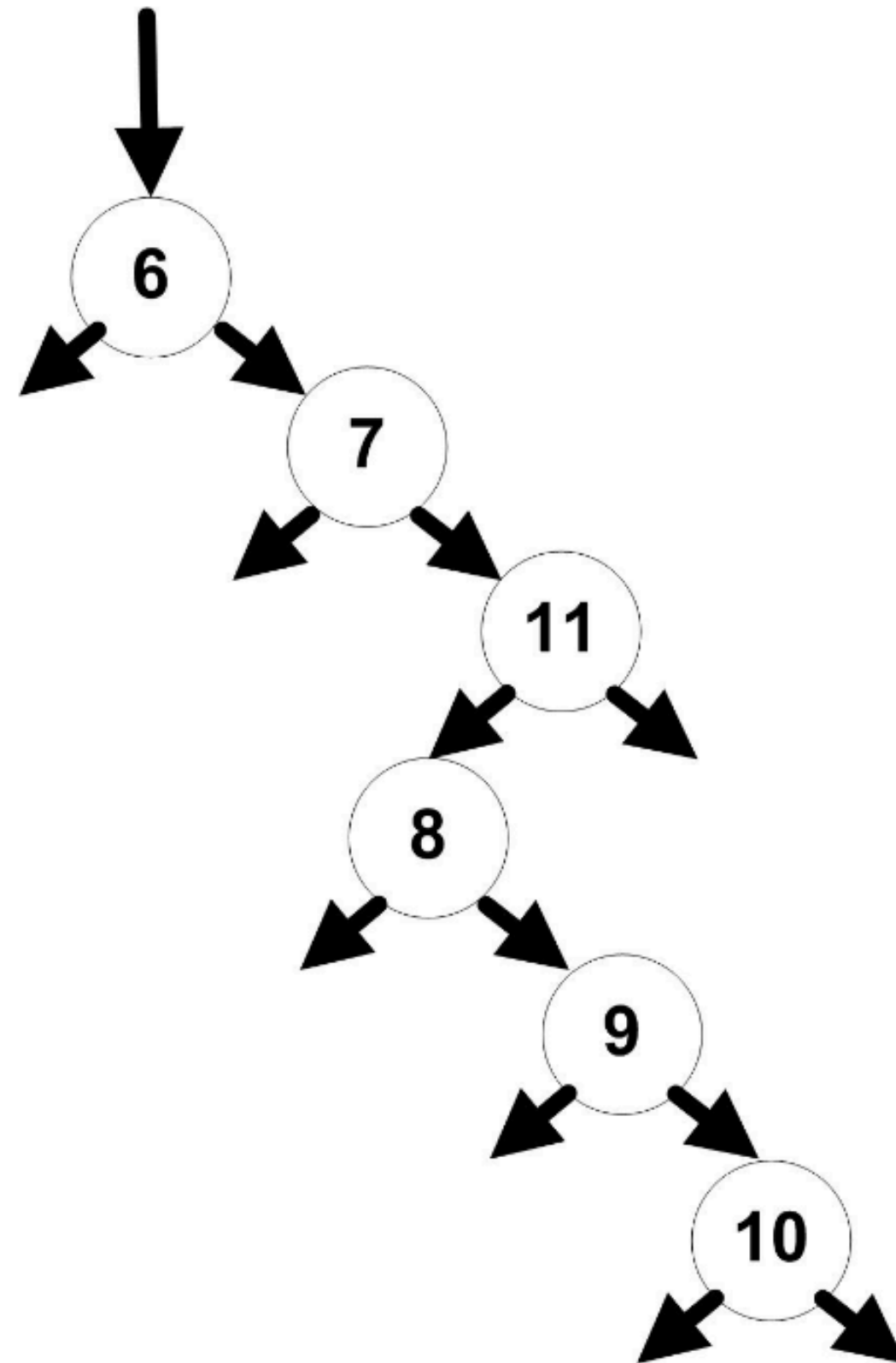
Qual o fator dos nós?



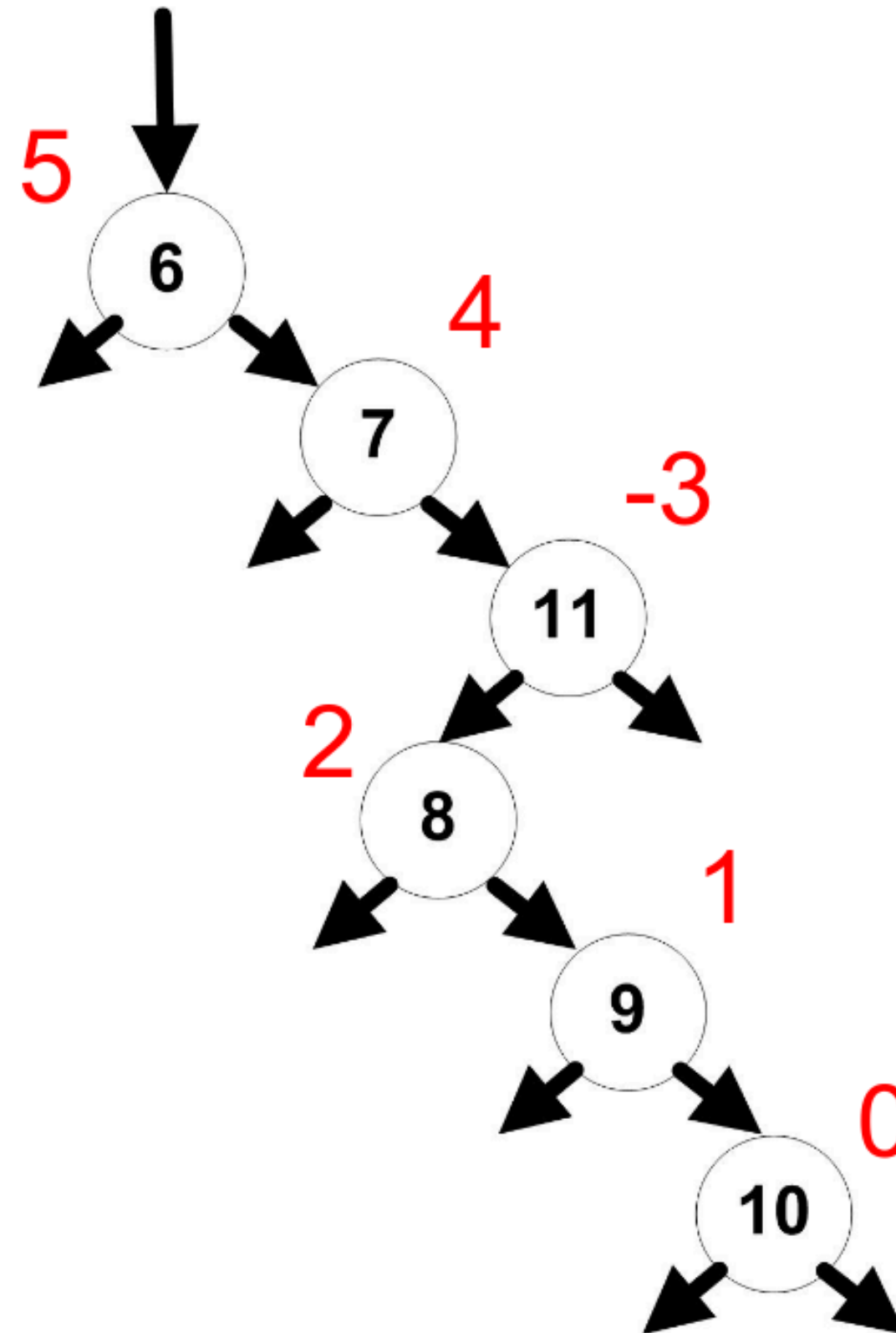
Qual o fator dos nós?



Qual o fator dos nós?



Qual o fator dos nós?



Decidindo a rotação

Fator do nó	Tipo de rotação
2	Rotação para a esquerda
-2	Rotação para a direita

Decidindo a rotação

Fator do nó	Tipo de rotação
2	Rotação para a esquerda  (R_E e R_{DE})
-2	Rotação para a direita  (R_D e R_{ED})

Decidindo a rotação

Fator do nó		O tipo de rotação depende do fator do filho pertencente a maior subárvore
		Ou seja, o pai...
2		Rotação para a esquerda $(R_E \text{ e } R_{DE})$
-2		Rotação para a direita $(R_D \text{ e } R_{ED})$

Decidindo a rotação

Fator do nó	Fator do filho à direita	Fator do filho à esquerda	Tipo de rotação
2	1 0		Simples à esquerda
	-1		Dupla dir-esq
-2		-1 0	Simples à direita
		1	Dupla esq-dir

Exercício!

- Crie uma árvore AVL através de inserções sucessivas do 4, 35, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente.

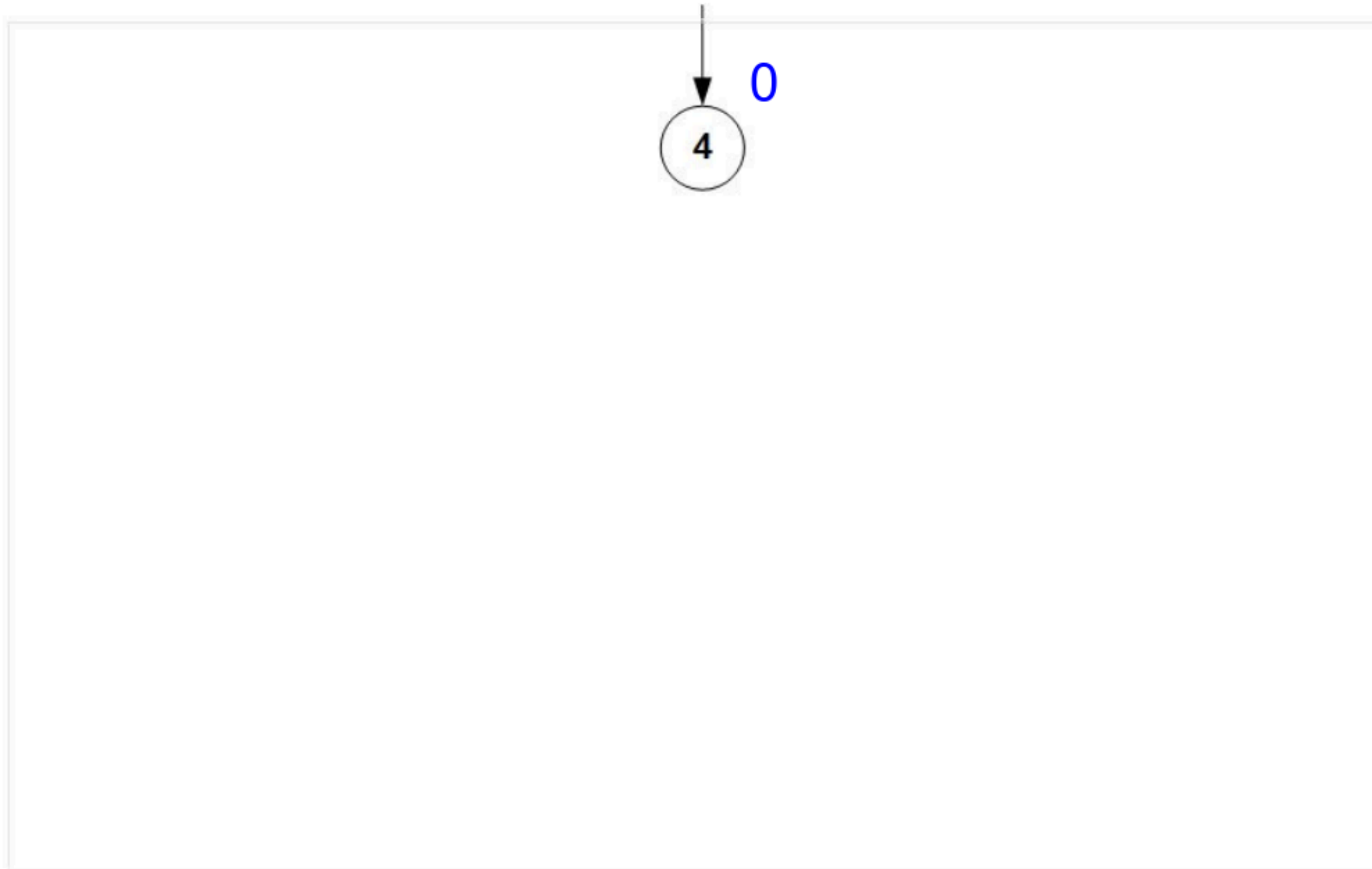
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do 4, 35, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente.



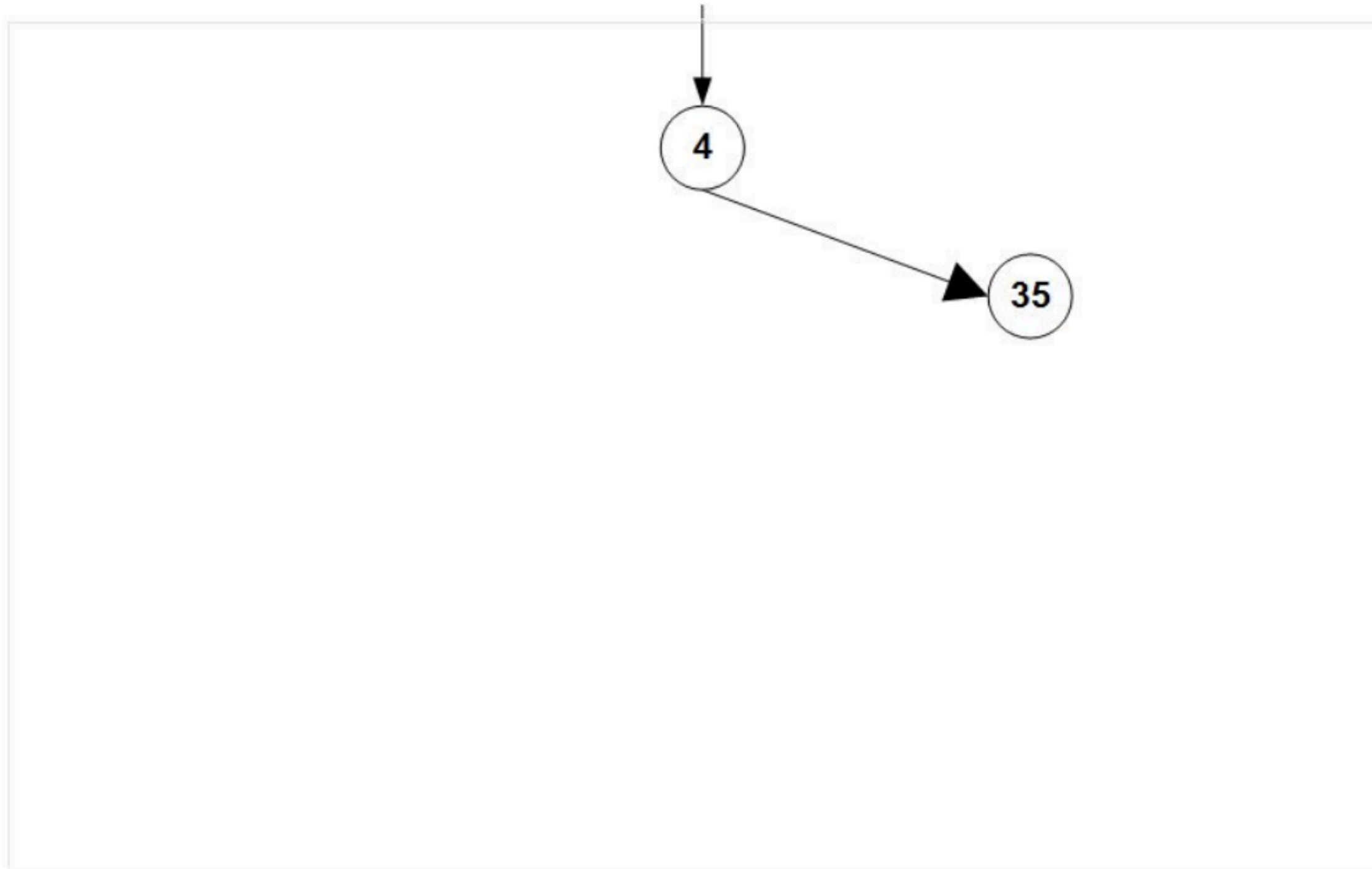
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do 4, 35, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente



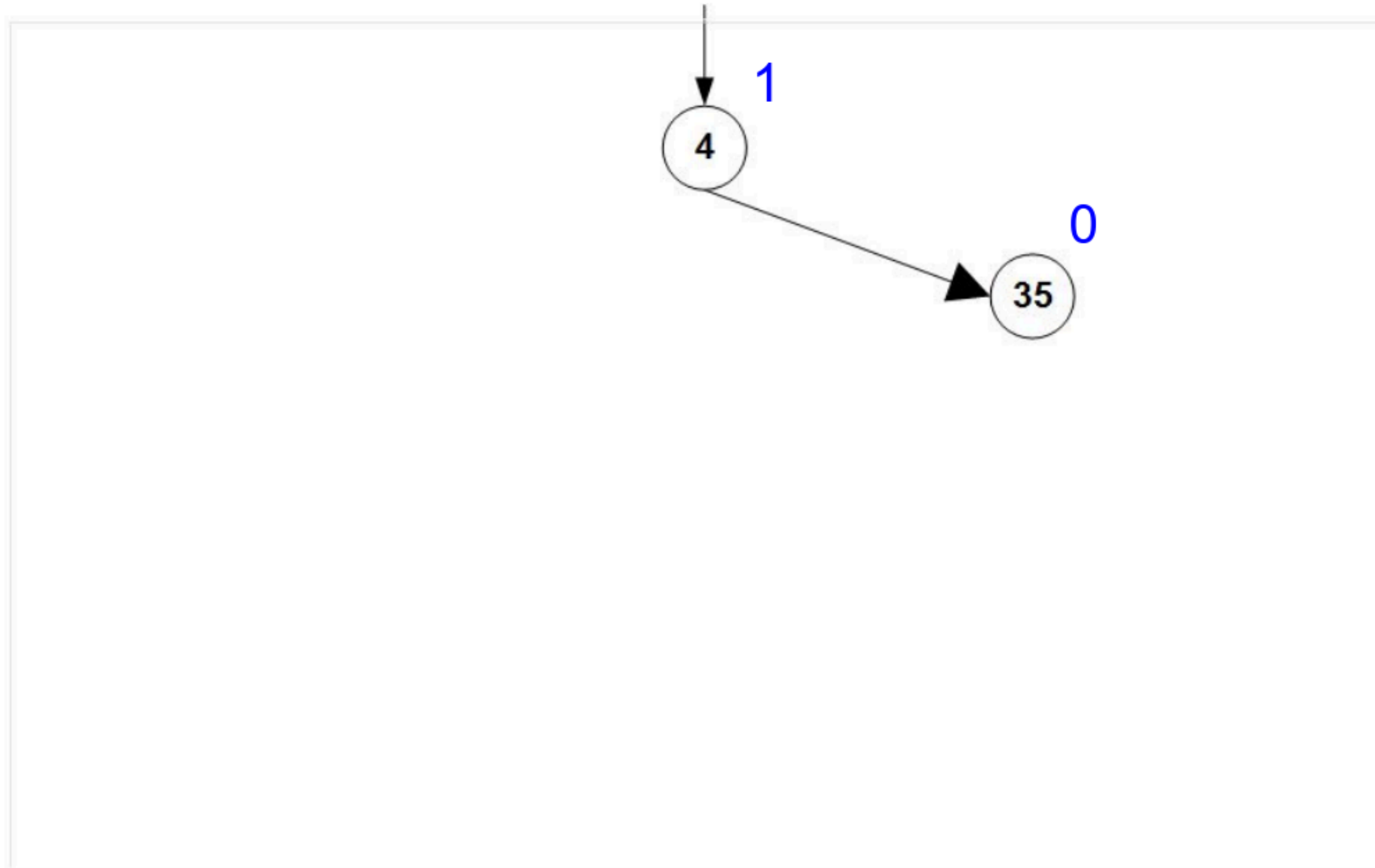
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente



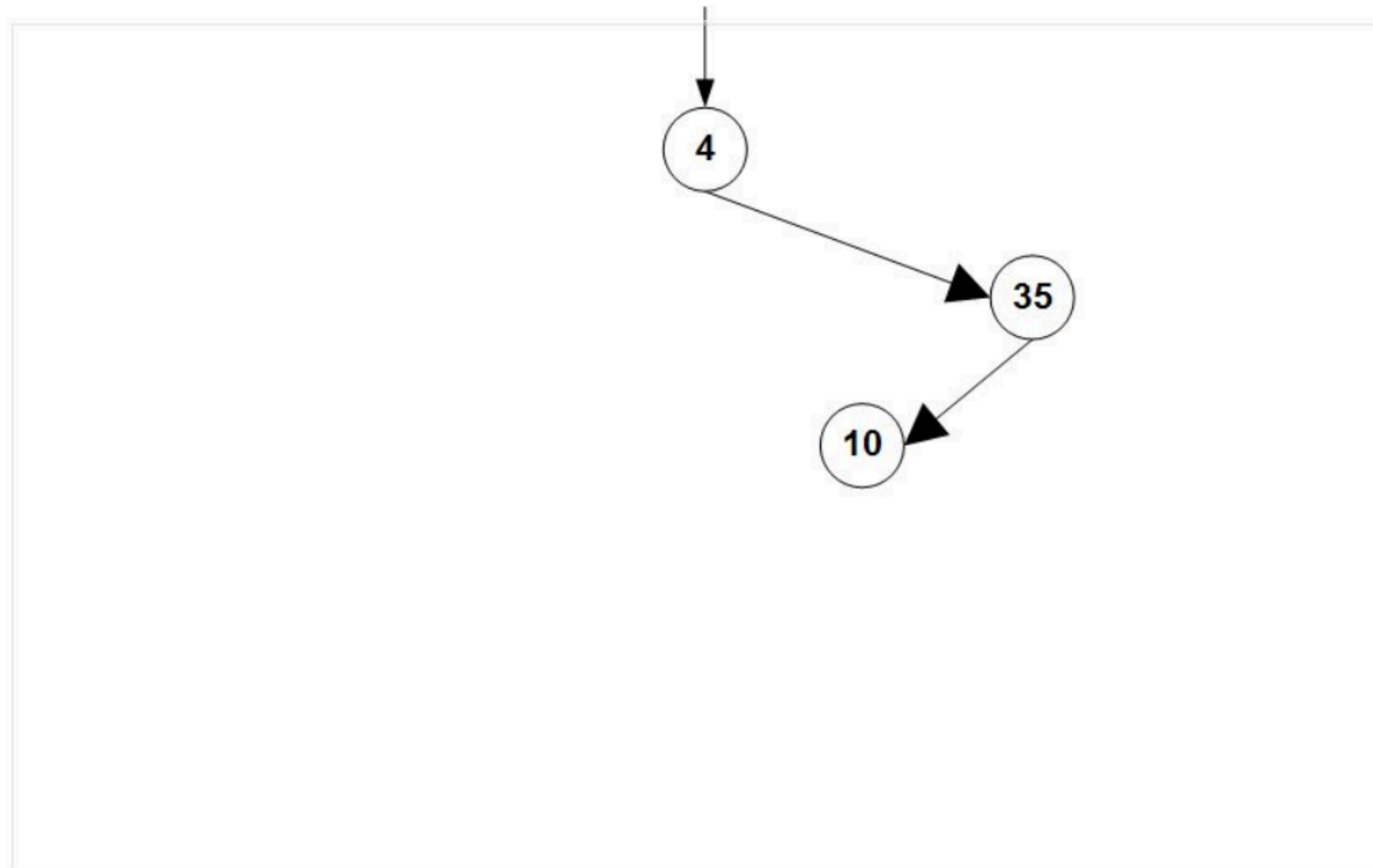
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente



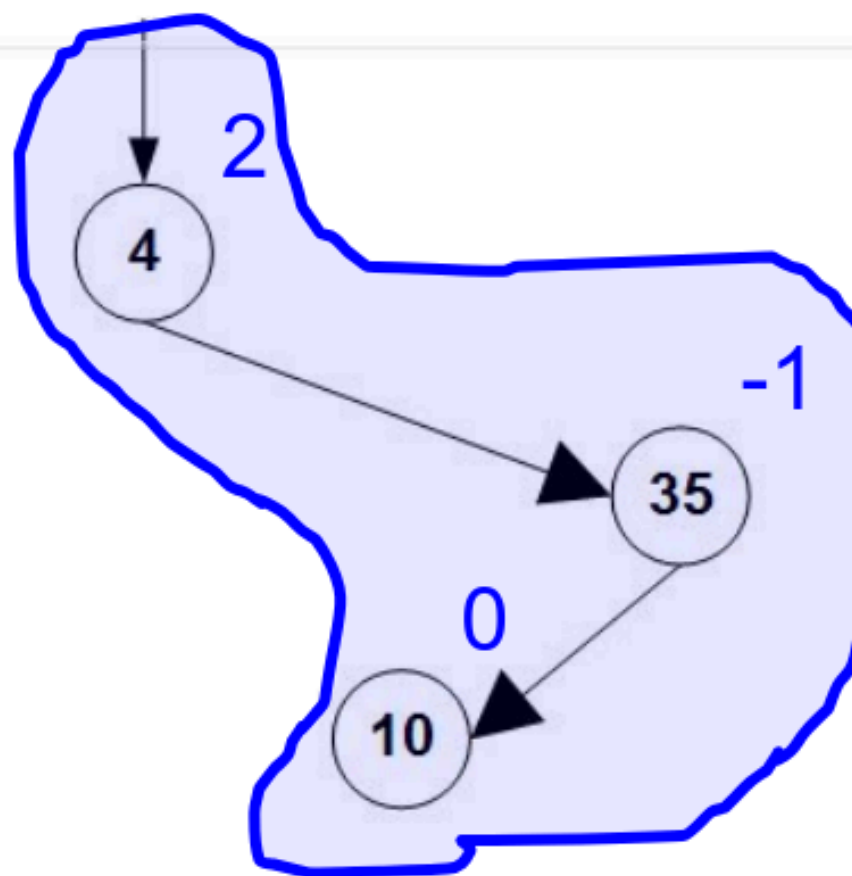
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente



Exercício!

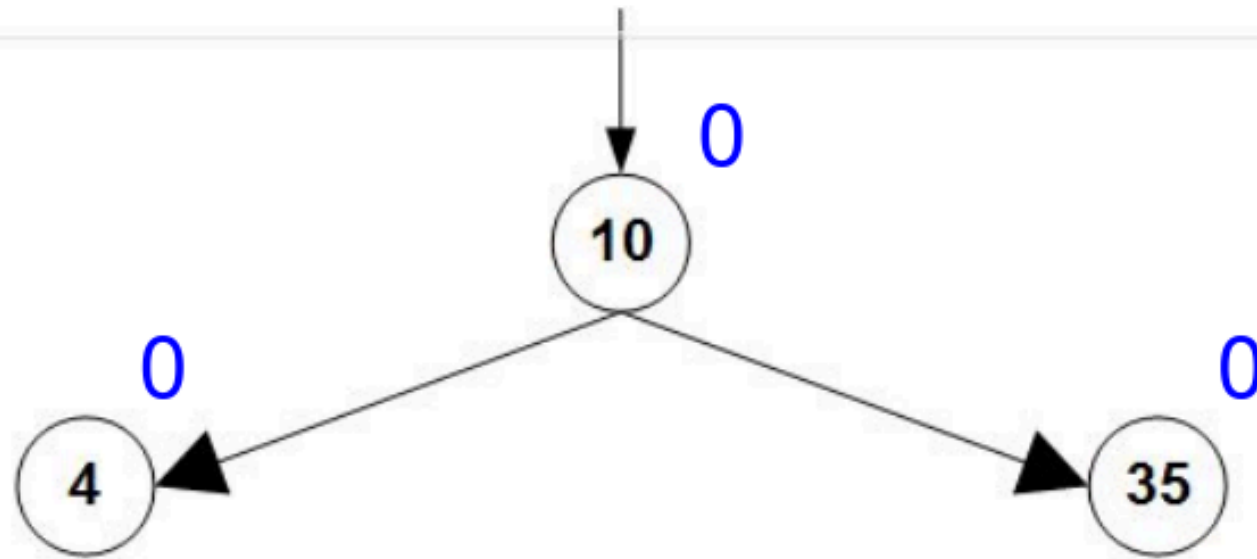
- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente rotação Dir (35) Esq (4)



Como o fator(4) = 2 e seu filho à direita -1, faremos uma rotação DirEsq

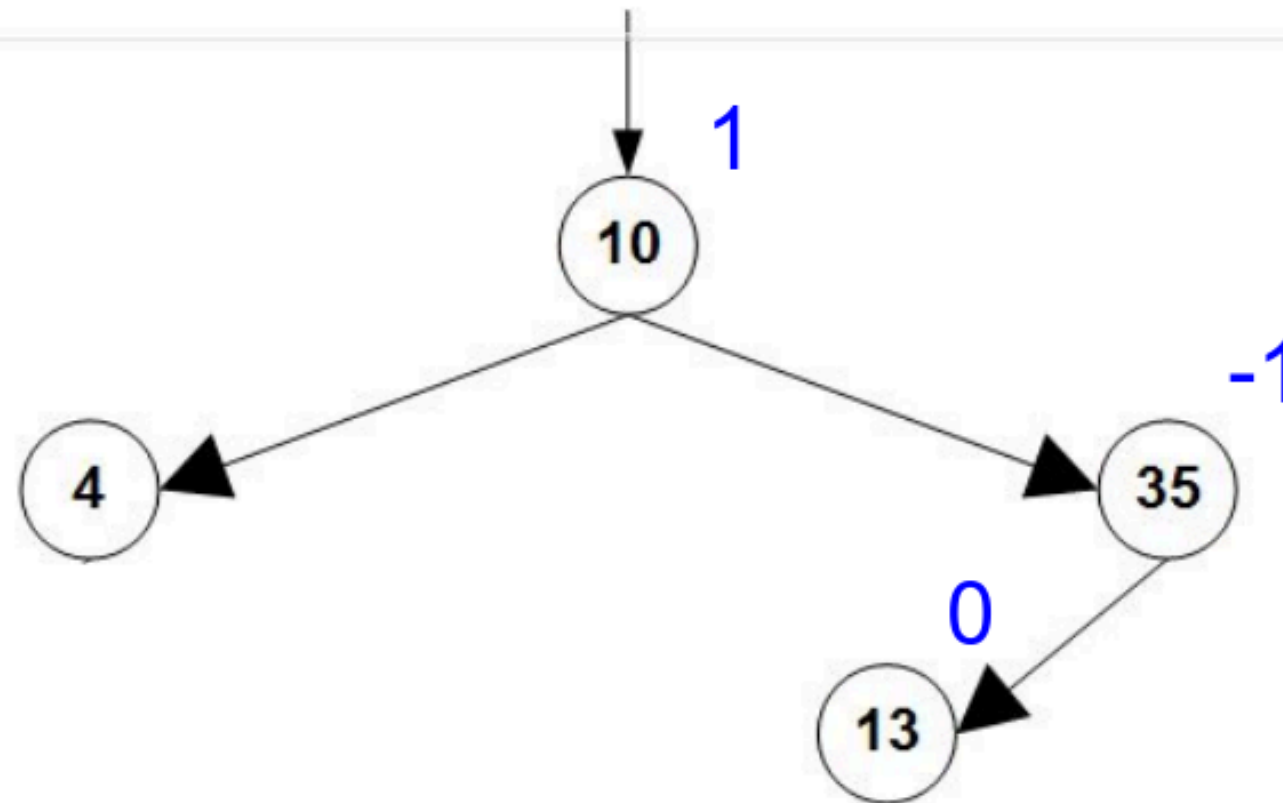
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente rotação Dir (35) Esq (4)



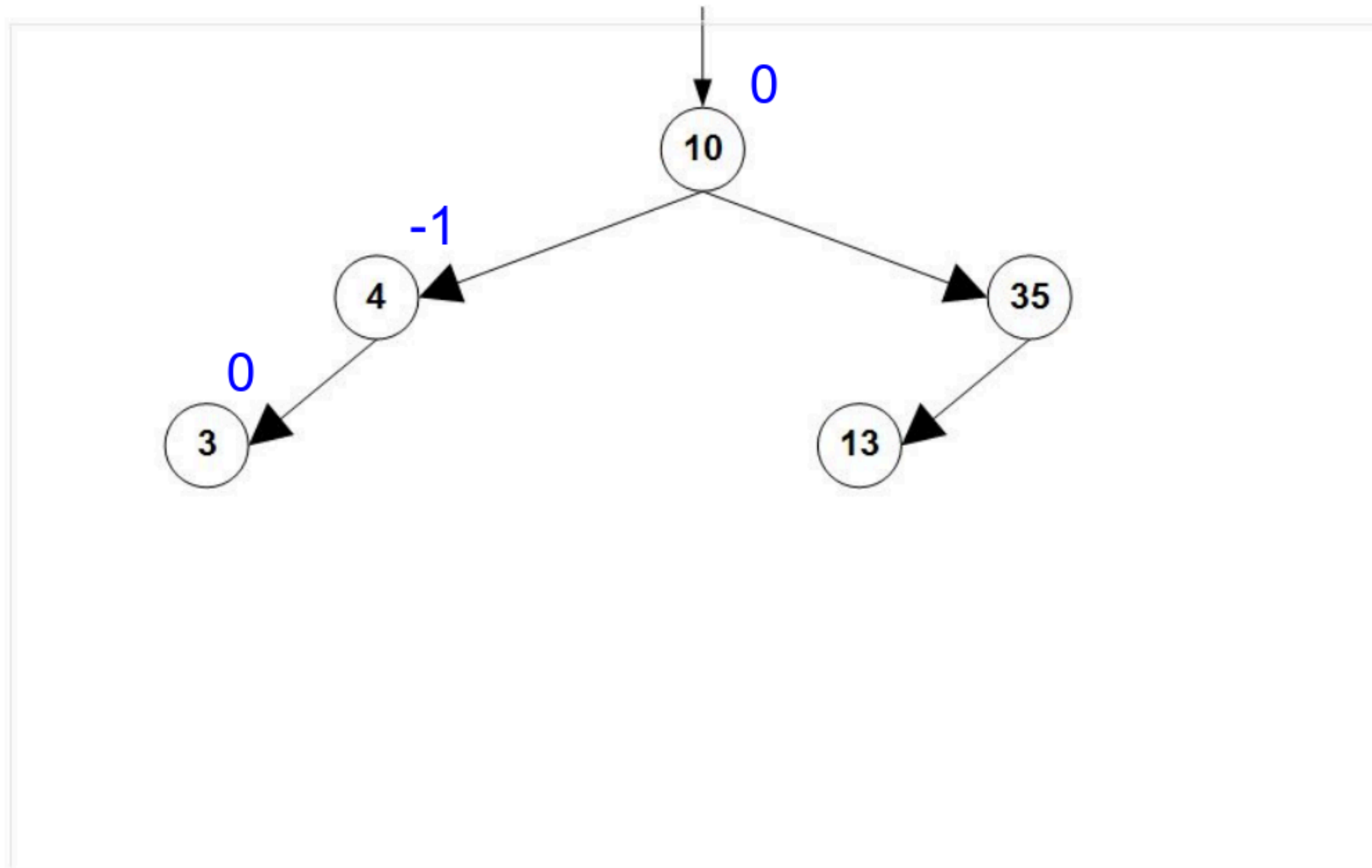
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, 3, 30, 15, 12, 7, 40 e 20 respectivamente



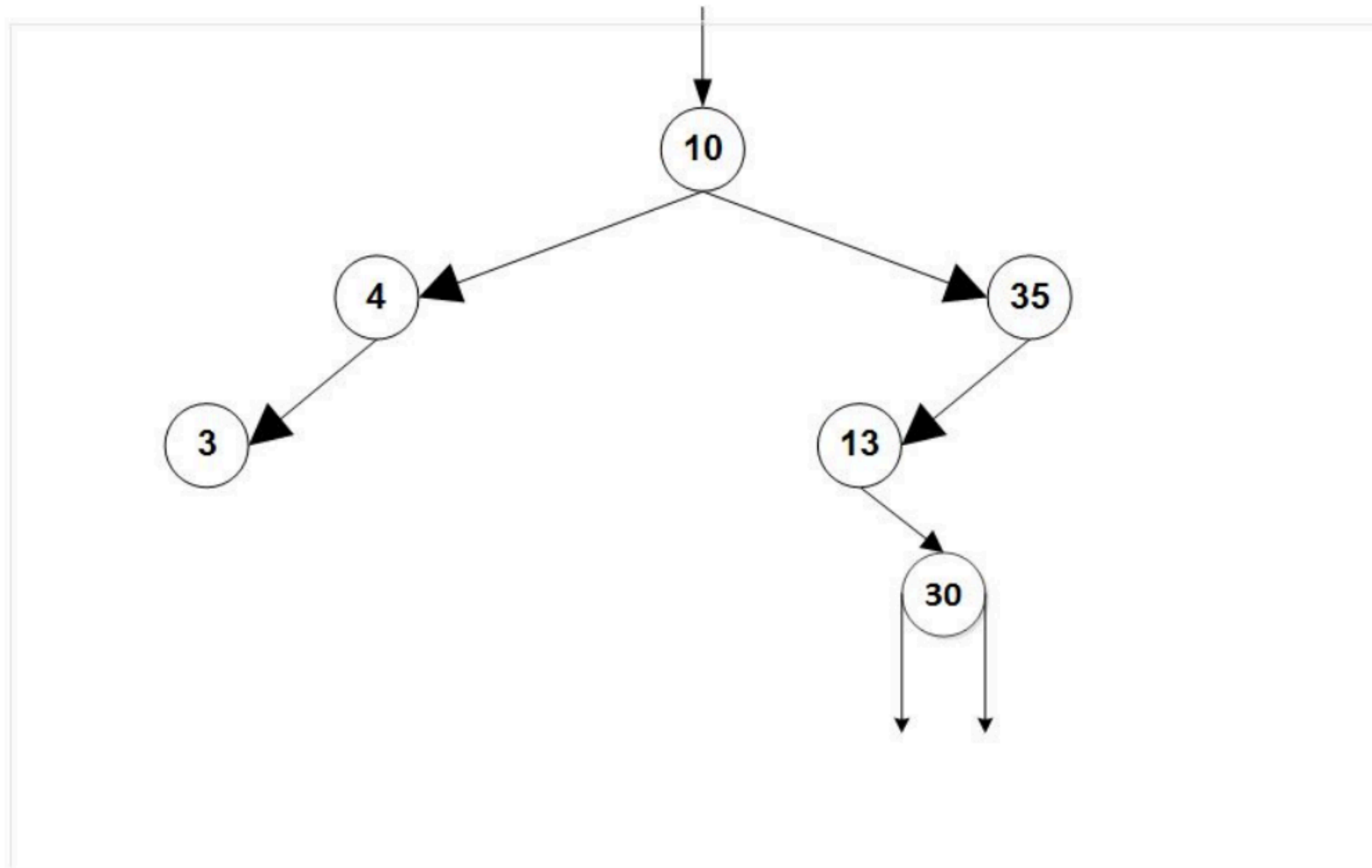
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, 30, 15, 12, 7, 40 e 20 respectivamente



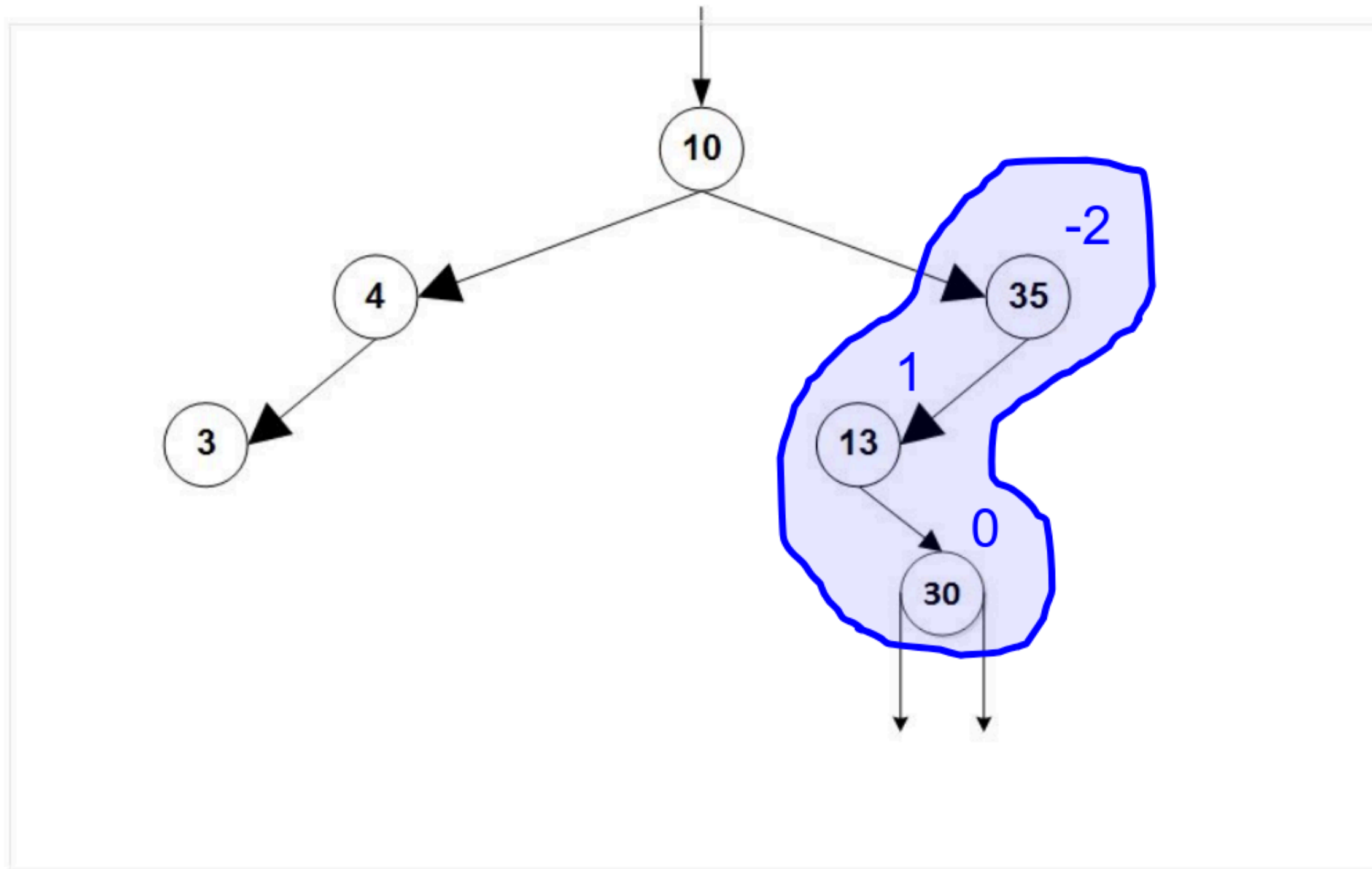
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, 15, 12, 7, 40 e 20 respectivamente



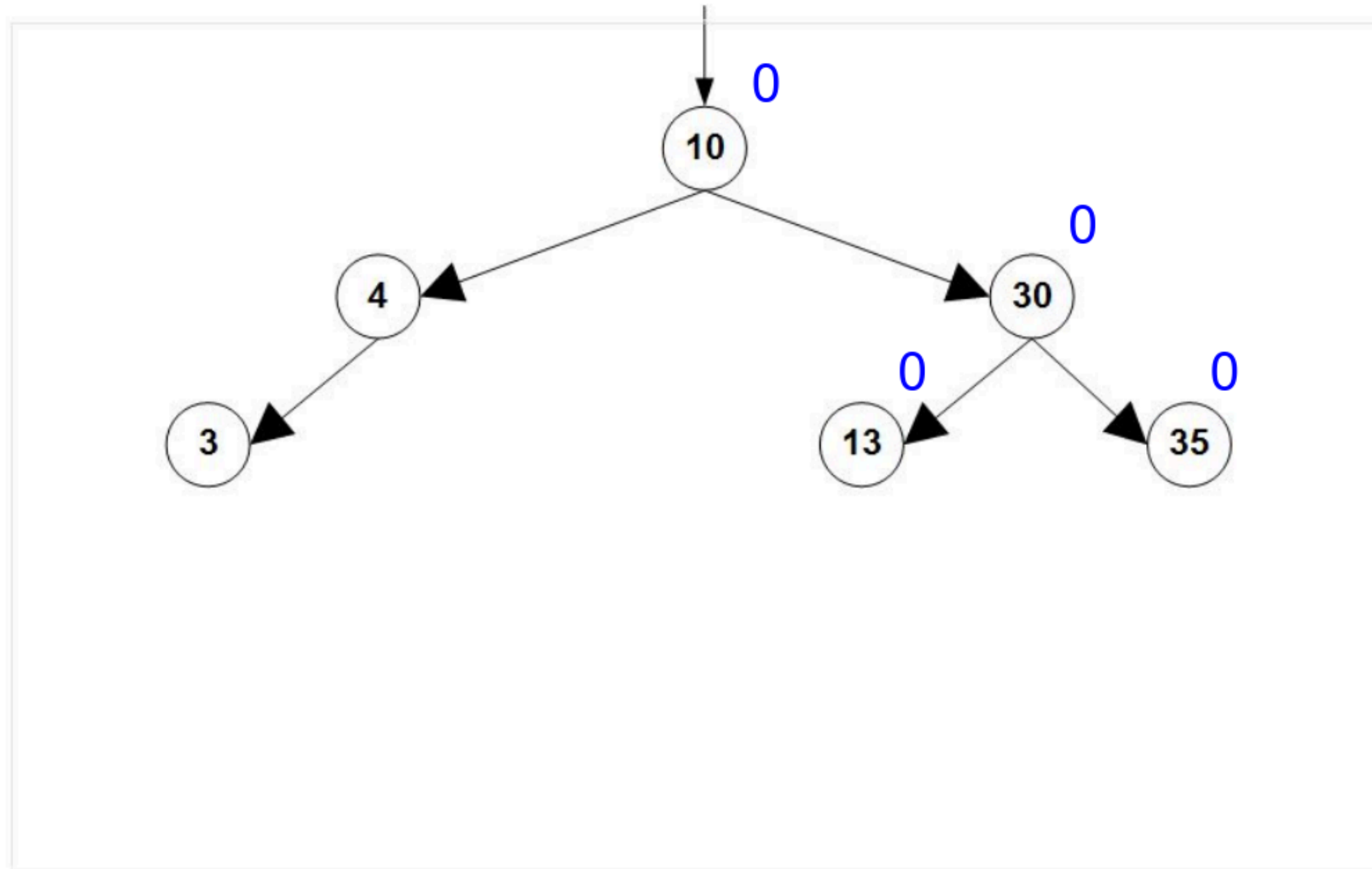
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, 15, 12, 7, 40 e 20 respectivamente rotação Esq (13) Dir (35)



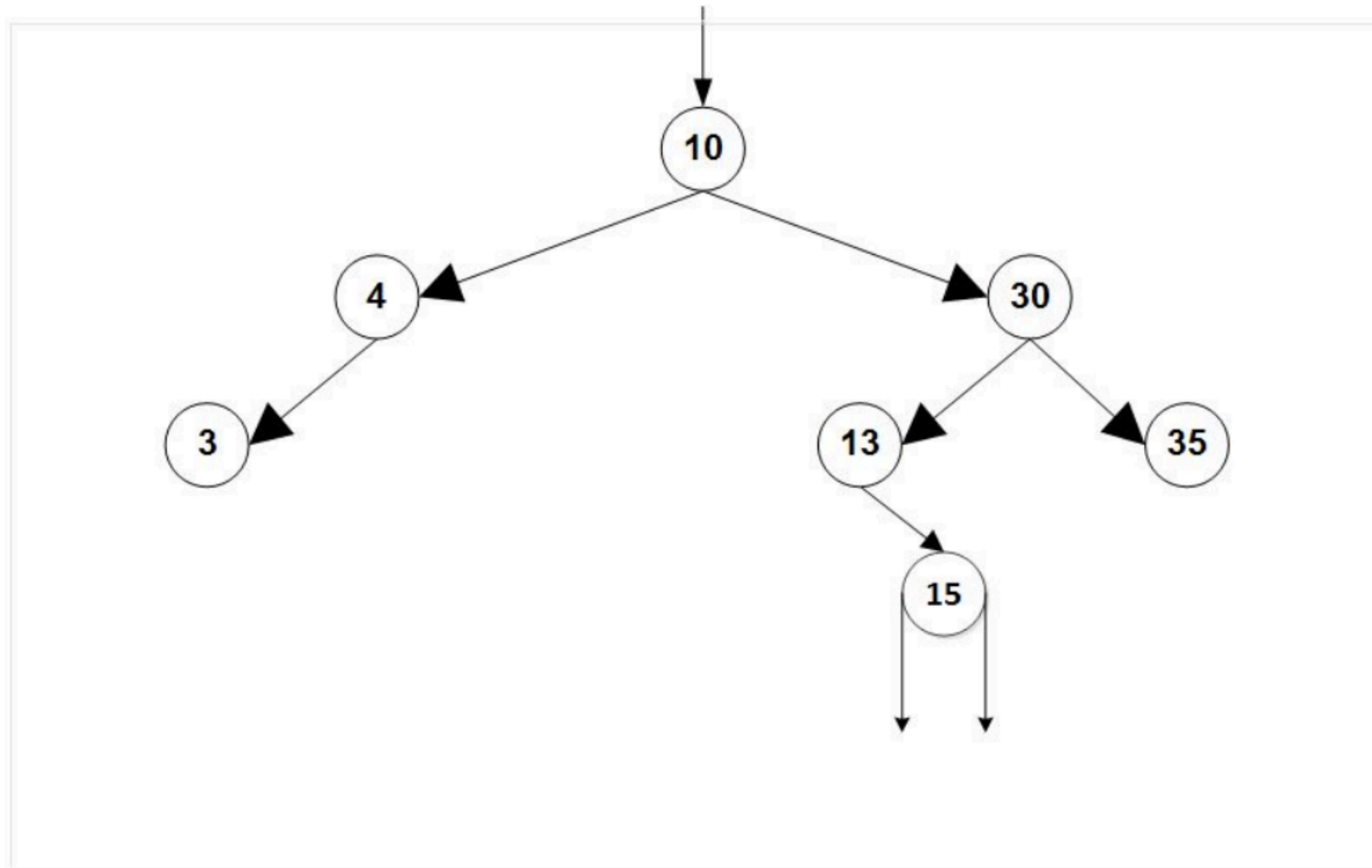
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, 15, 12, 7, 40 e 20 respectivamente rotação Esq (13) Dir (35)



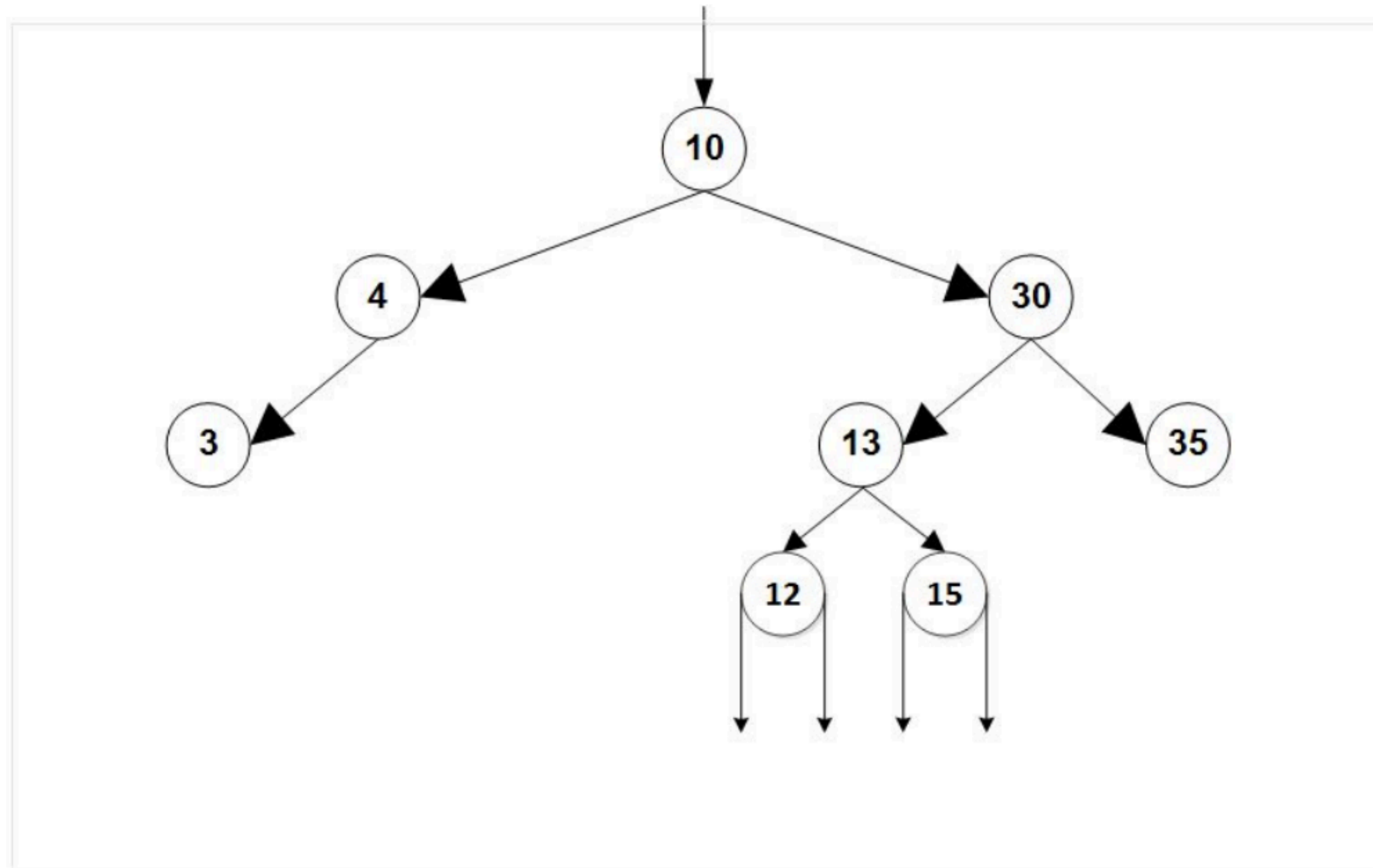
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, 12, 7, 40 e 20 respectivamente



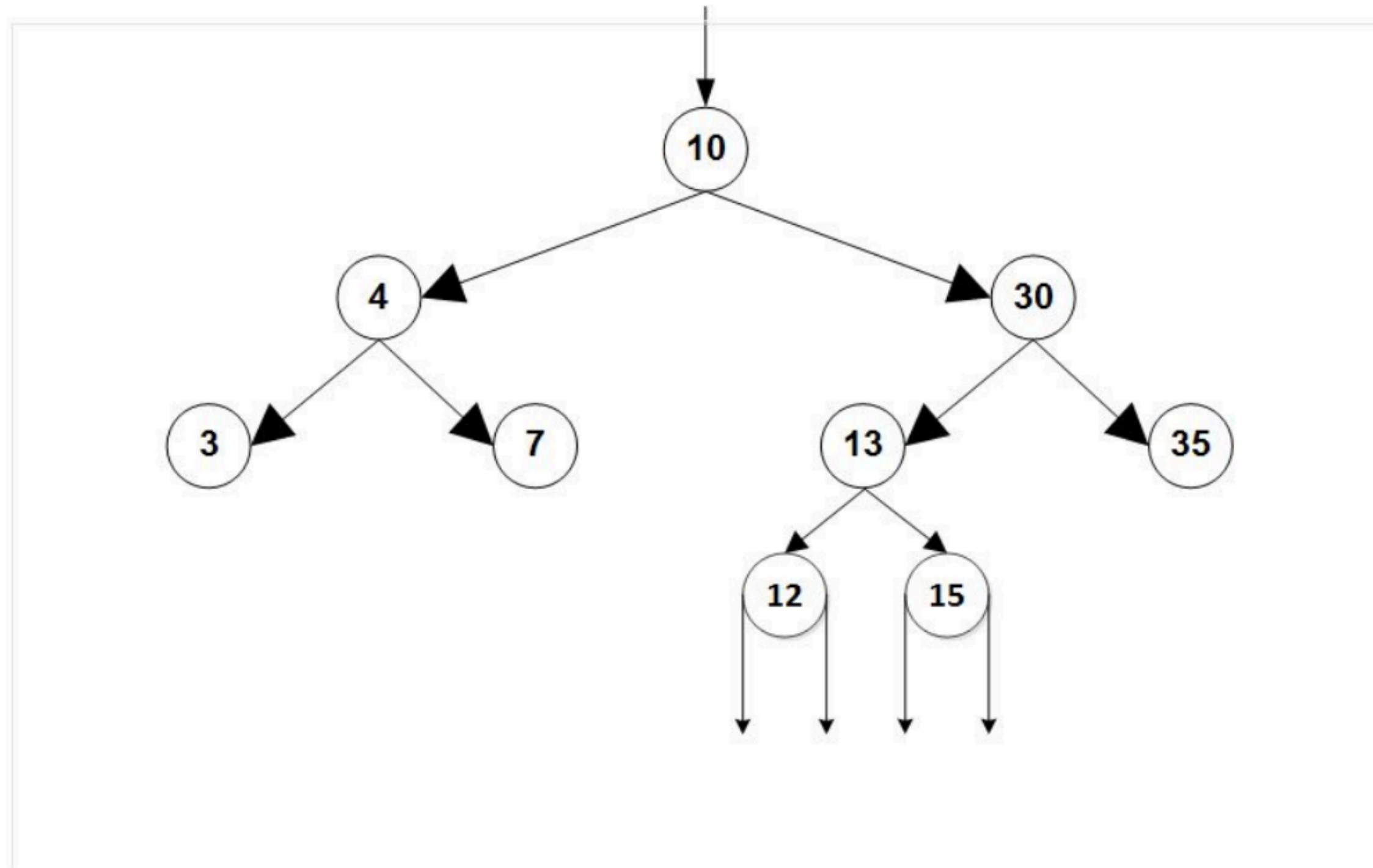
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, 7, 40 e 20 respectivamente



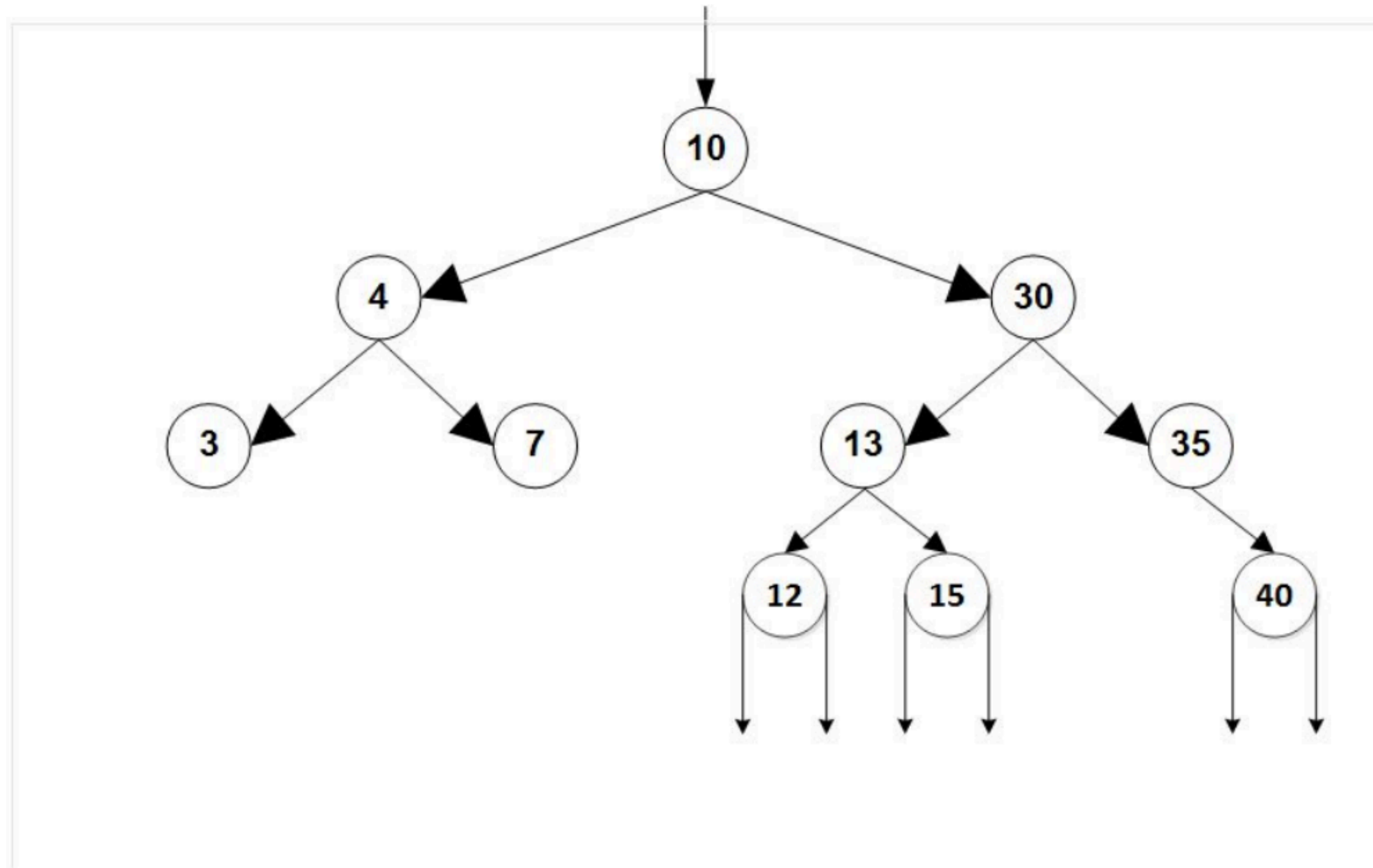
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, **7**, 40 e 20 respectivamente



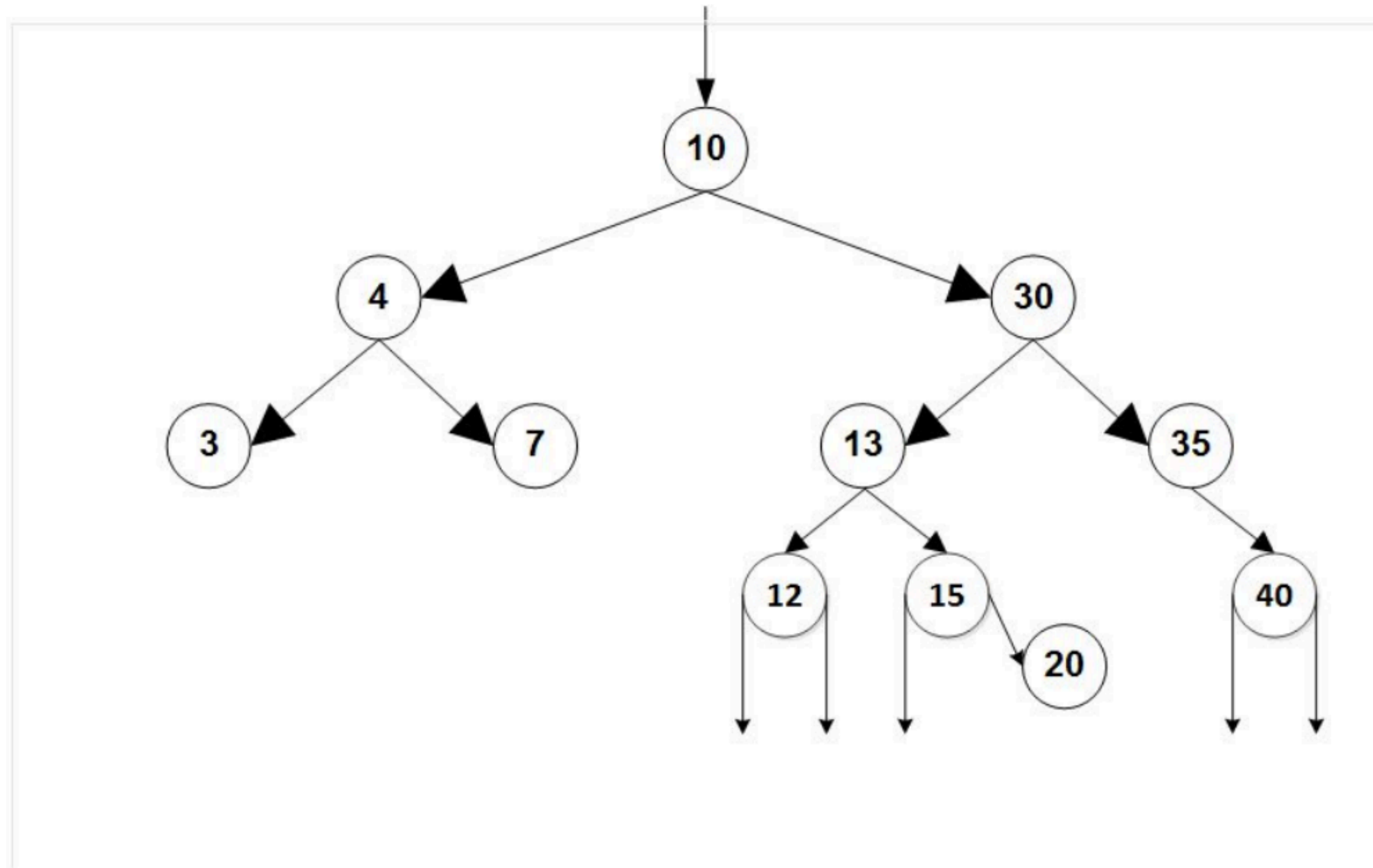
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, **7**, **40** e 20 respectivamente



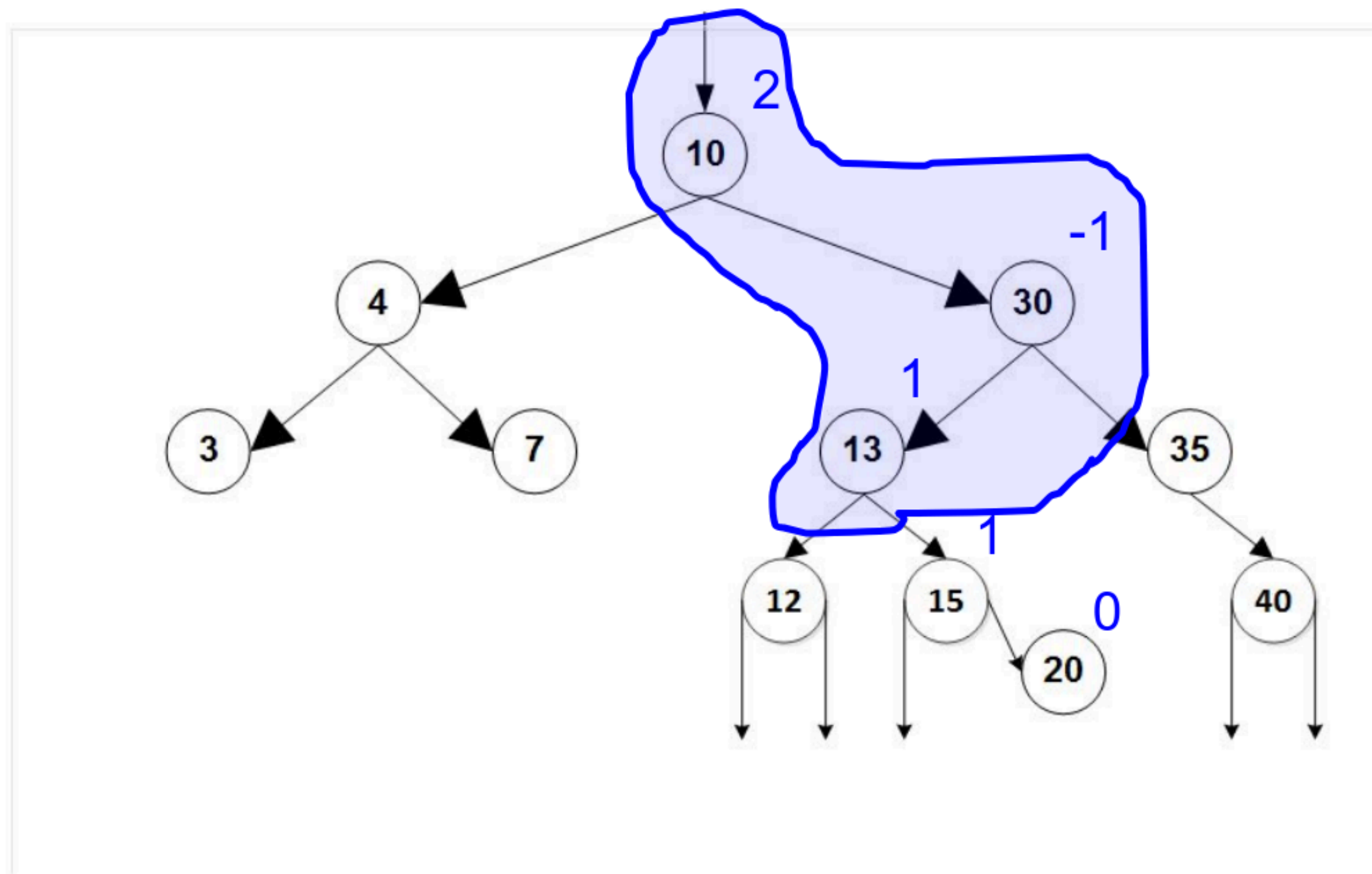
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, **7**, **40** e **20** respectivamente



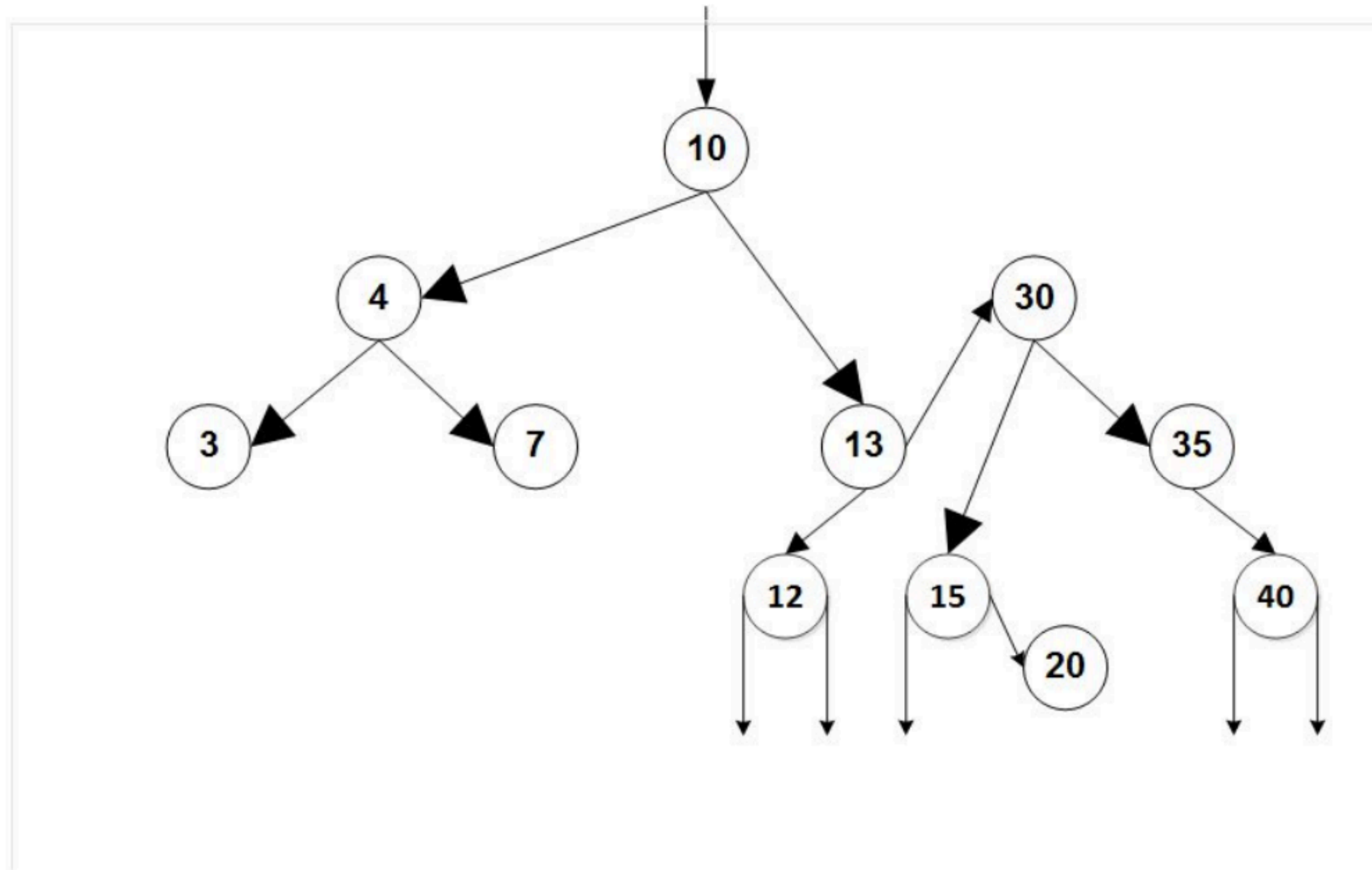
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, **7**, **40** e **20** respectivamente rotação Dir (30) Esq (10)



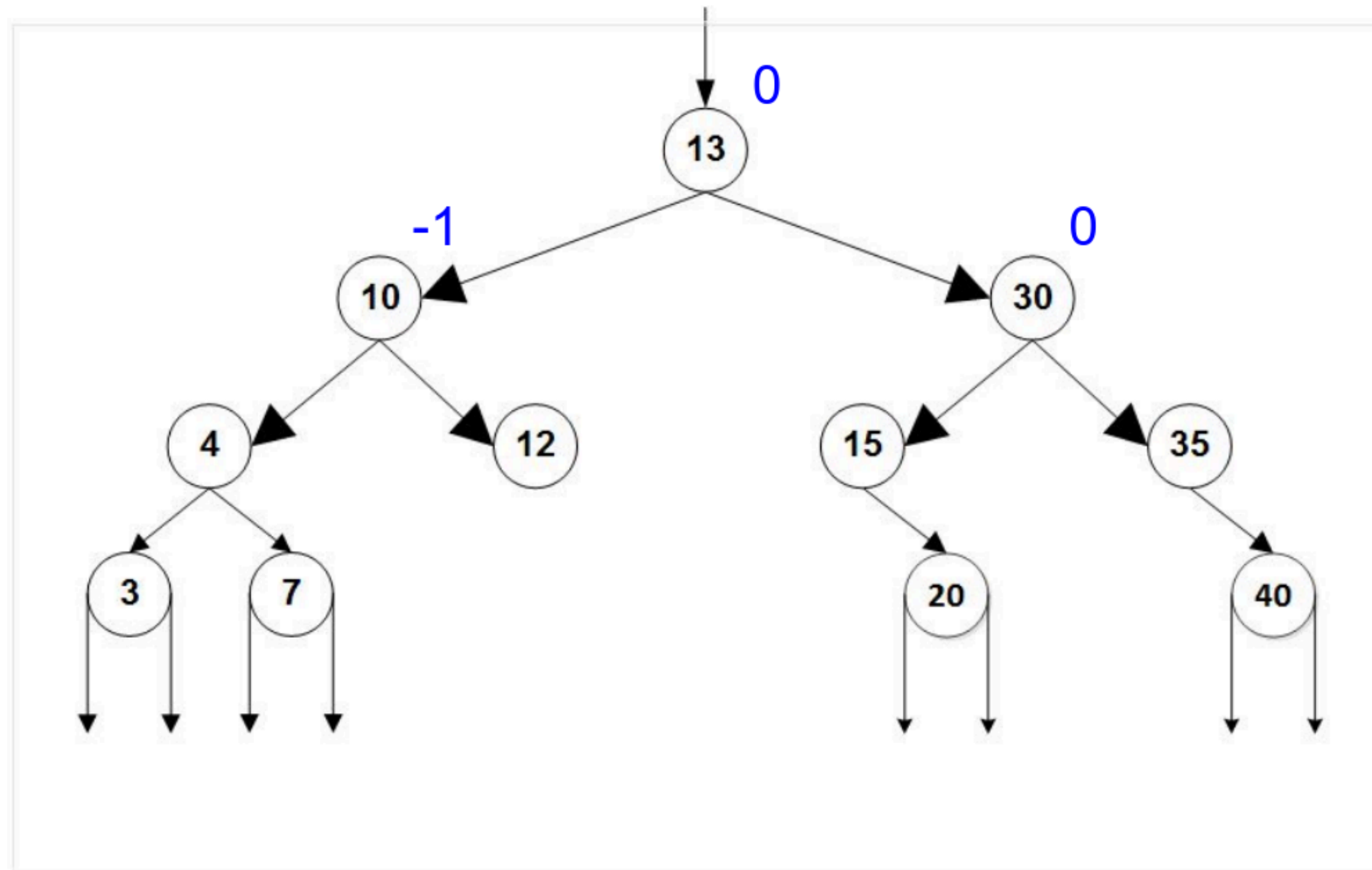
Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, **7**, **40** e **20** respectivamente rotação Dir (30) Esq (10)

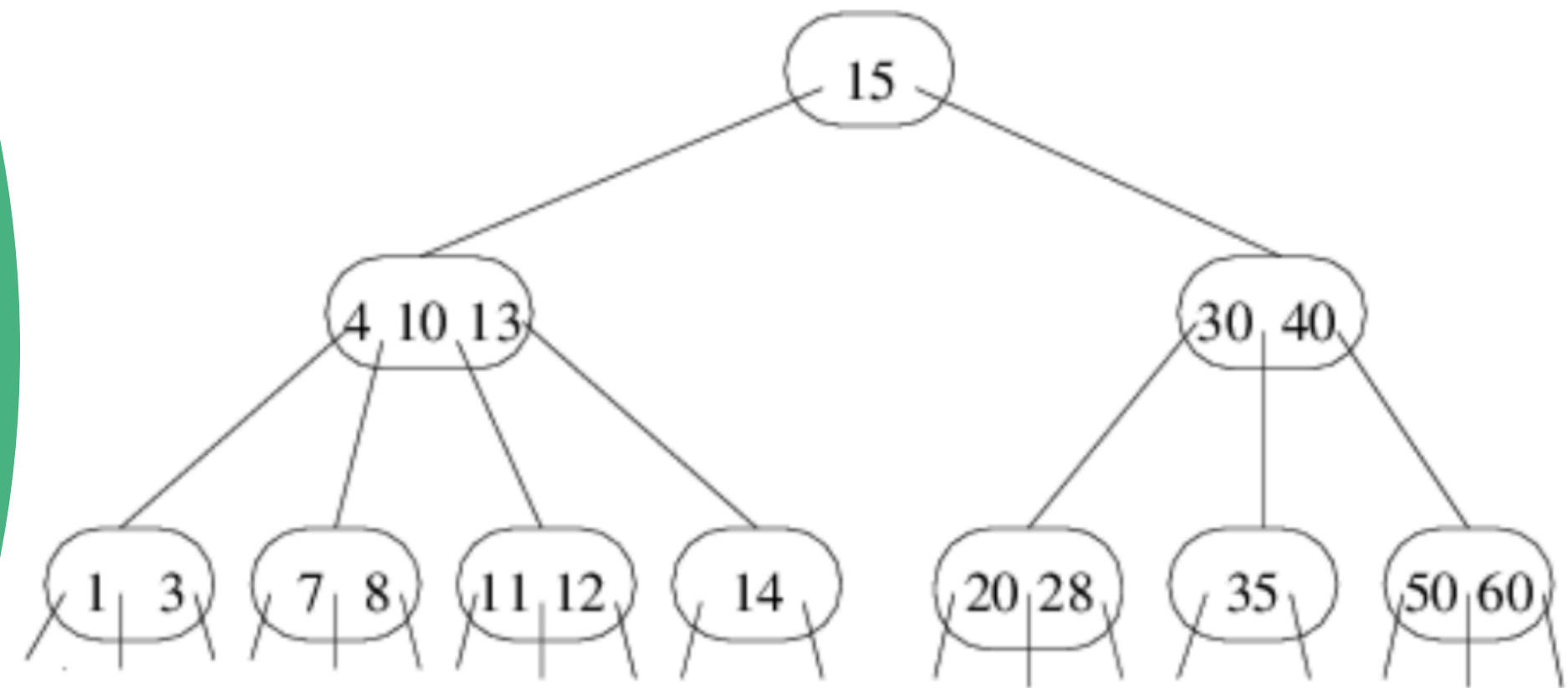


Exercício!

- Crie uma árvore AVL através de inserções sucessivas do **4**, **35**, **10**, **13**, **3**, **30**, **15**, **12**, **7**, **40** e **20** respectivamente

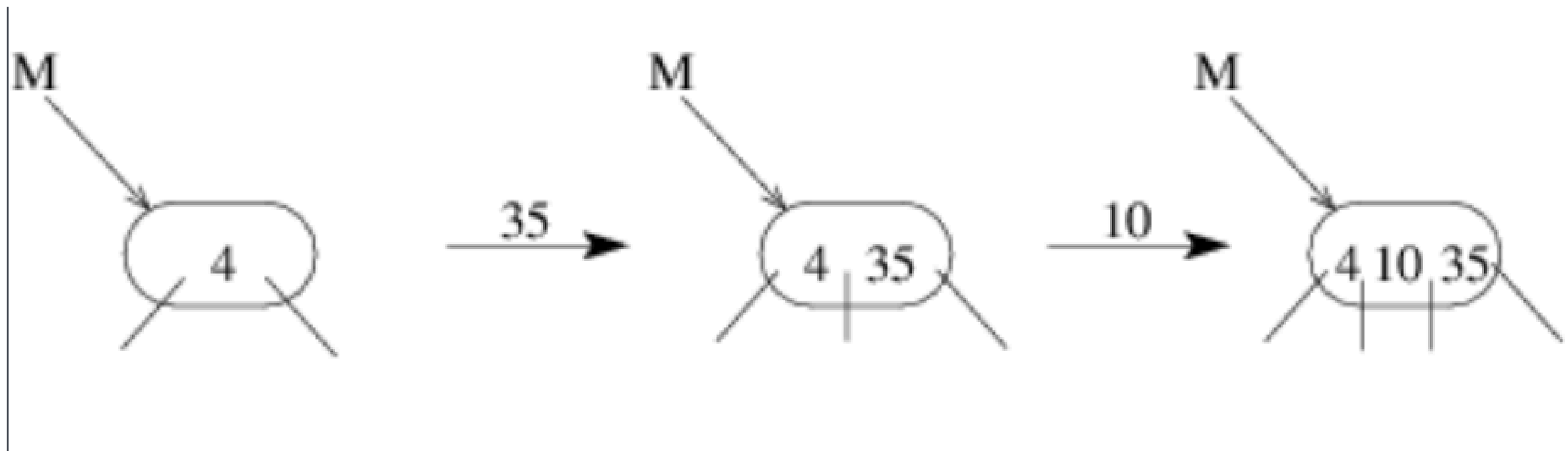


Árvore 2-3-4

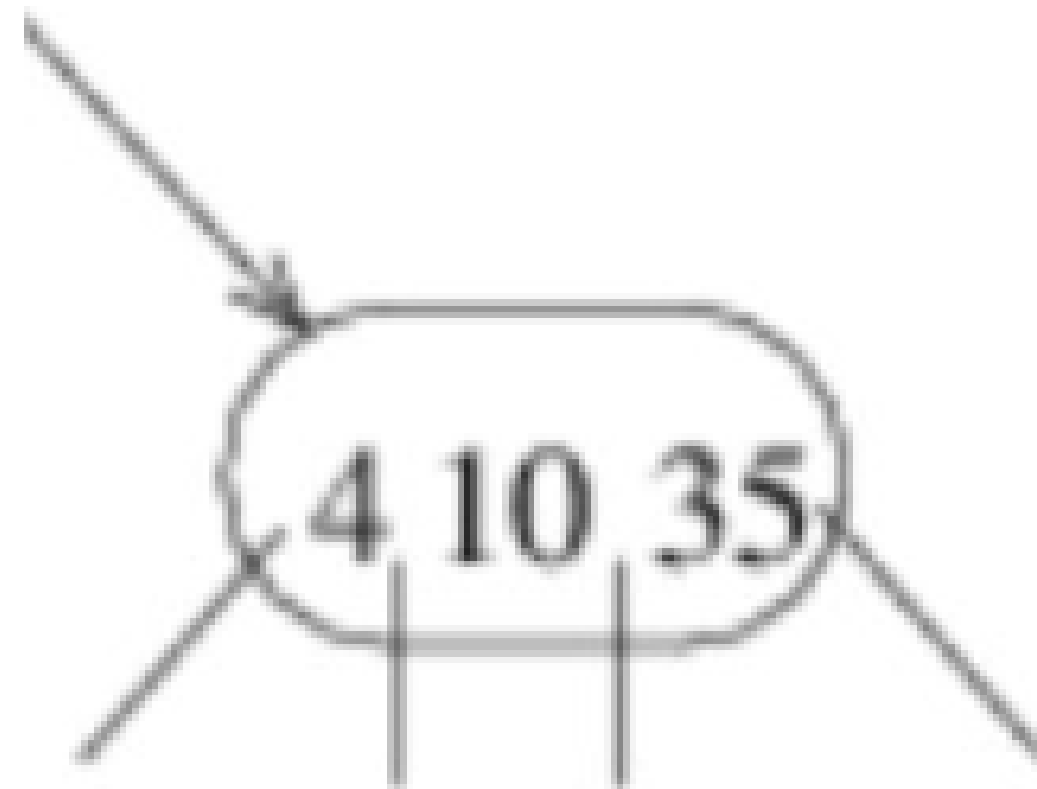


- Apresentaremos esta técnica fazendo, como exemplo, a inserção dos números 4, 35, 10, 13, 3, 30, 15, 12, 7, 40, 20, 11 e 6

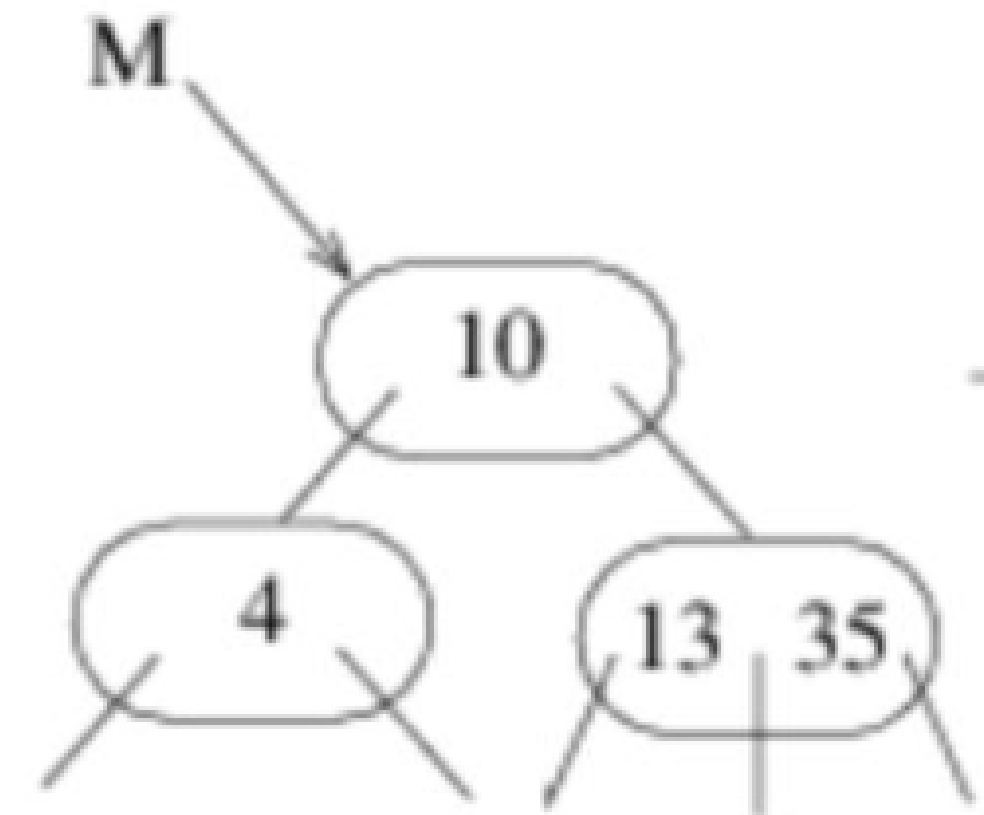
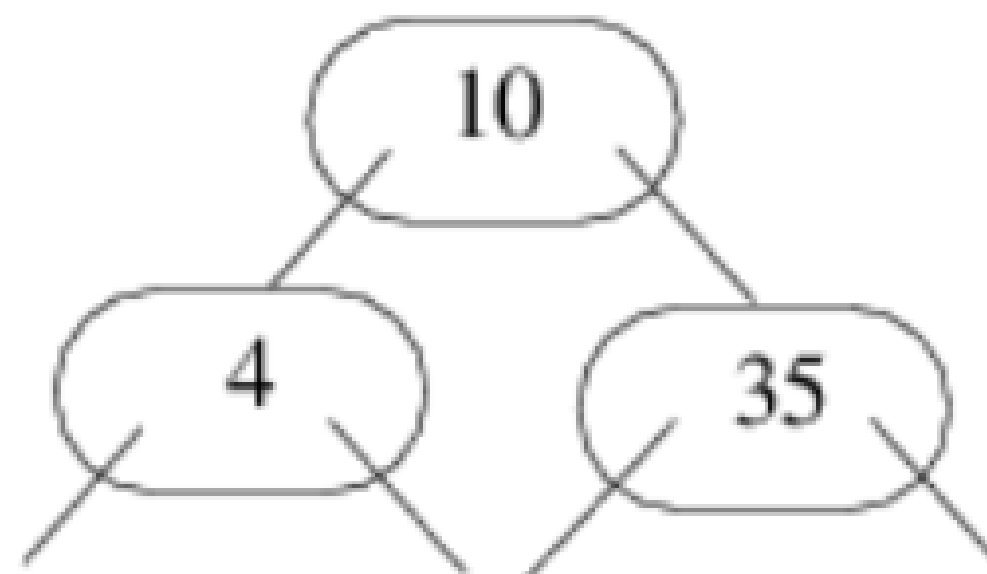
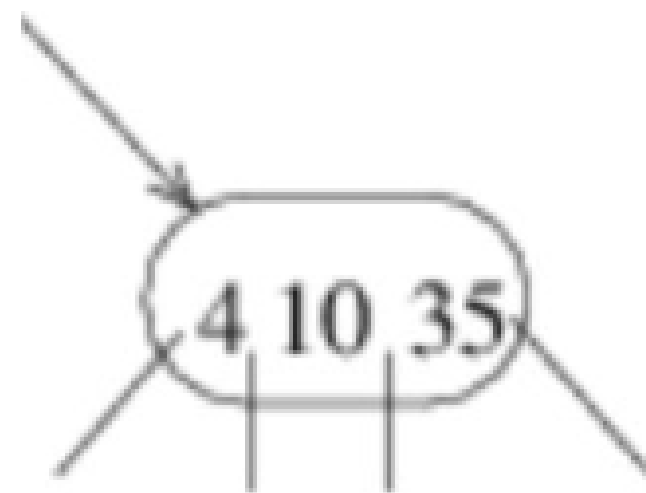
Elementos: **4**, **35**, **10**, 13, 3, 30, 15, 12, 7, 40, 20, 11 e 6



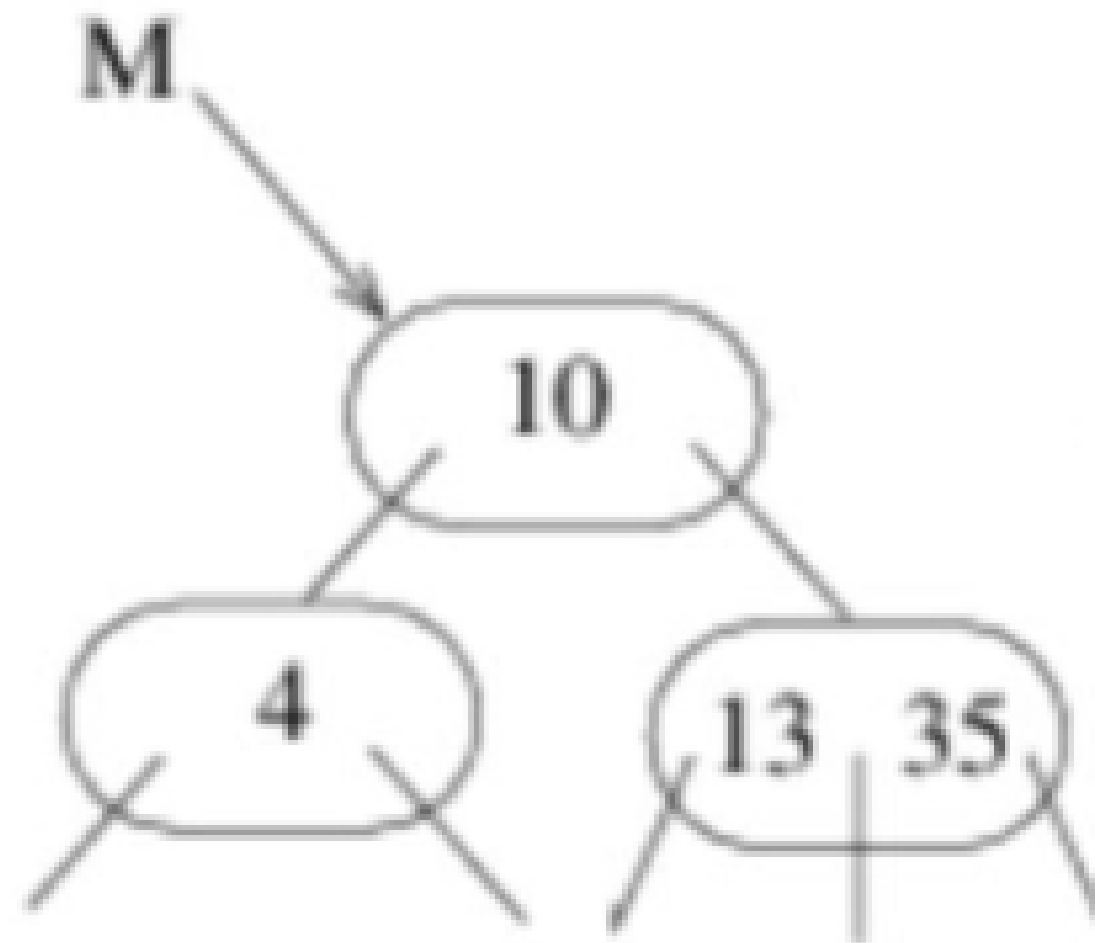
Elementos: 4, 35, 10, **13**, 3, 30, 15, 12, 7, 40, 20, 11 e 6



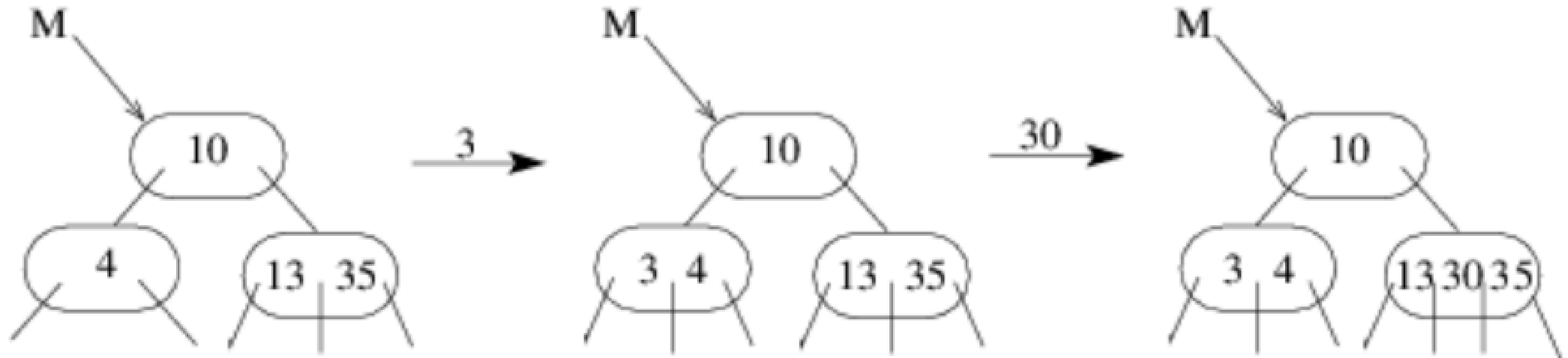
Elementos: 4, 35, 10, **13**, 3, 30, 15, 12, 7, 40, 20, 11 e 6



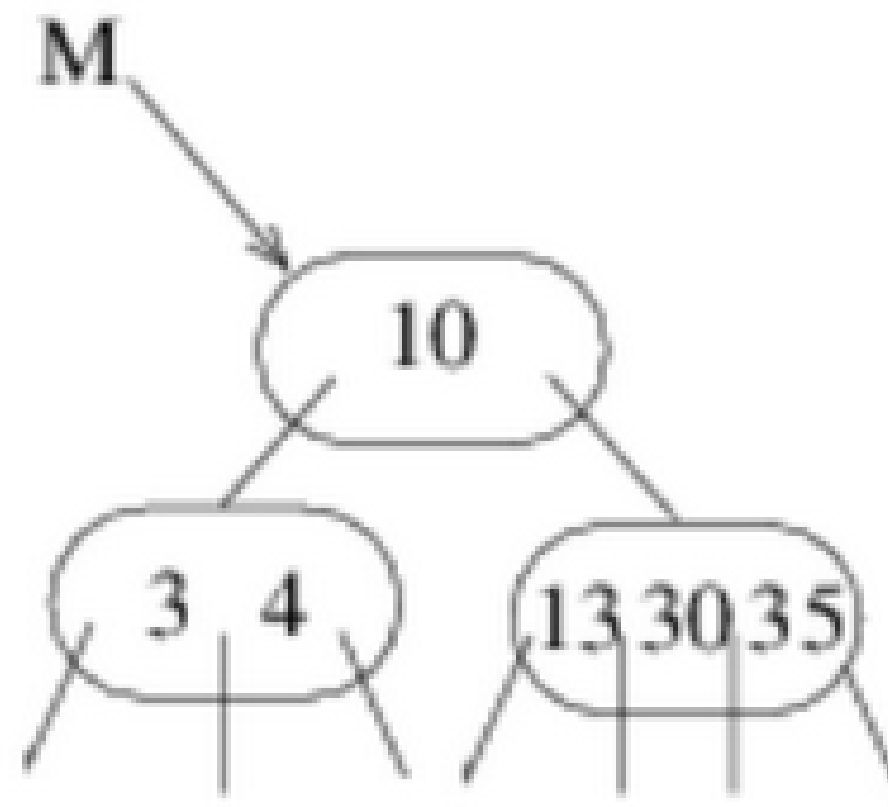
Elementos: 4, 35, 10, 13, **3**, **30**, 15, 12, 7, 40, 20, 11 e 6



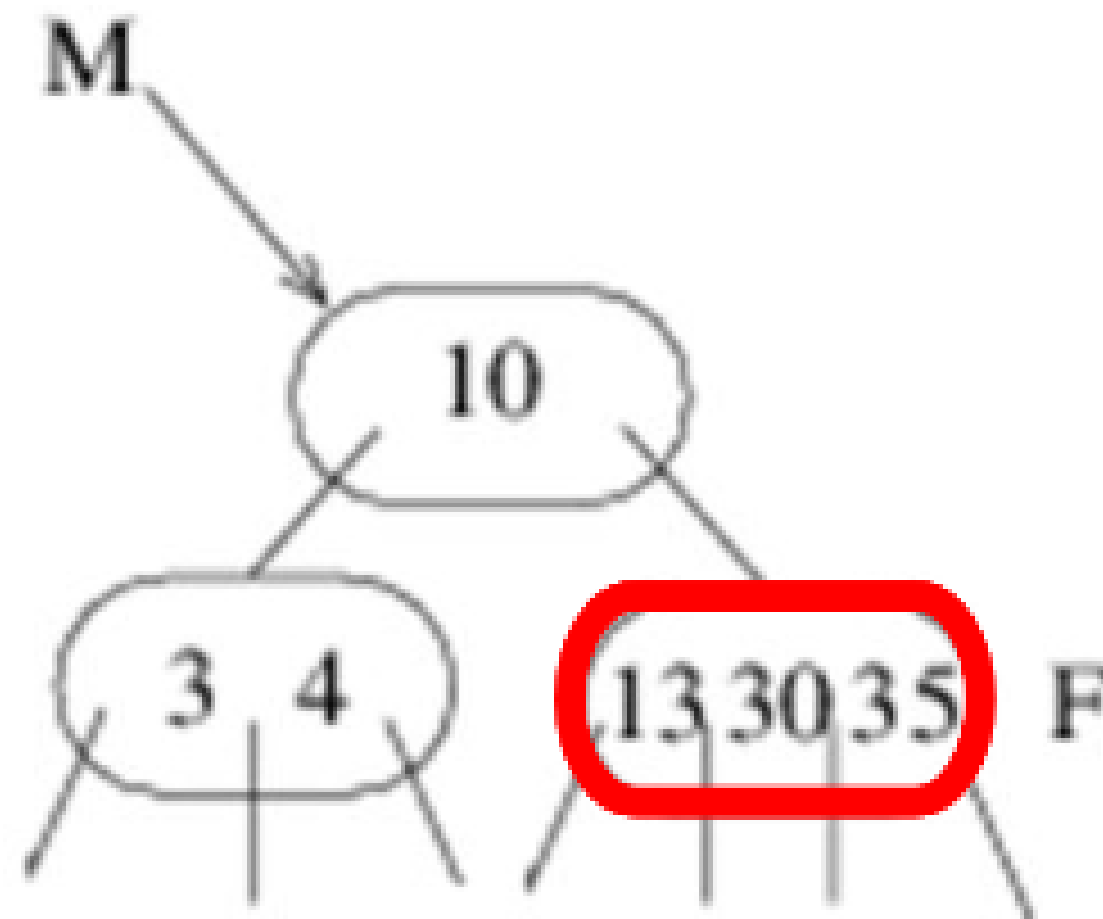
Elementos: 4, 35, 10, 13, **3**, **30**, 15, 12, 7, 40, 20, 11 e 6



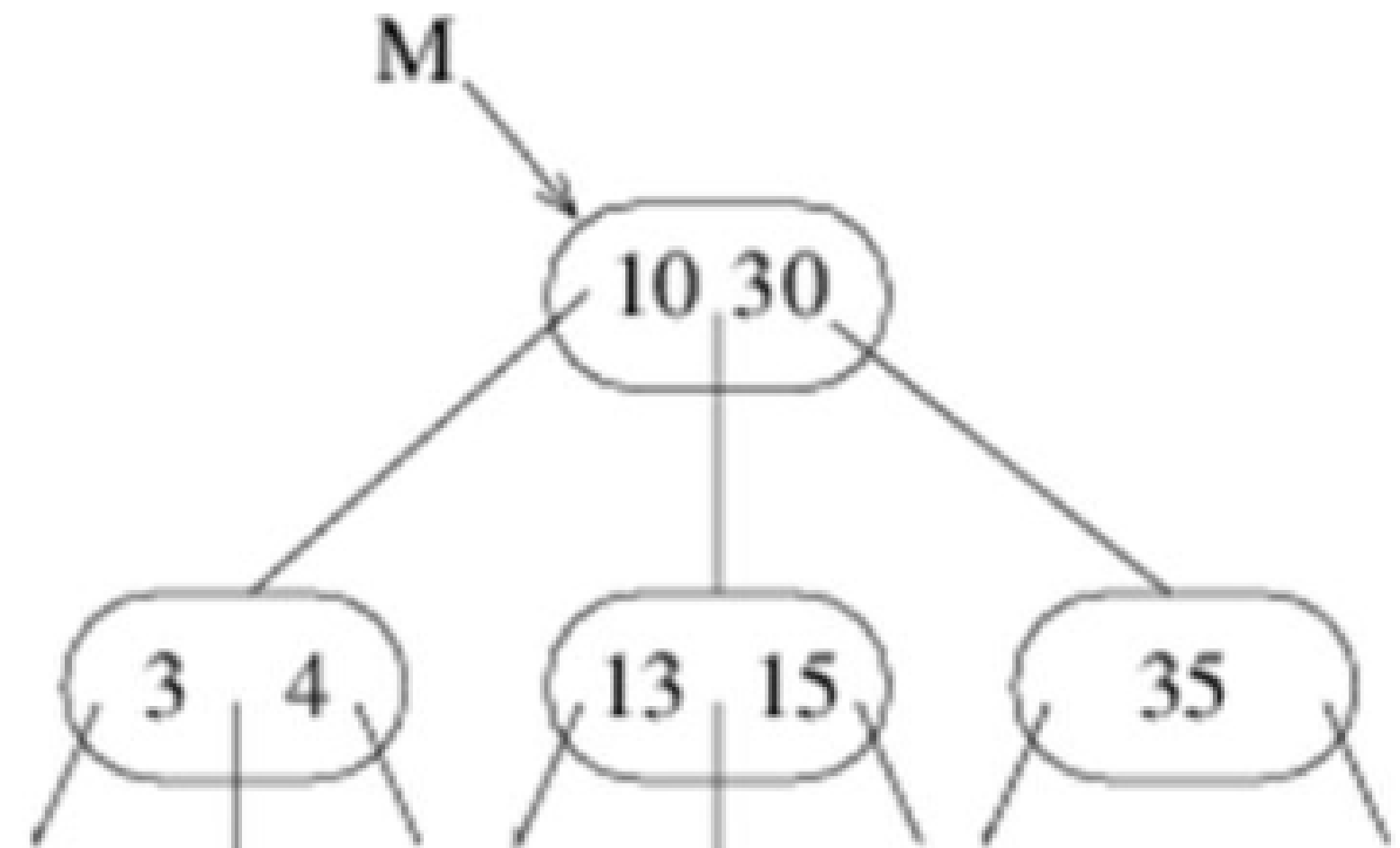
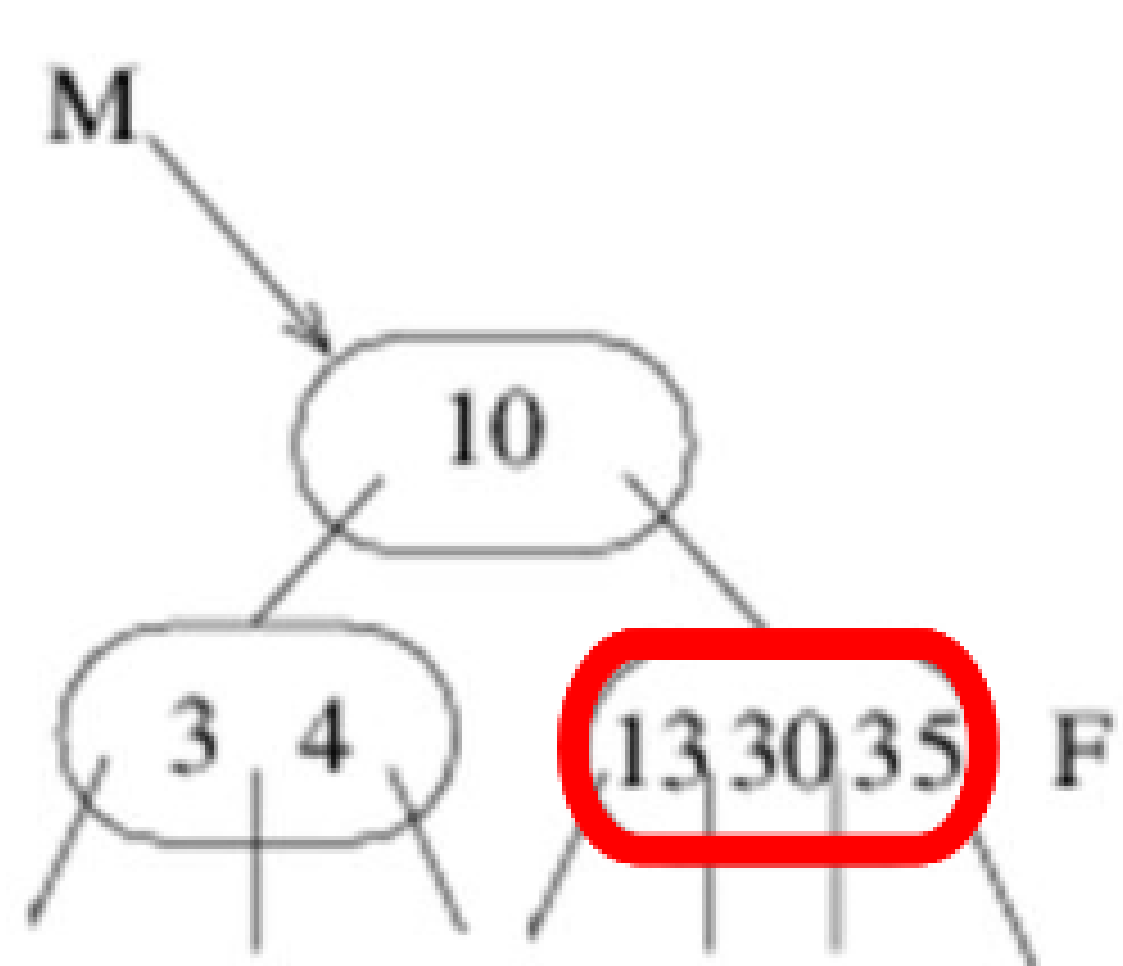
Elementos: 4, 35, 10, 13, 3, 30, **15**, 12, 7, 40, 20, 11 e 6



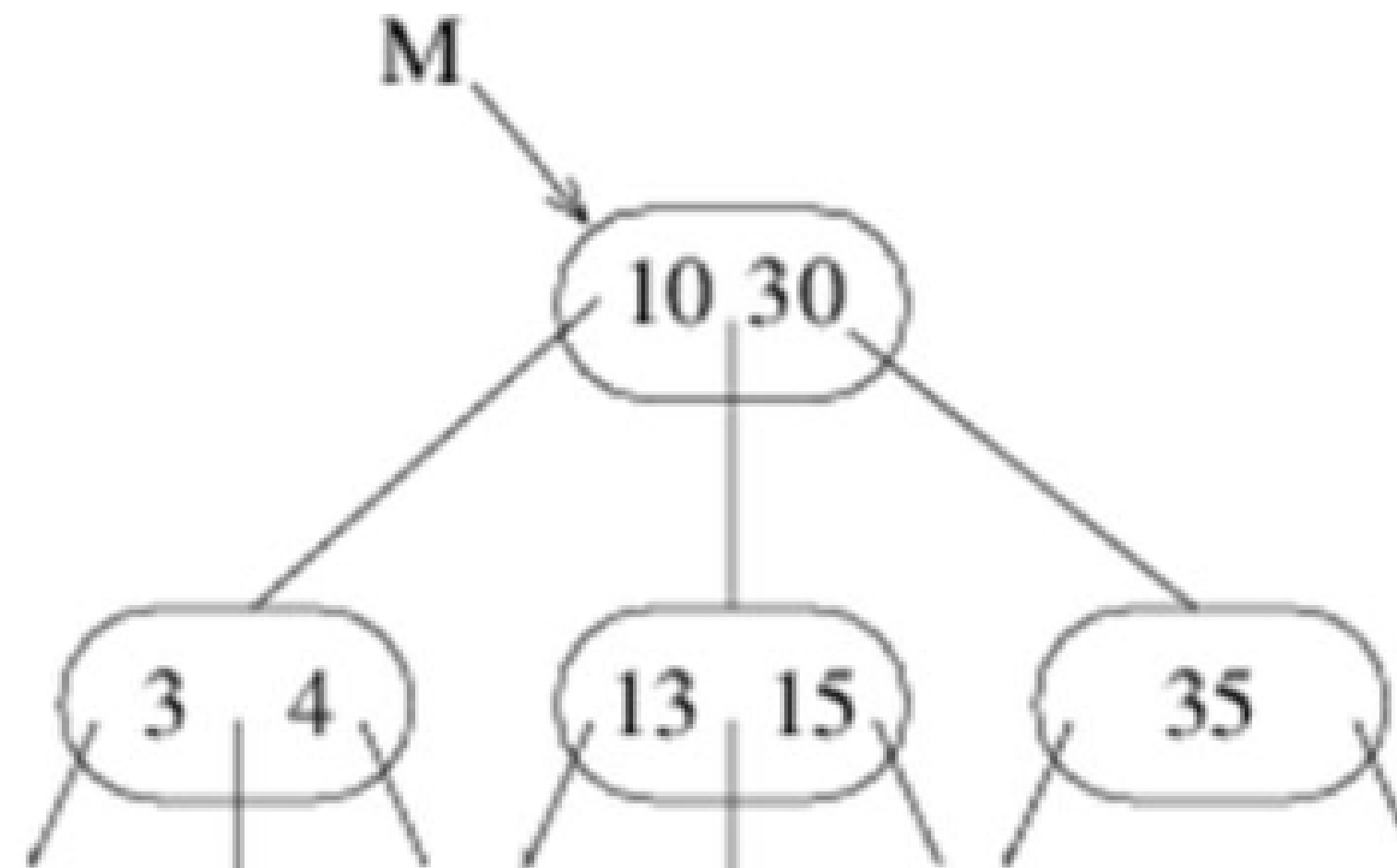
Elementos: 4, 35, 10, 13, 3, 30, **15**, 12, 7, 40, 20, 11 e 6



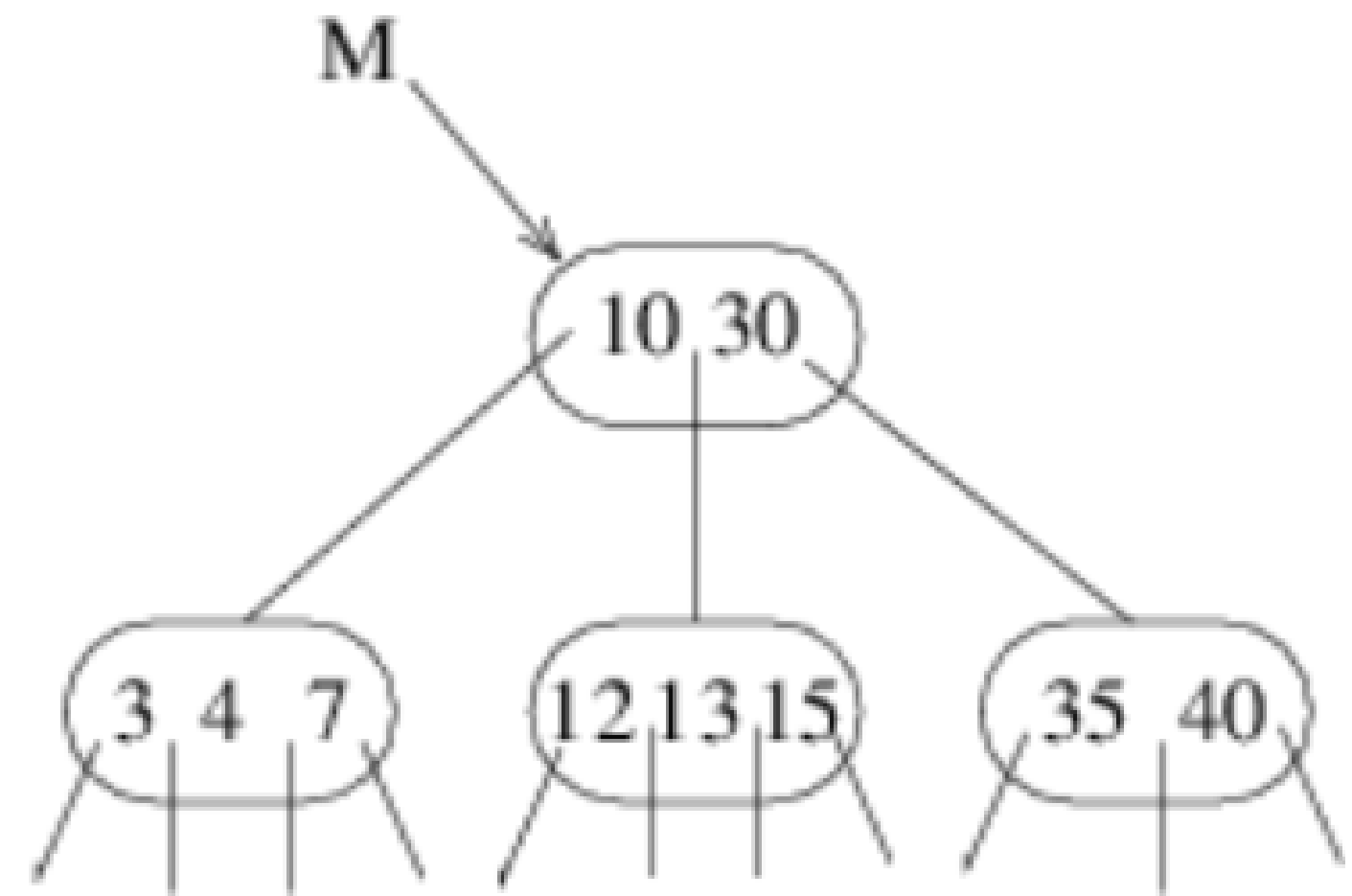
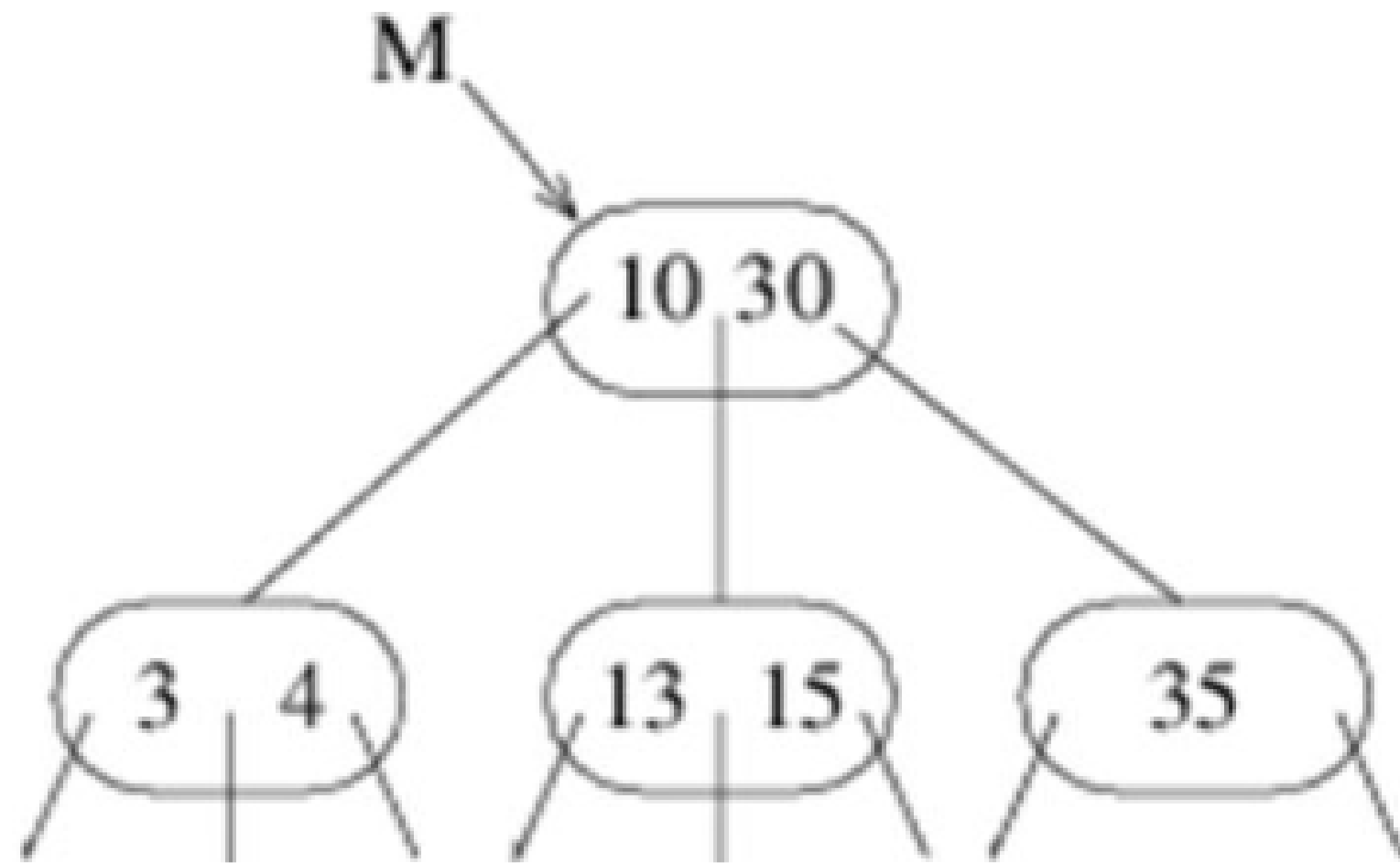
Elementos: 4, 35, 10, 13, 3, 30, **15**, 12, 7, 40, 20, 11 e 6



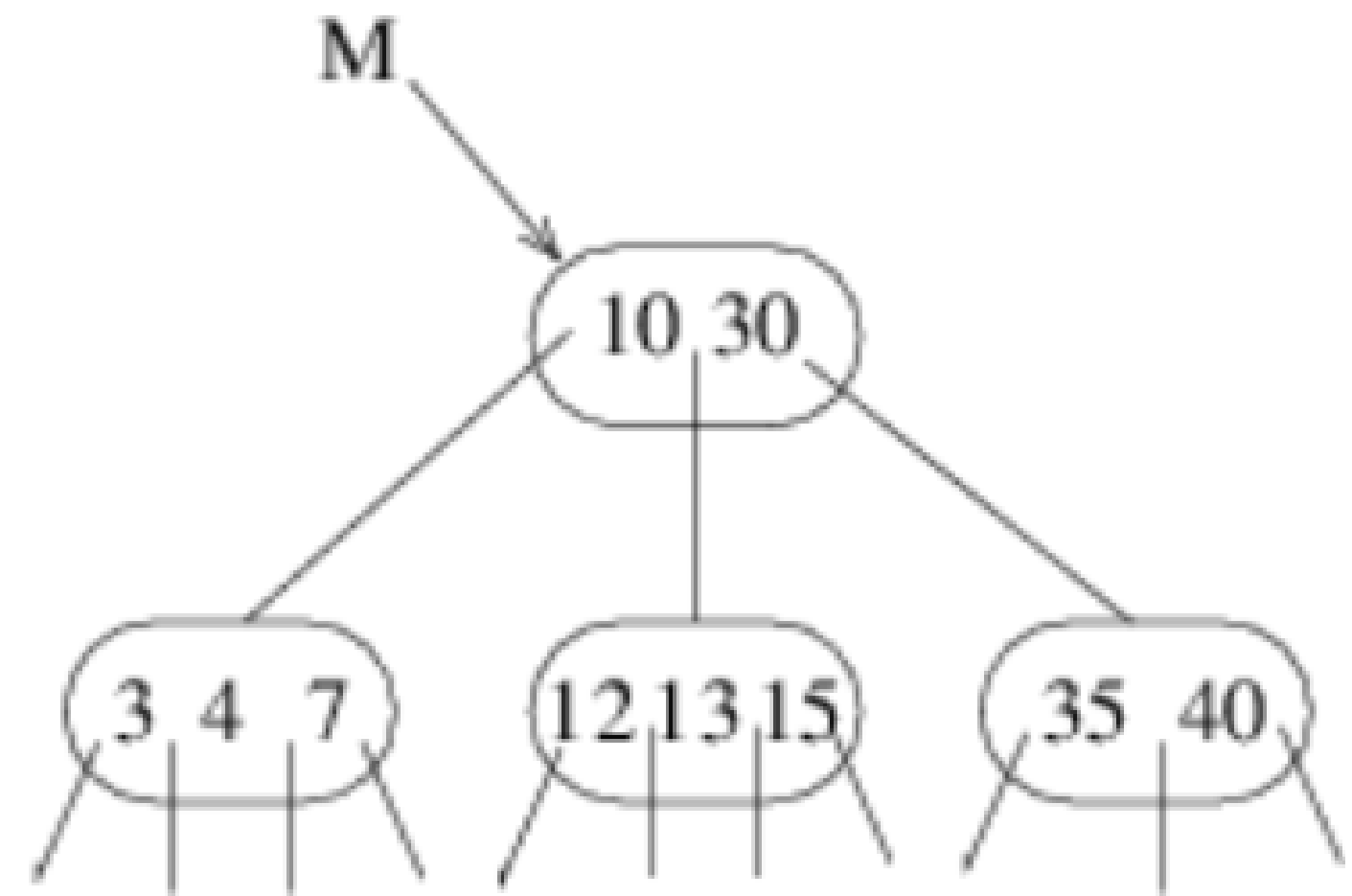
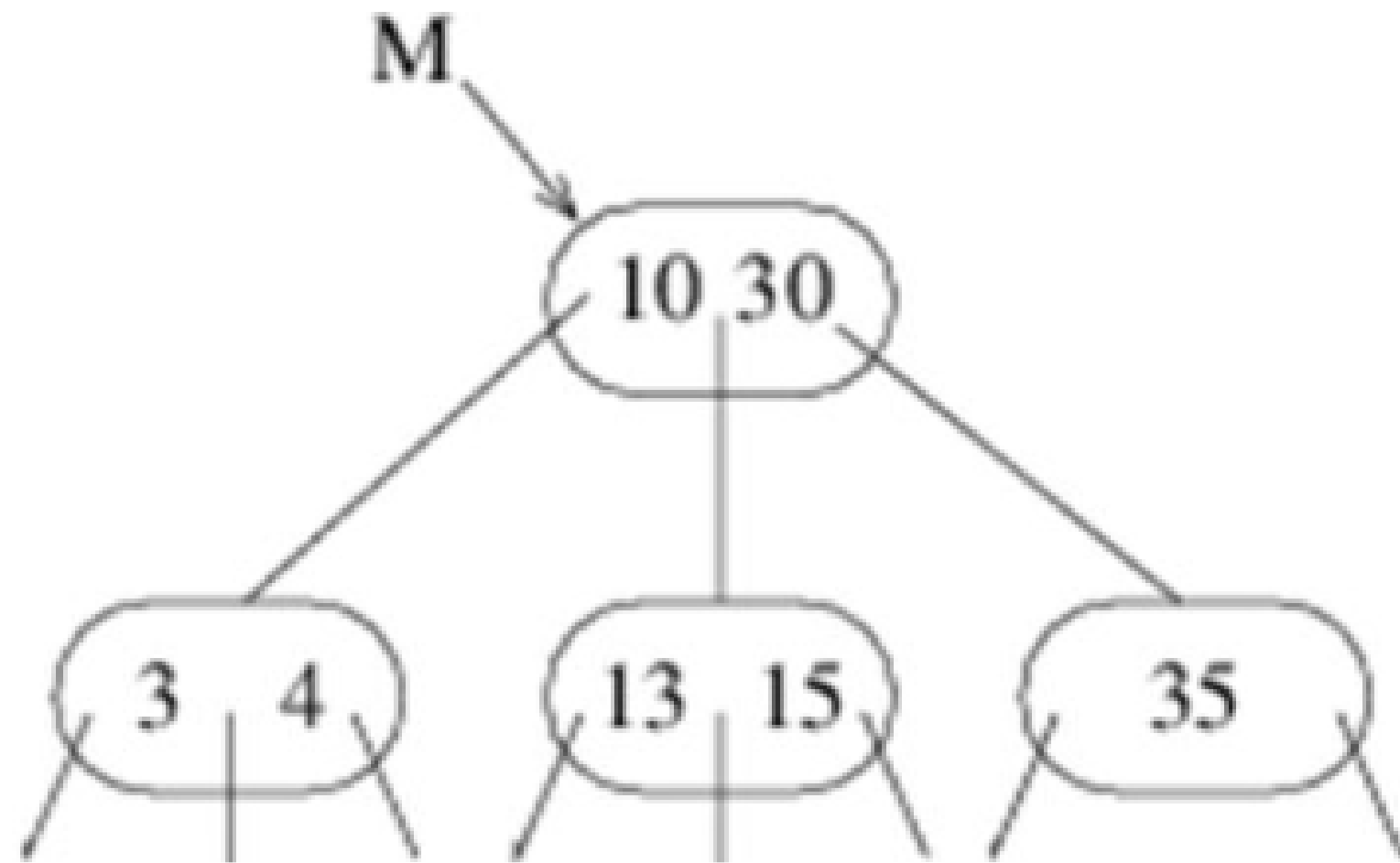
Elementos: 4, 35, 10, 13, 3, 30, 15, **12**, **7**, **40**, 20, 11 e 6



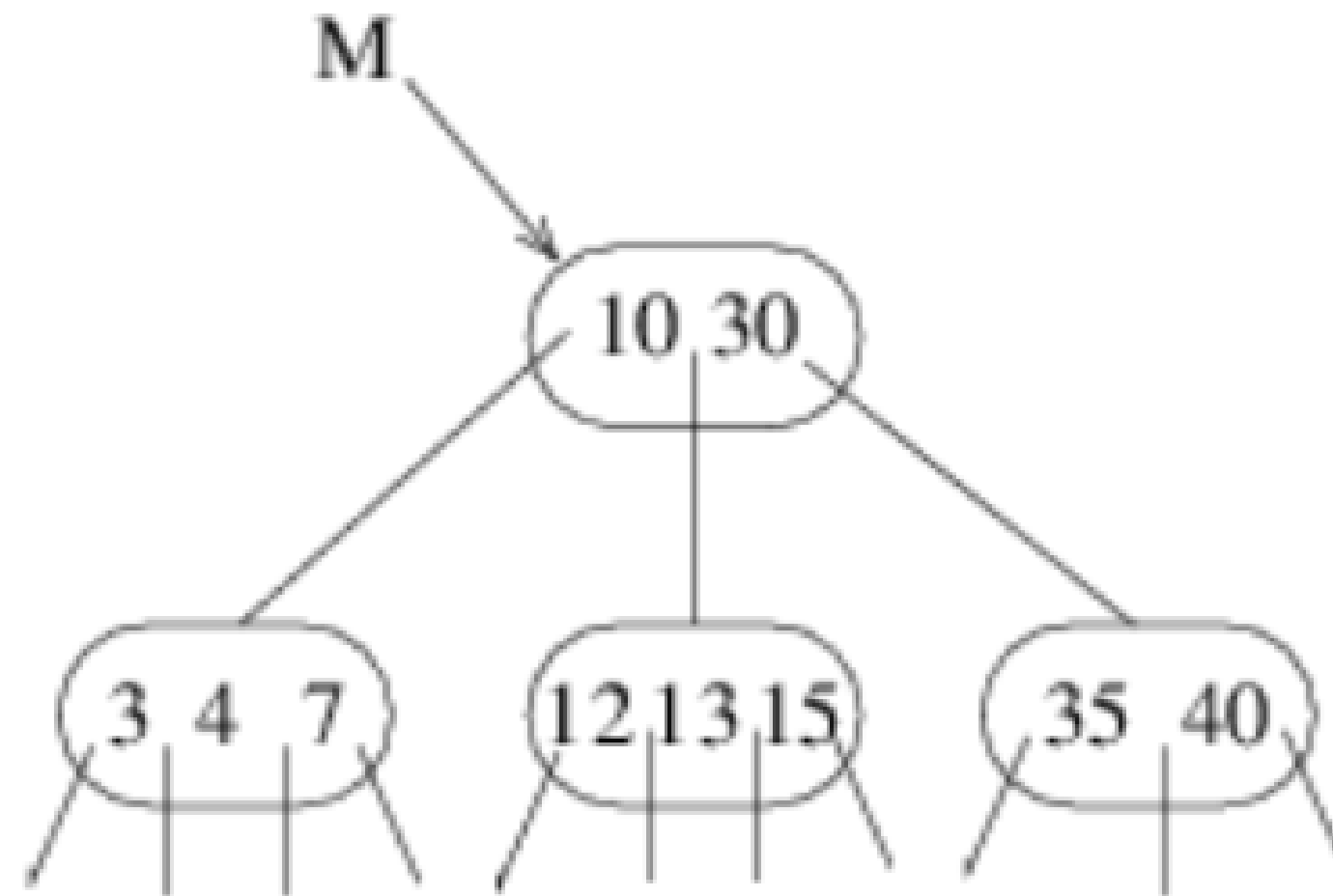
Elementos: 4, 35, 10, 13, 3, 30, 15, **12**, **7**, **40**, 20, 11 e 6



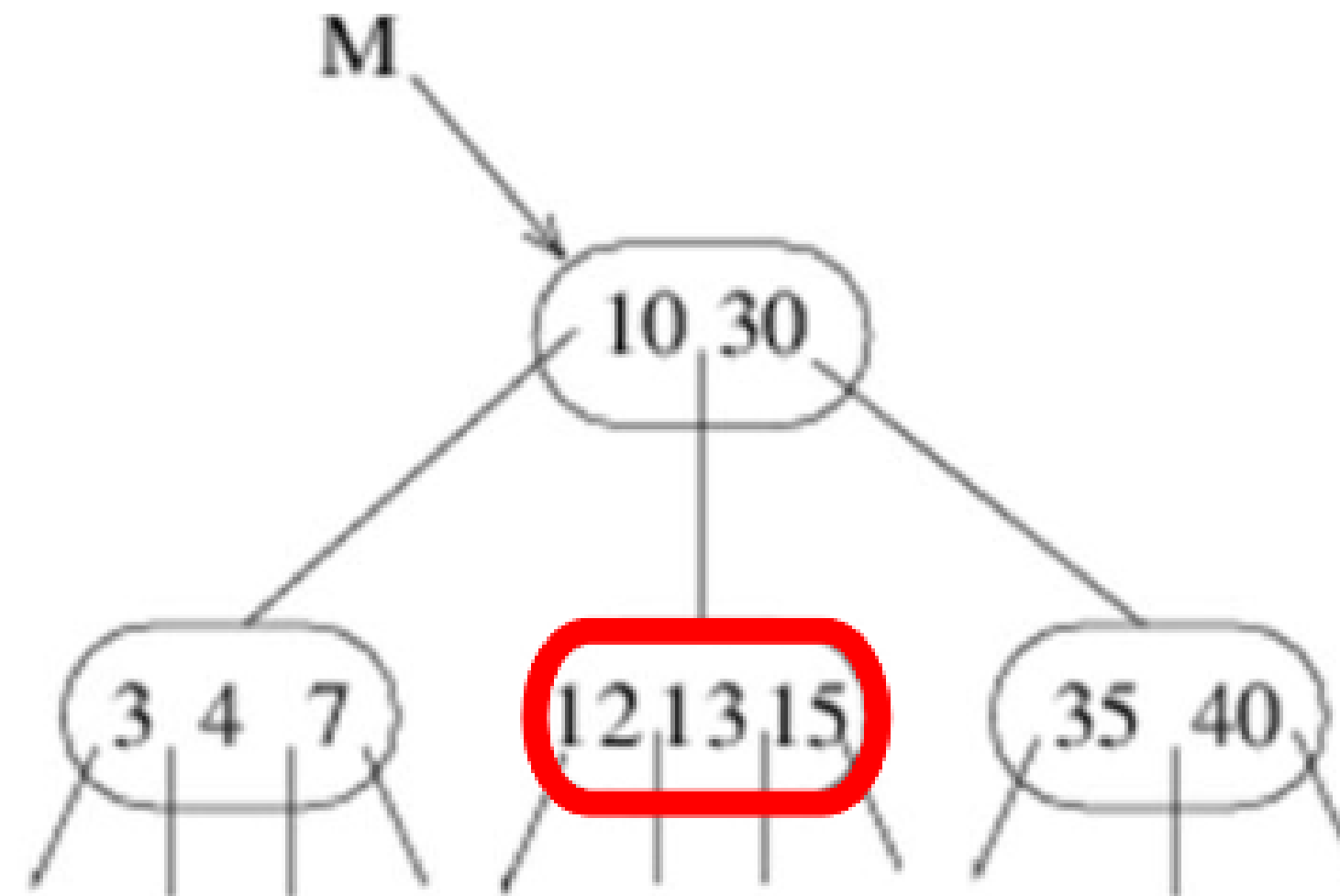
Elementos: 4, 35, 10, 13, 3, 30, 15, **12**, **7**, **40**, 20, 11 e 6



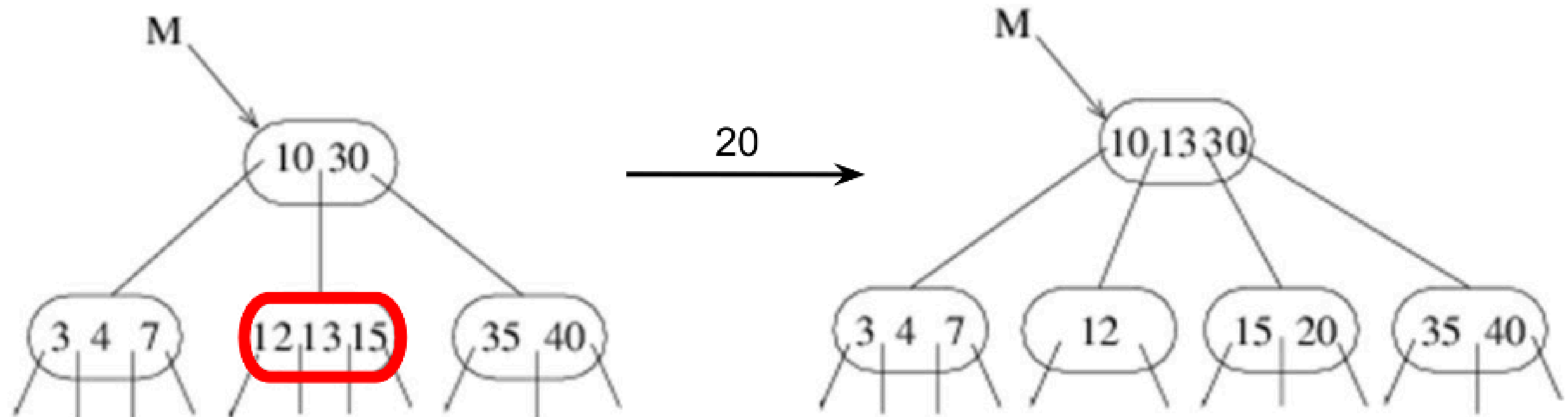
Elementos: 4, 35, 10, 13, 3, 30, 15, 12, 7, 40, **20**, 11 e 6



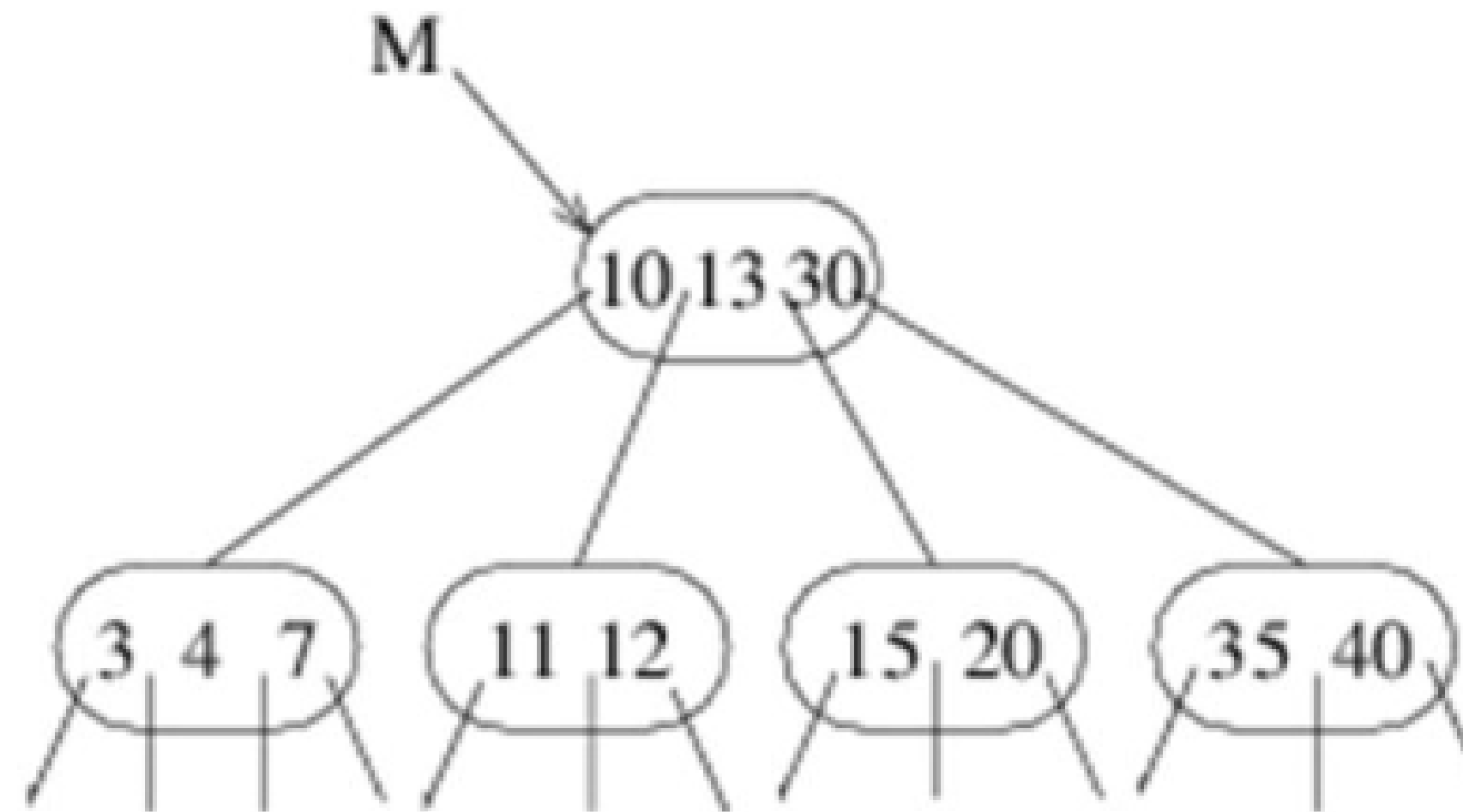
Elementos: 4, 35, 10, 13, 3, 30, 15, 12, 7, 40, **20**, 11 e 6



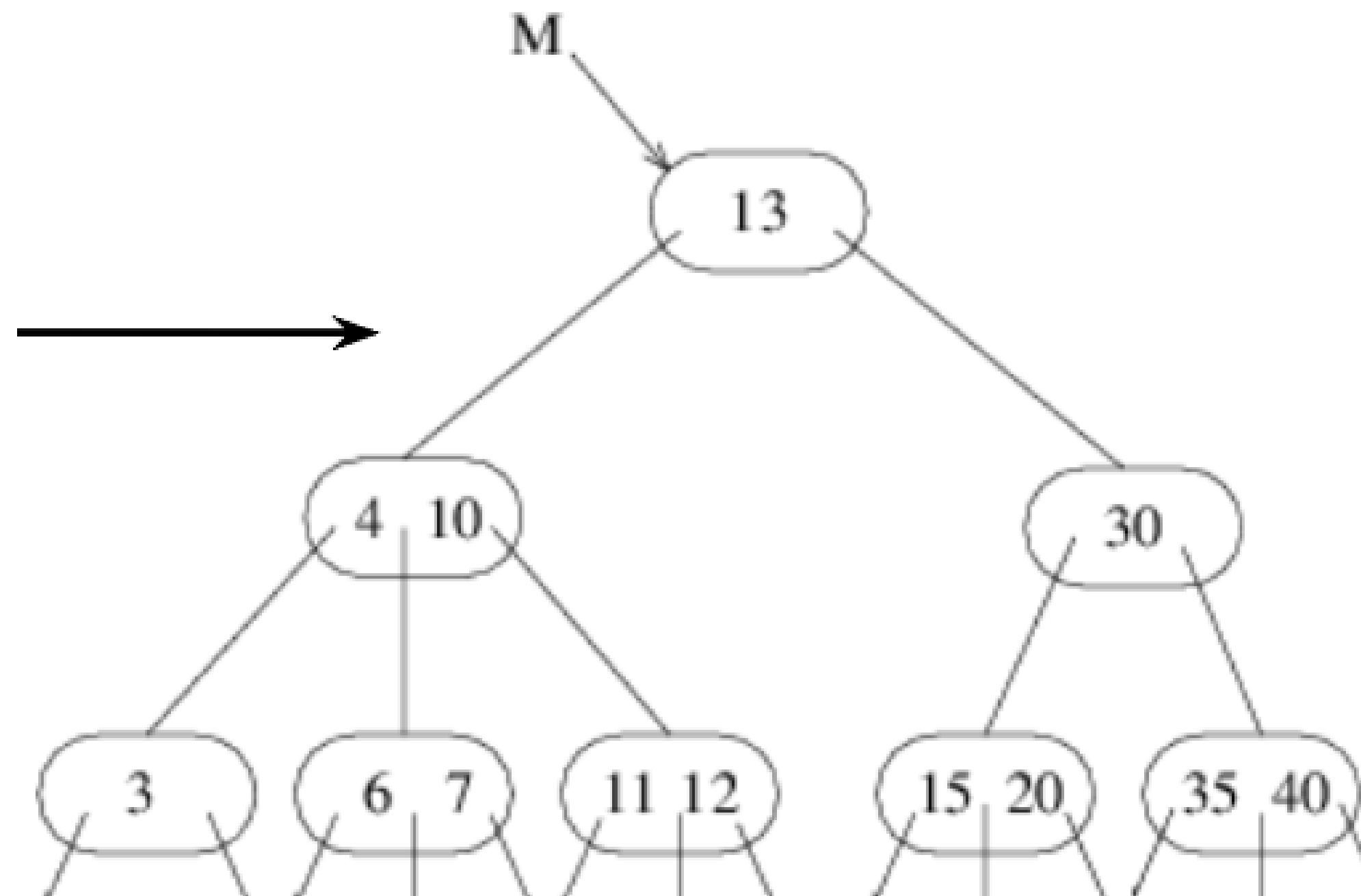
Elementos: 4, 35, 10, 13, 3, 30, 15, 12, 7, 40, **20**, 11 e 6



Elementos: 4, 35, 10, 13, 3, 30, 15, 12, 7, 40, 20, 11 e **6**



Elementos: 4, 35, 10, 13, 3, 30, 15, 12, 7, 40, 20, 11 e **6**



Árvore Alvinegra

Árvore Alvinegra - Classe Nó

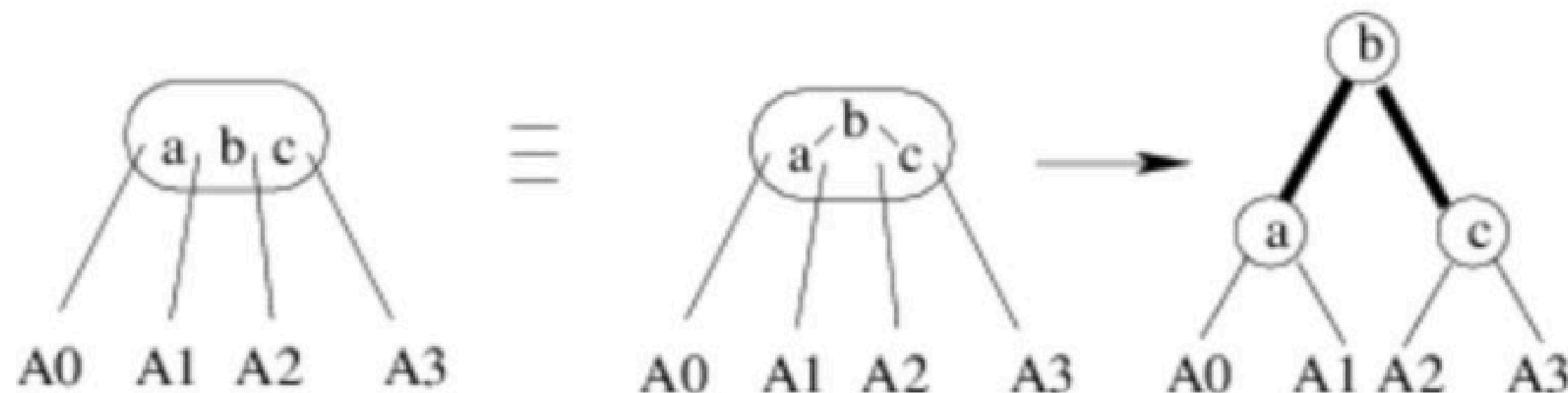
```
class NoAN{
    public boolean cor;
    public int elemento;
    public NoAN esq, dir;
    public NoAN (){
        this(-1);
    }
    public NoAN (int elemento){
        this(elemento, false, null, null);
    }
    public NoAN (int elemento, boolean cor){
        this(elemento, cor, null, null);
    }
    public NoAN (int elemento, boolean cor, NoAN esq, NoAN dir){
        this.cor = cor;
        this.elemento = elemento;
        this.esq = esq;
        this.dir = dir;
    }
}
```

Árvore Alvinegra

- Simula a hierarquia 2.3.4, colorindo as arestas entre os nós de duas formas:

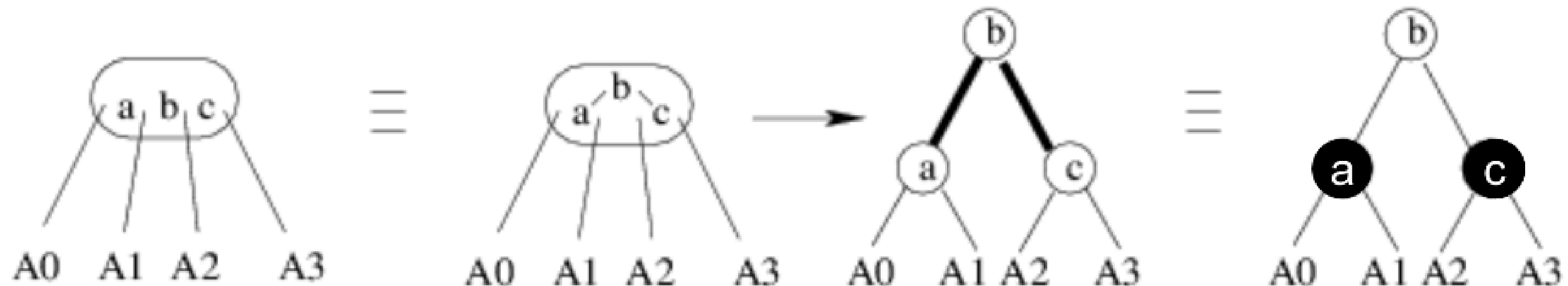
- **Aresta preta (traço grosso):** Entre gêmeos (mesmo nó na 2.3.4)

- **Aresta branca (traço fino):** Entre não gêmeos na 2.3.4



Árvore Alvinegra

- Nossa atribuição de cores, na verdade, será nos nós em vez das arestas:
- **Nó preto:** Nó gêmeo do pai na 2.3.4
- **Nó branco:** Nó **NÃO** gêmeo do pai na 2.3.4



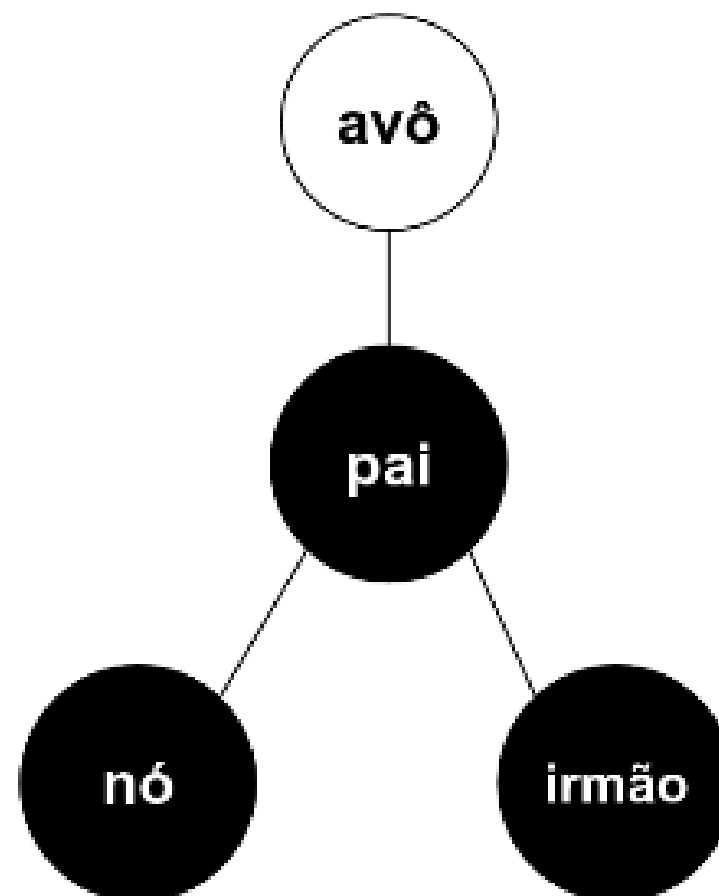
Árvore Alvinegra

- Qual é o número máximo de nós pretos consecutivos?

Árvore Alvinegra

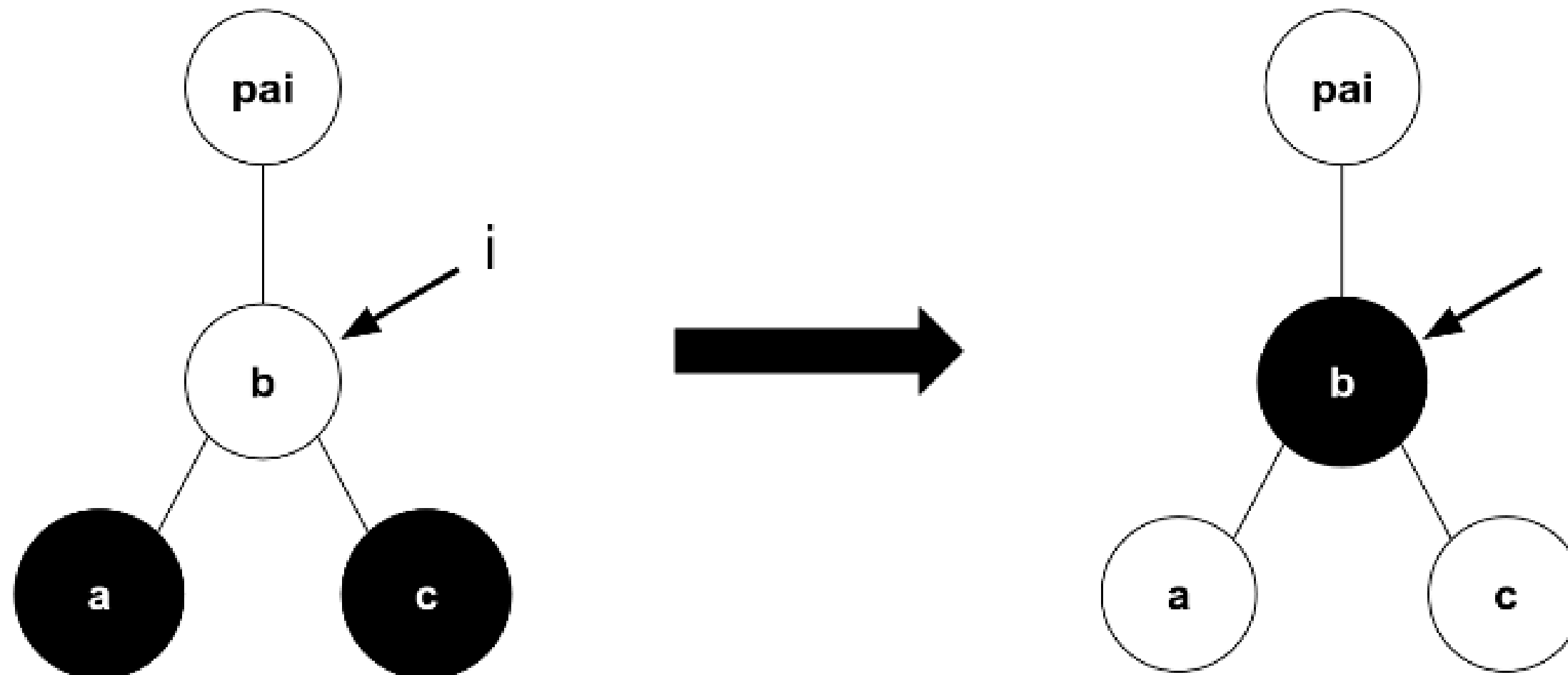
- Qual é o número máximo de nós pretos consecutivos?

Resposta: Nunca teremos nós pretos consecutivos. Isso significaria mais de três elementos em um nó 2.3.4. Nesse caso, o nó e seu irmão seriam gêmeos do pai e esse, gêmeo do avô

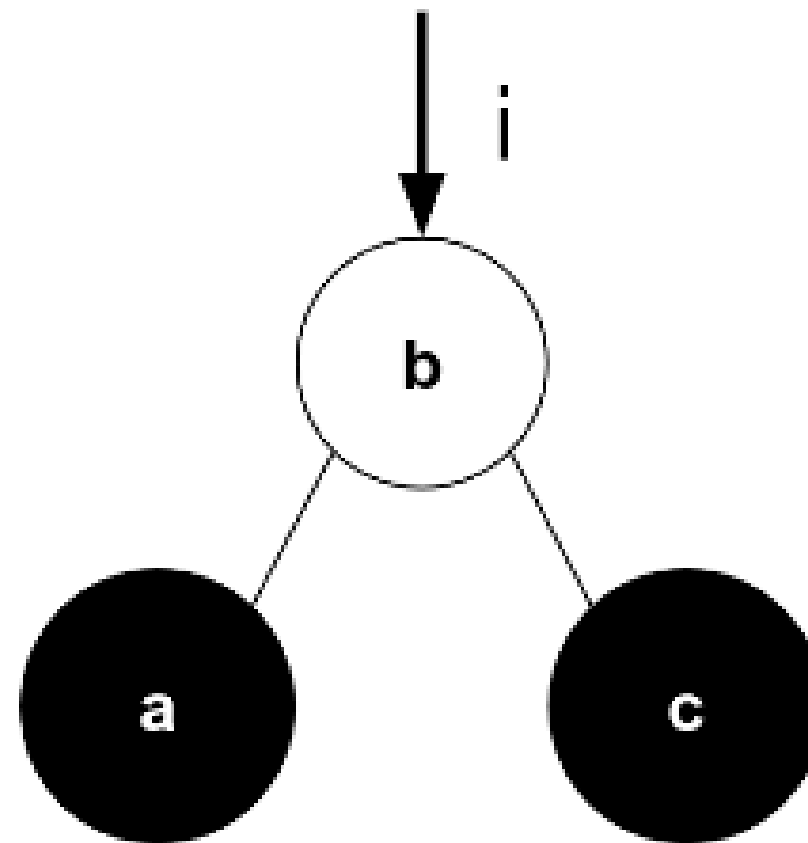


Árvore Alvinegra

- Pode ser feita simulando a inserção com fragmentação na descida na 2.3.4
- A fragmentação de um 4-nó corresponde a inversão de cores dos elementos gêmeos



Árvore Alvinegra

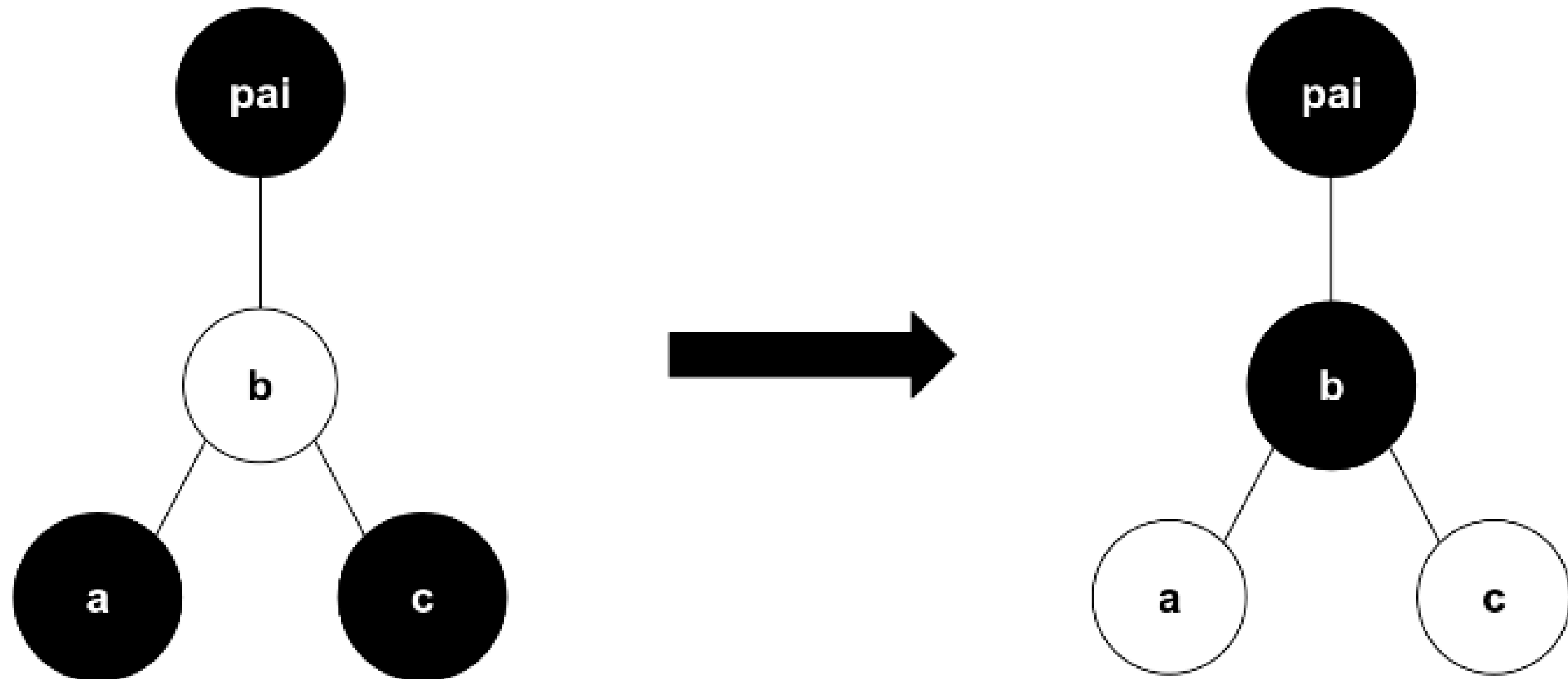


```
//supondo que seja um nó do tipo quatro  
void fragmentarNoTipoQuatro(NoAN i){  
    i.cor = !i.cor;  
    i.esq.cor = !i.esq.cor;  
    i.dir.cor = !i.dir.cor;  
}
```

```
//supondo que seja um nó do tipo quatro  
void fragmentarNoTipoQuatro(NoAN i){  
    i.cor = true;  
    i.esq.cor = false;  
    i.dir.cor = false;  
}
```

Árvore Alvinegra Efeito Colateral

- Acontece quando o pai do nó(b) é preto



Árvore Alvinegra Balanceamento

- A cor preta indica uma tendência de inserção e, conseqüentemente, dois nós pretos consecutivos indica necessidade de balanceamento
- Após a inversão de cores ou a inserção em uma folha, se o pai do nó também for preto, rotacionamos seu avô considerando o **alinhamento entre avô, pai e nó**

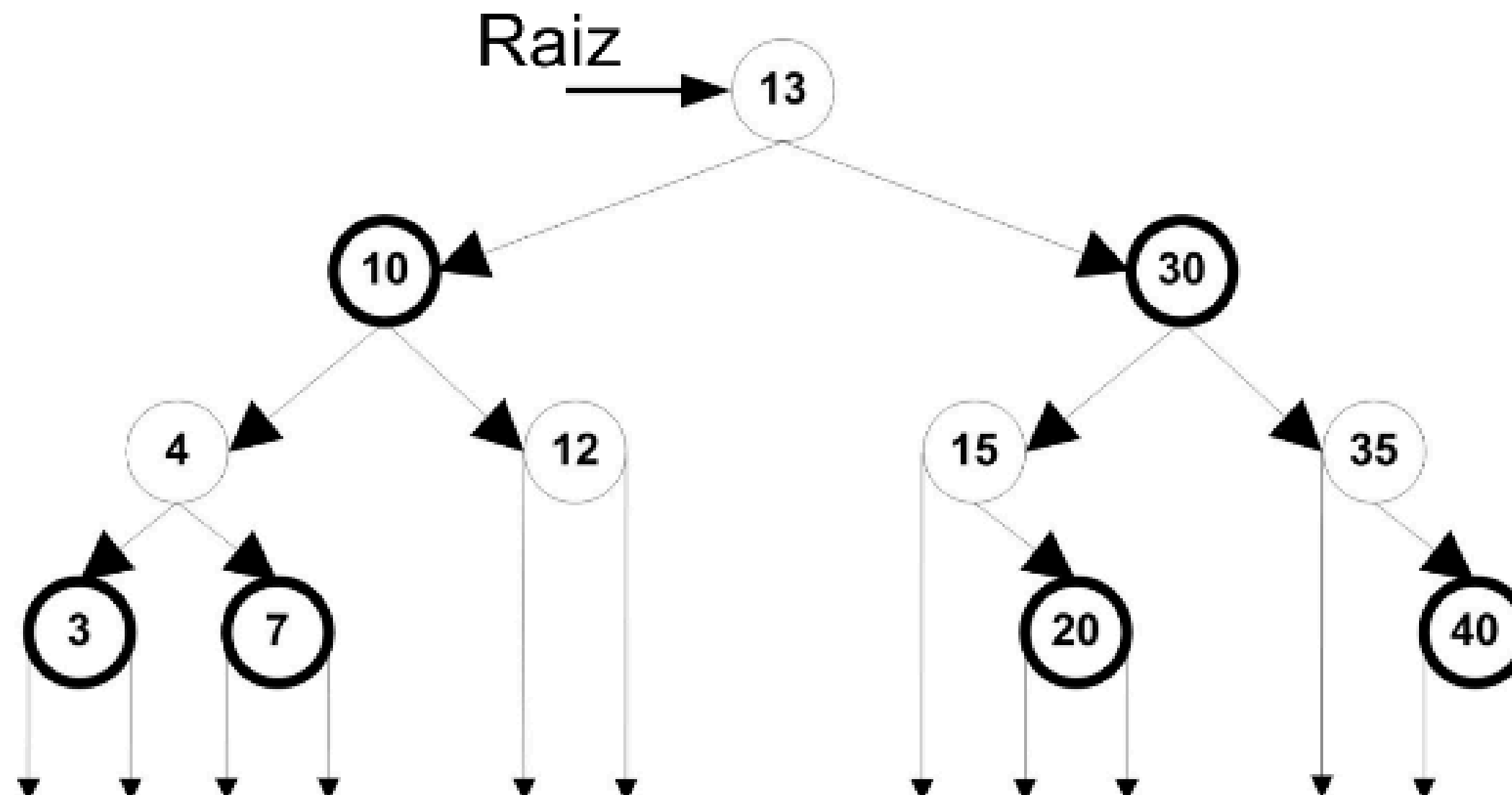
Árvore Alvinegra Balanceamento

- Crie uma árvore alvinegra através de inserções sucessivas do 4, 35, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente

Árvore Alvinegra Balanceamento

- Crie uma árvore alvinegra através de inserções sucessivas do 4, 35, 10, 13, 3, 30, 15, 12, 7, 40 e 20 respectivamente

Árvore Alvinegra Balanceamento



Estrutura híbrida (ou Doidona)

Como tratar o elemento?

- Ao invés de criar uma variável `int elem` por exemplo para ser o elemento, cria-se um objeto para se referir ao objeto

A inserção é feita recursivamente.

Exemplo:

Tenho uma estrutura “Doidona” de uma tabela hash T1, que se faltar espaço é dividido em 3 estruturas.

- A 1a é uma tabela hash com rehash, que se faltar espaço leva para uma árvore binária;
- A 2a é uma lista flexível que adiciona os elementos sempre no final da lista;
- E a 3a é uma árvore AVL

```
class Doidona{
    Objeto T1[6];
    Objeto T3[5];
    Arvore arvoreBinaria;
    Lista lista;
    AVL arvoreAVL;
}
```

```
class Objeto{
    int elem;
    public Objeto(int x){
        this.elem = x;
    }
}
```

```
class Arvore{
    Objeto elem;
    Arvore dir, esq;
}
```

```
class Celula{
    Objeto elem;
    Celula prox;
}
```

```
class Lista{
    Celula primeira;
    Celula ultima;
}
```

```
class AVL{
    Objeto elem;
    AVL dir, esq;
}
```

```

public void inserir(Objeto objeto){
    int elem = objeto.elem;
    int i = elem % 6; //hash da T1
    if(T1[i] == null){
        T1[i] = elem;
    }else{
        i = elem % 3; //hash da T2
        if(i == 0){
            i = elem % 5; //hash da T3
            if(T3[i] == null){
                T3[i] = elem;
            } else {
                i = elem - 1 % 5; //rehash da T3
                if(t3[i] == null){
                    t3[i] = elem;
                } else {
                    arvoreBinaria.inserir(objeto);
                }
            }
        }
        }else if (i == 1){
            lista.inserirFim(objeto);
        }else{ //if (i == 2)
            arvoreAVL.inserir(objeto);
        }
    }
}

```

```

public boolean search(int elem){
    int i = elem % 6; //hash da T1
    if(T1[i] == null){
        return false;
    } else if(T1[i].elem == elem){
        return true;
    } else {
        i = elem % 3; //hash da T2
        if(i == 0){
            i = elem % 5; //hash da T3
            if(T3[i] == null){
                return false;
            } else if (T3[i].elem == elem){
                return true;
            } else {
                i = elem - 1 % 5; //rehash da T3
                if(T3[i] == null){
                    return false;
                } else if(T3[i].elem == elem){
                    return true;
                }else {
                    arvoreBinaria.search(elem);
                }
            }
        }
        }else if (i == 1){
            lista.search(elem);
        }else{ //if (i == 2)
            arvoreAVL.search(elem);
        }
    }
}

```

```

public void inserir(Objeto objeto){
    int elem = objeto.elem;
    int i = elem % 6; //hash da T1
    if(T1[i] == null){
        T1[i] = elem;
    }else{
        i = elem % 3; //hash da T2
        if(i == 0){
            i = elem % 5; //hash da T3
            if(T3[i] == null){
                T3[i] = elem;
            } else {
                i = elem - 1 % 5; //rehash da T3
                if(t3[i] == null){
                    t3[i] = elem;
                } else {
                    arvoreBinaria.inserir(objeto);
                }
            }
        }else if (i == 1){
            lista.inserirFim(objeto);
        }else{ //if (i == 2)
            arvoreAVL.inserir(objeto);
        }
    }
}

```

```

public boolean search(int elem){
    int i = elem % 6; //hash da T1
    if(T1[i] == null){
        return false;
    } else if(T1[i].elem == elem){
        return true;
    } else {
        i = elem % 3; //hash da T2
        if(i == 0){
            i = elem % 5; //hash da T3
            if(T3[i] == null){
                return false;
            } else if (T3[i].elem == elem){
                return true;
            } else {
                i = elem - 1 % 5; //rehash da T3
                if(T3[i] == null){
                    return false;
                } else if(T3[i].elem == elem){
                    return true;
                }else {
                    arvoreBinaria.search(elem);
                }
            }
        }else if (i == 1){
            lista.search(elem);
        }else{ //if (i == 2)
            arvoreAVL.search(elem);
        }
    }
}

```

Exemplo:

T1 (Tabela hash) - elem % 6

0	
1	
2	
3	
4	
5	

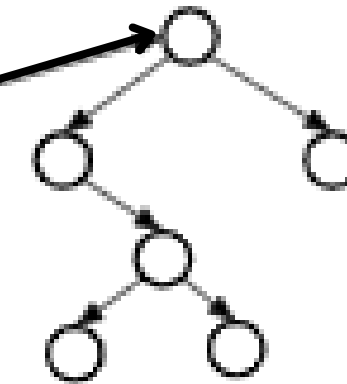
T2 - elem % 3

0
1
2

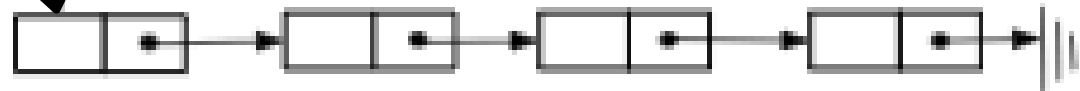
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	
1	
2	
3	
4	

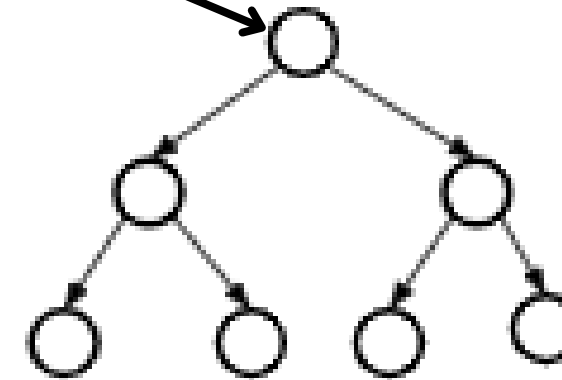
Árvore binária



Lista flexível (inserir no final)

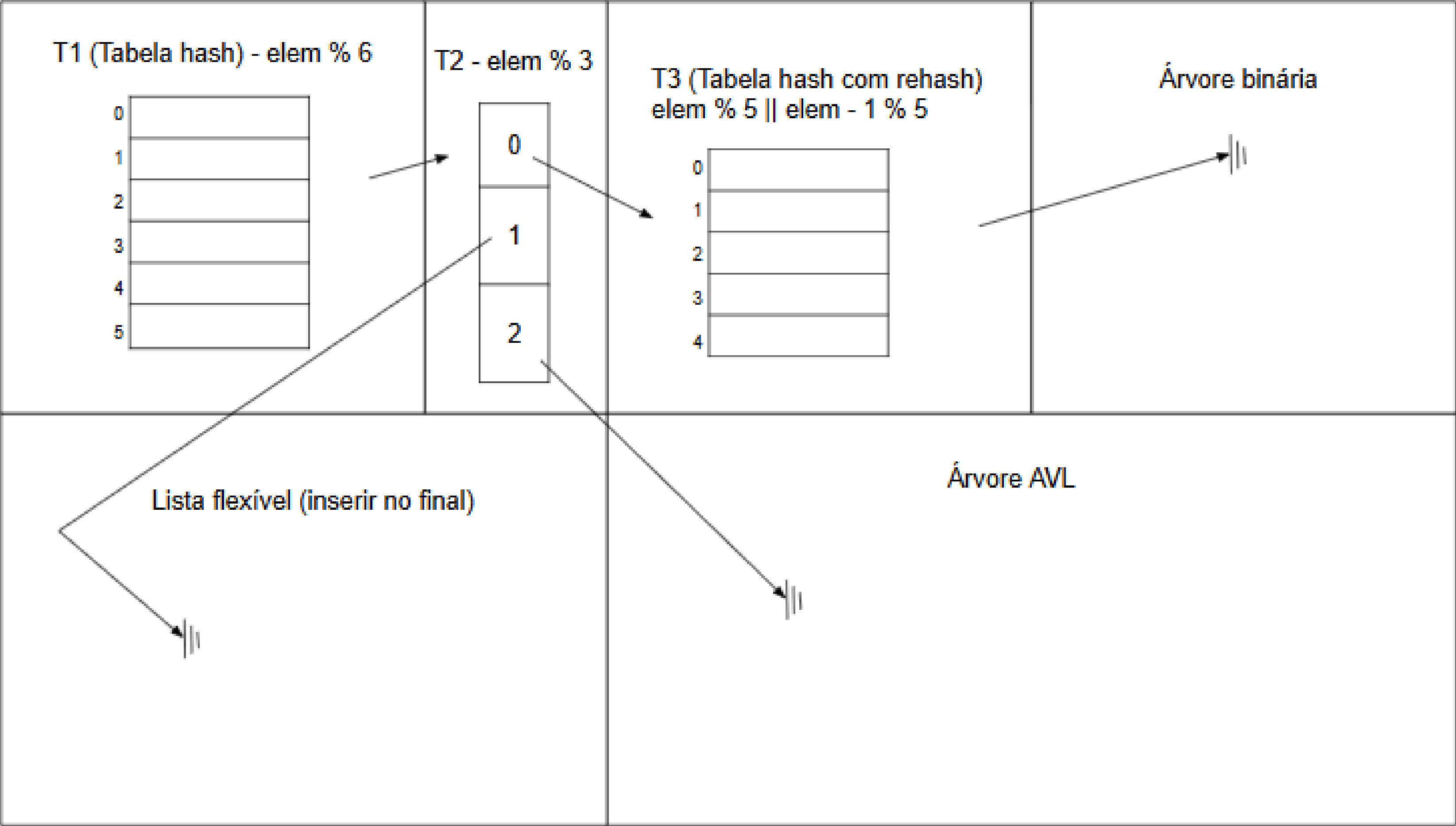


Árvore AVL

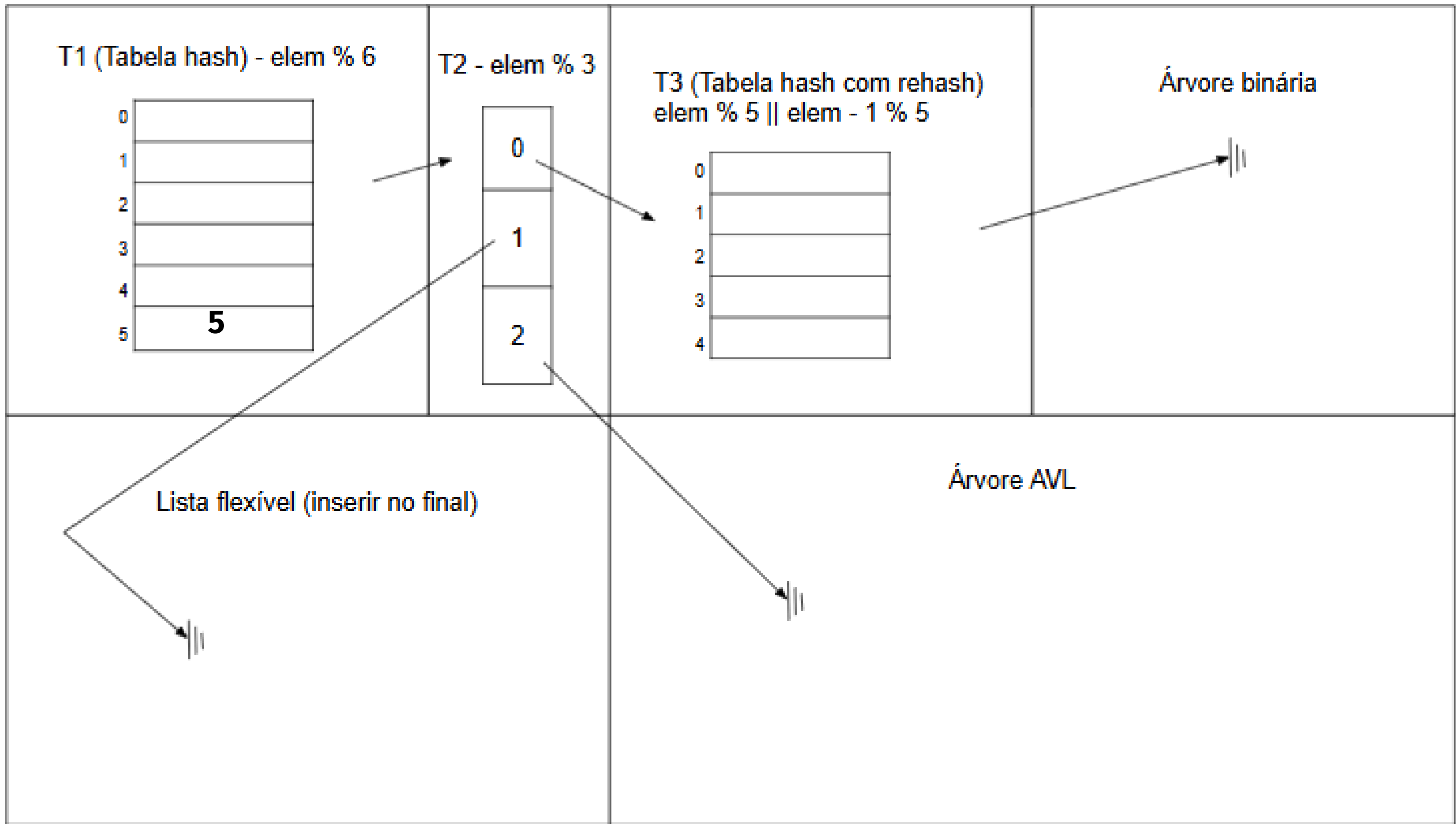


Tente inserir os seguintes elementos:

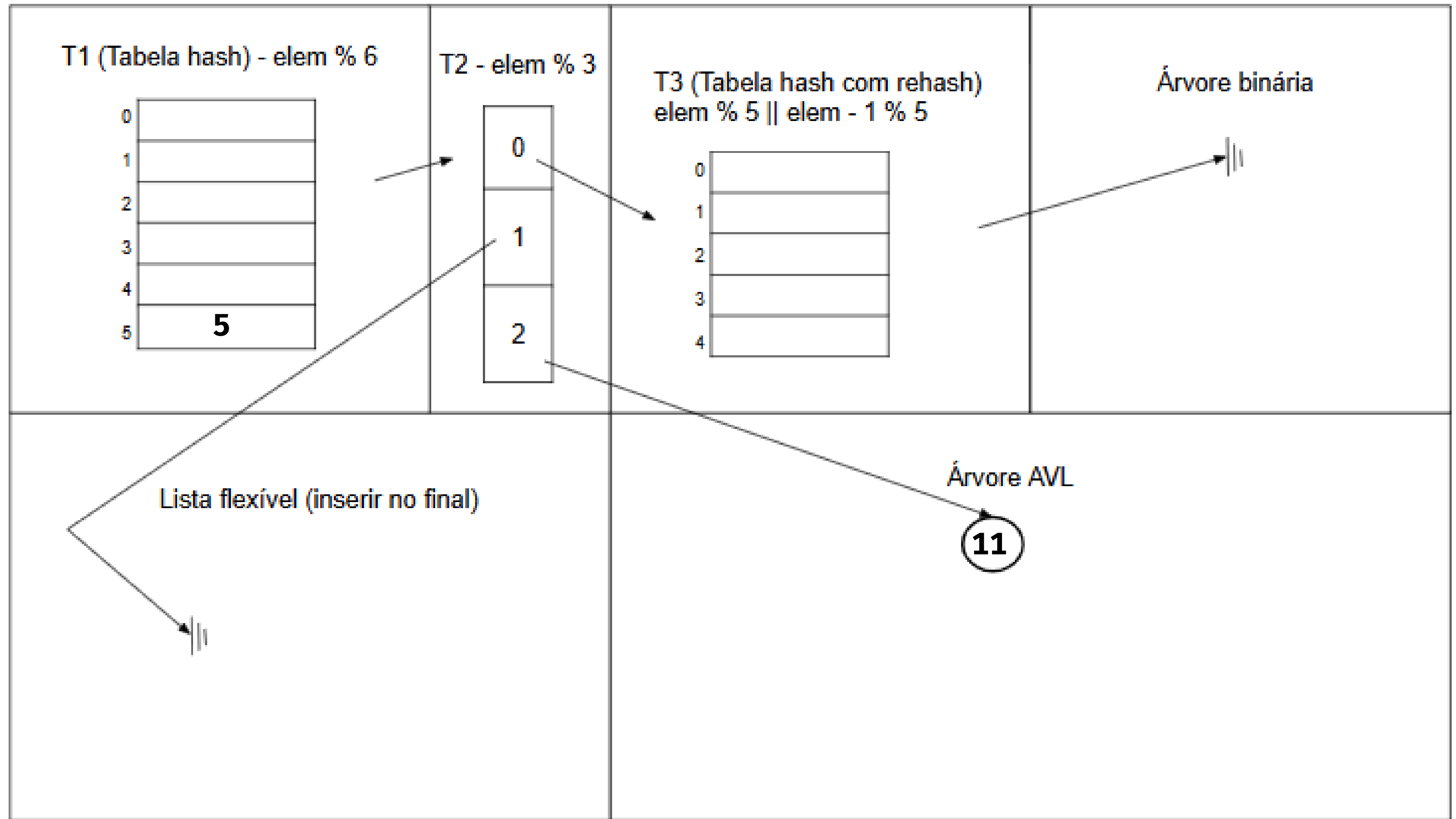
5, 11, 3, 9, 18, 21, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



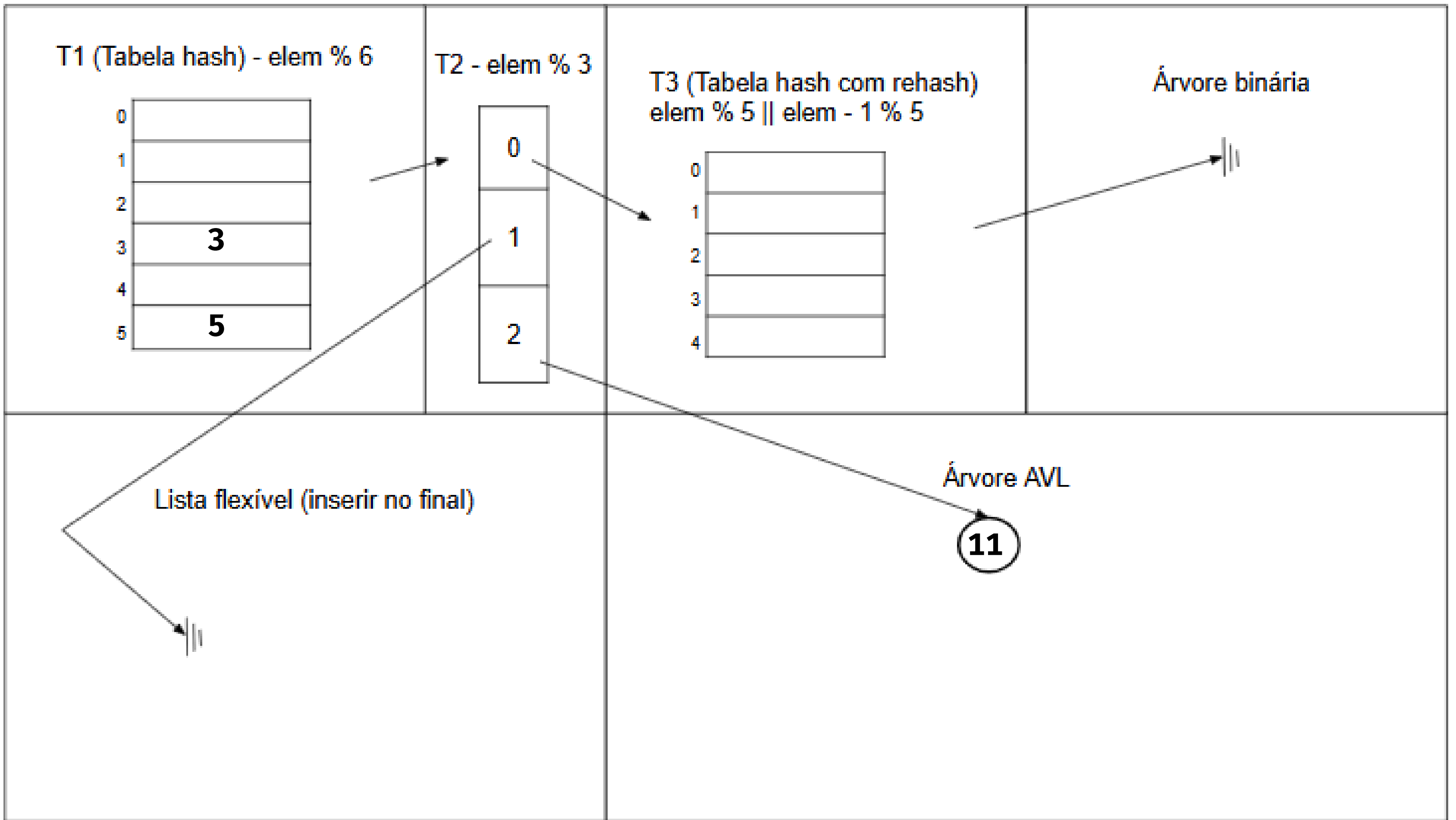
~~5~~, 11, 3, 9, 18, 21, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



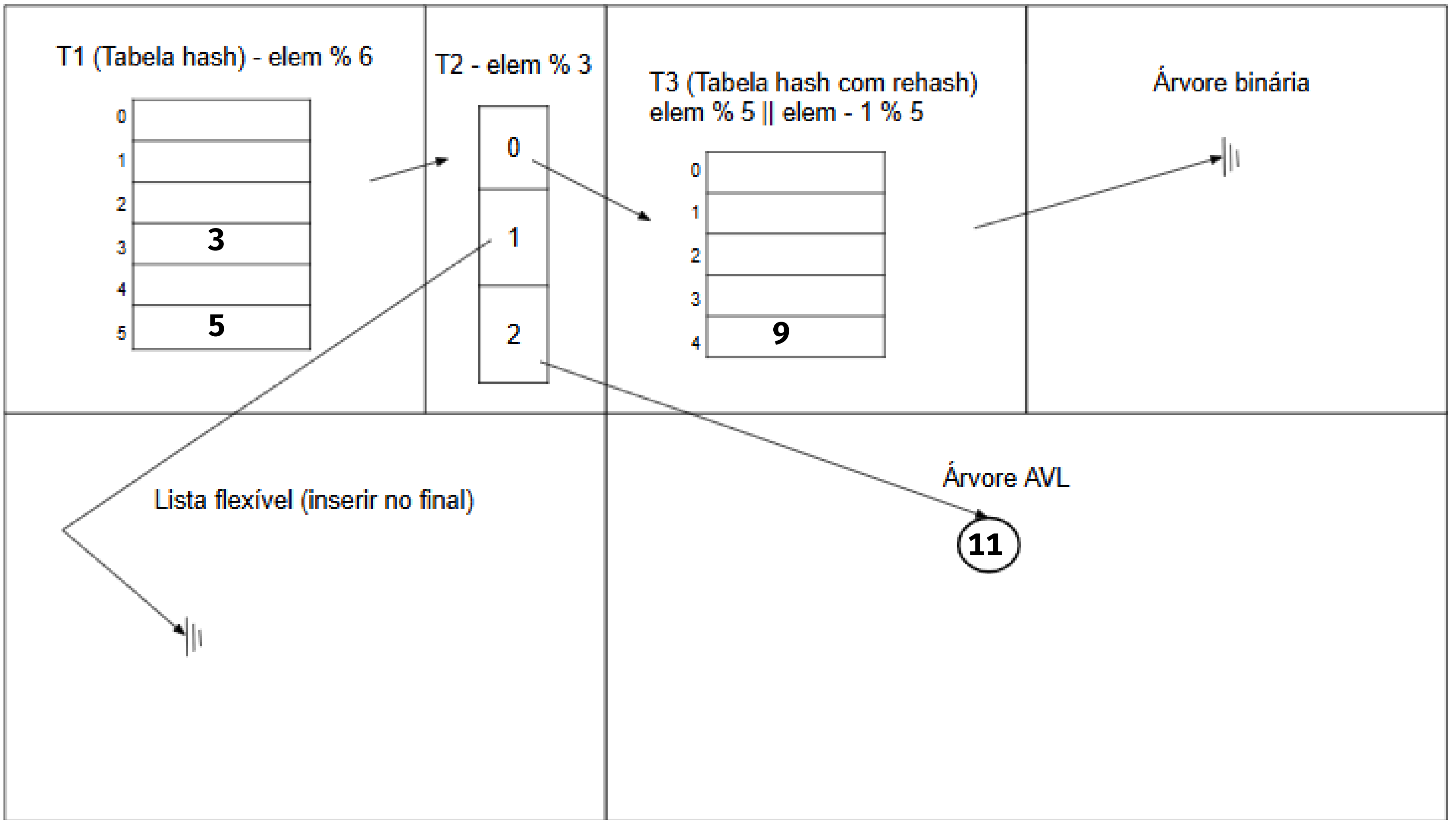
~~5~~, ~~11~~, 3, 9, 18, 21, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



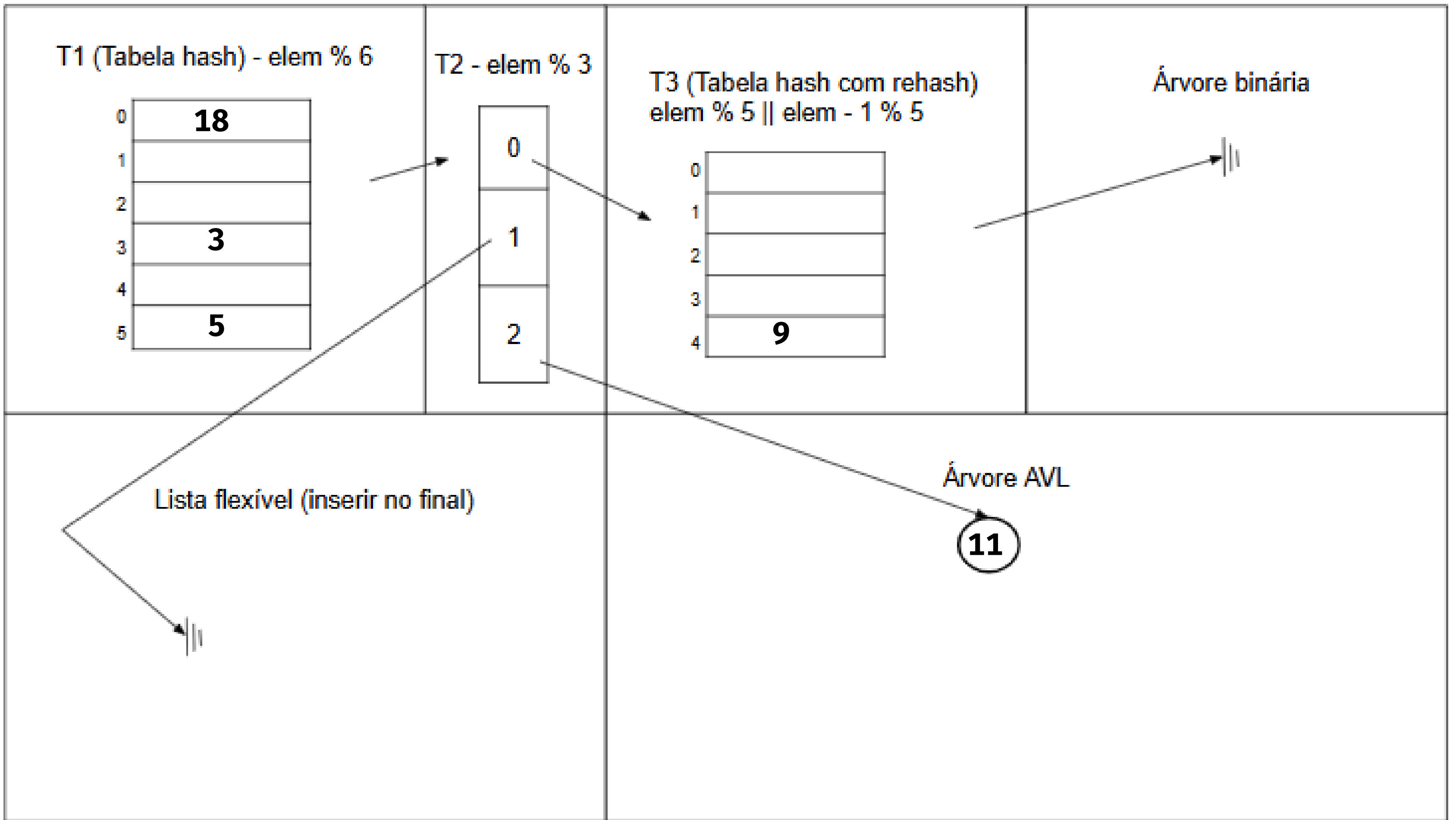
~~5~~, ~~11~~, ~~3~~, 9, 18, 21, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



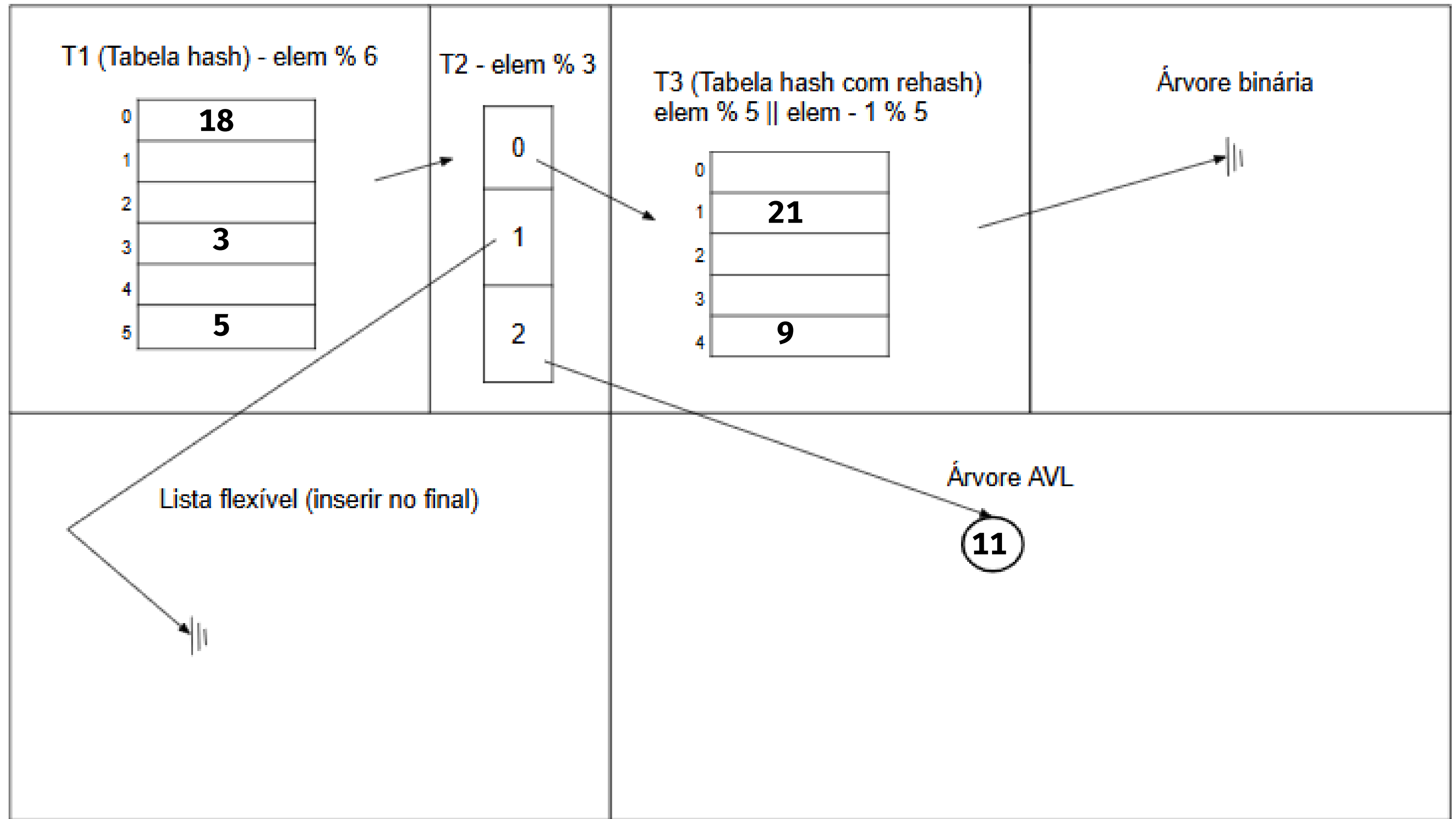
~~5~~, ~~11~~, ~~3~~, ~~9~~, 18, 21, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



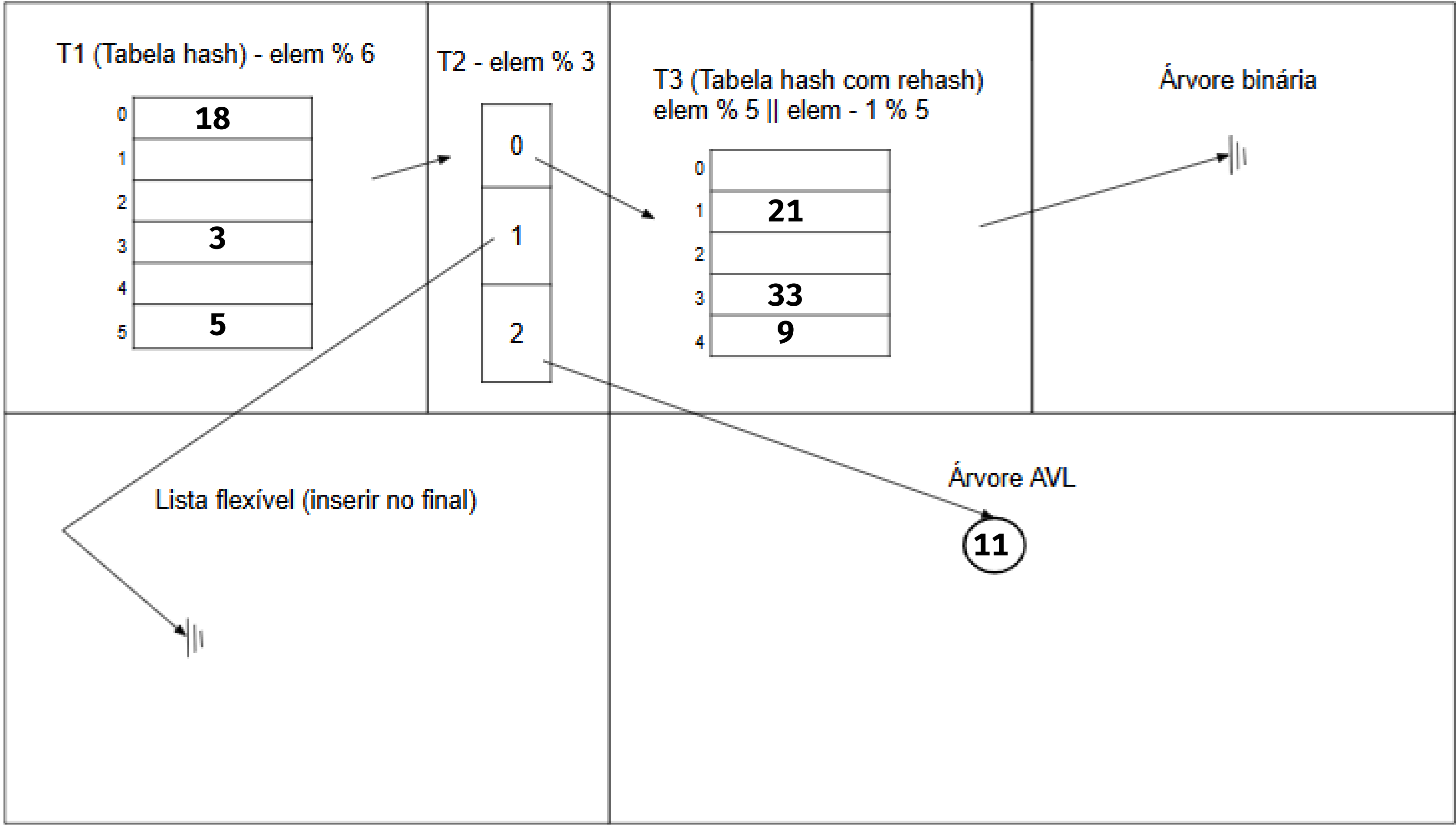
~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, 21, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



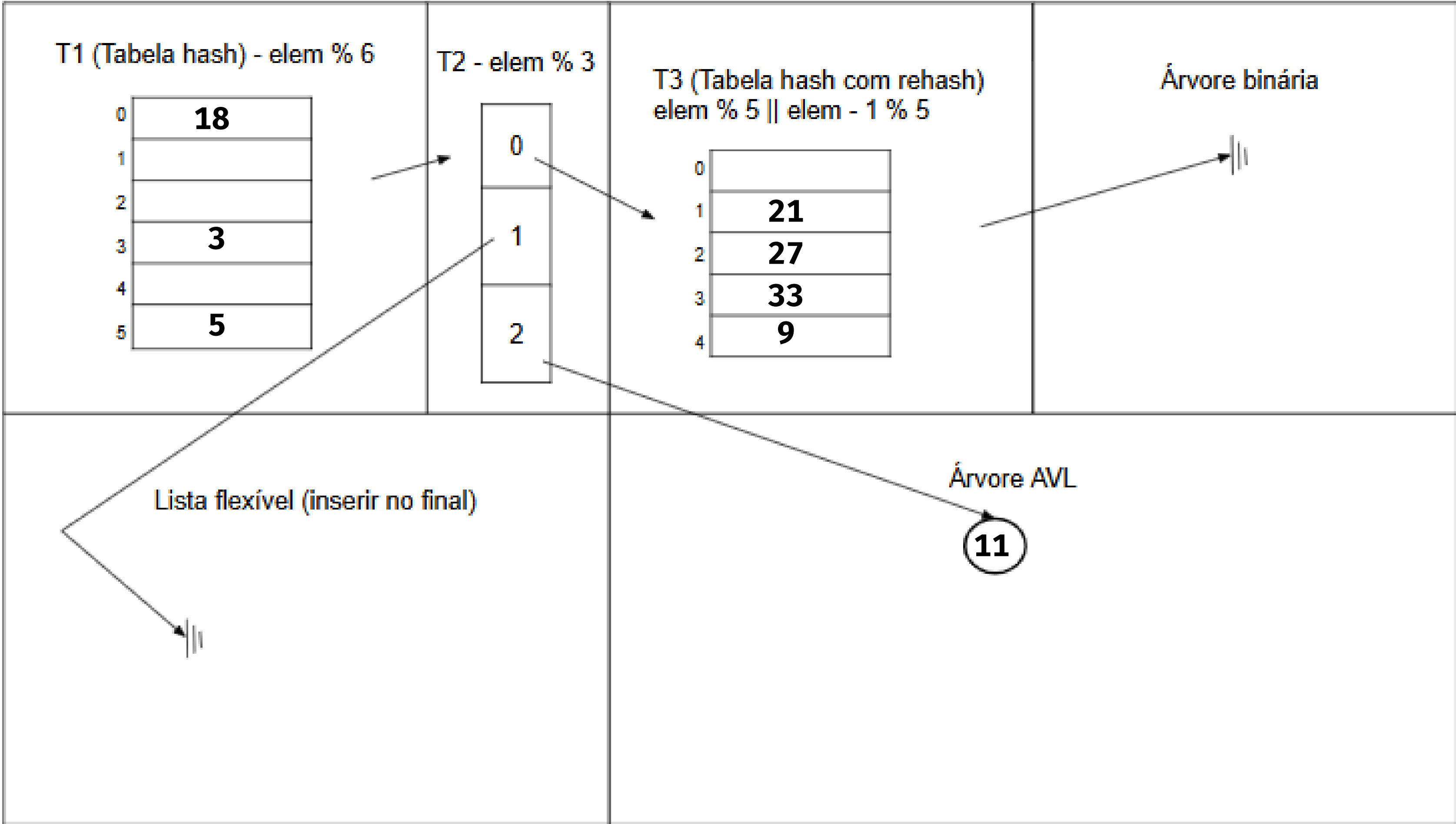
~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, 33, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



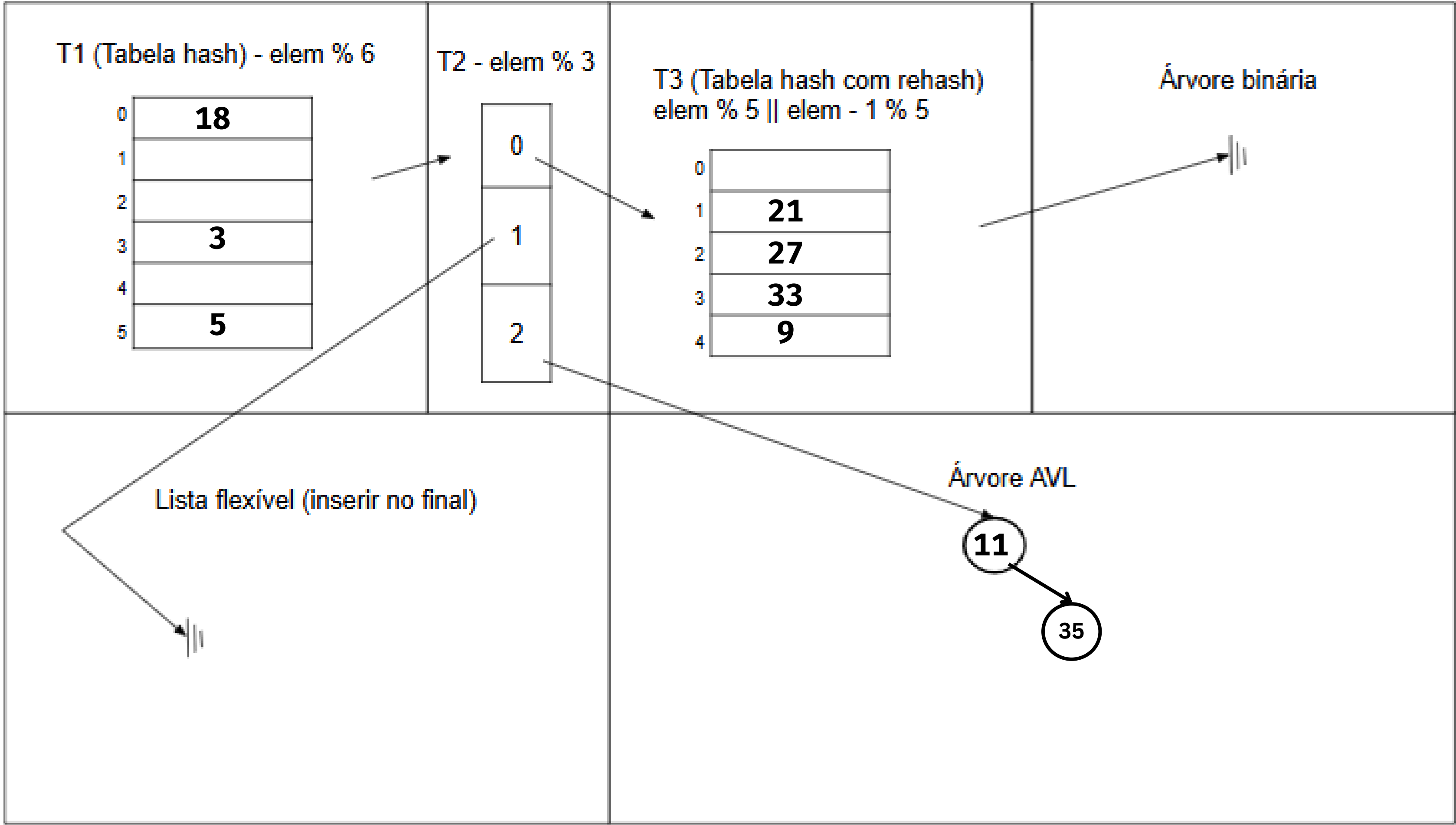
~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, 27, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



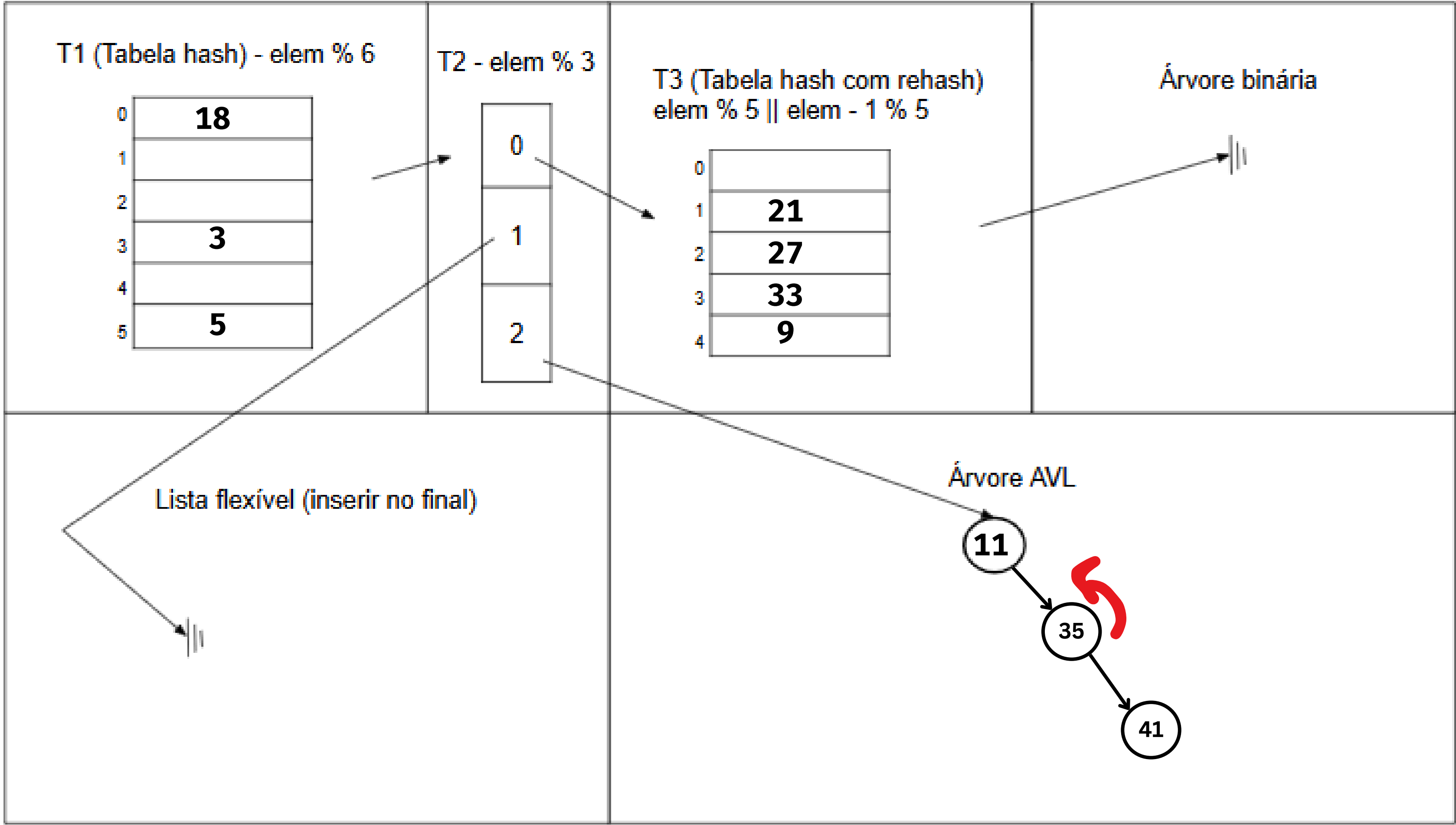
~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, 35, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



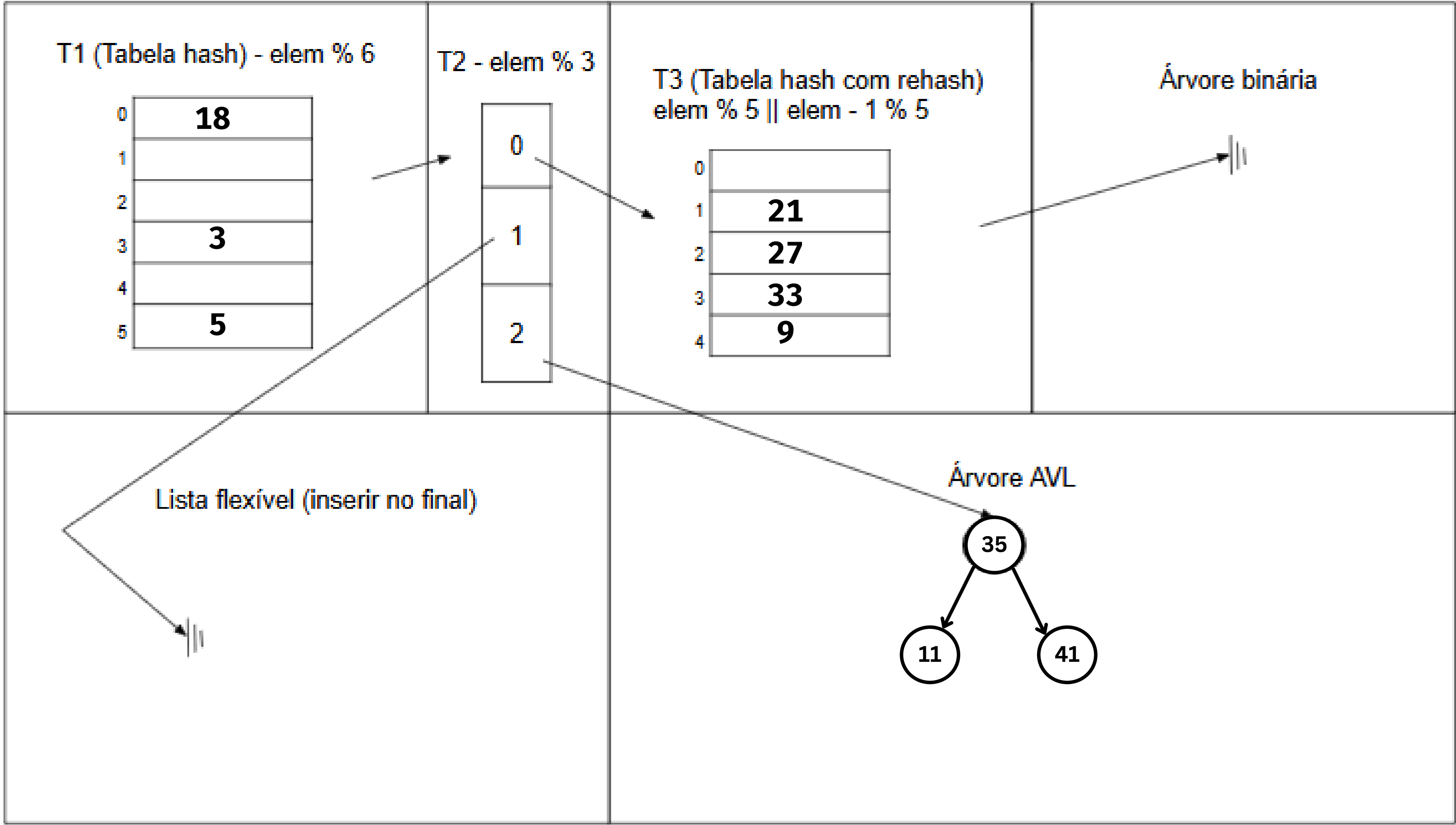
~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, 41, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, **41**, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, 36, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, 4, 8, 10, 20, 24, 48, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	
3	3
4	
5	5

T2 - elem % 3

0
1
2

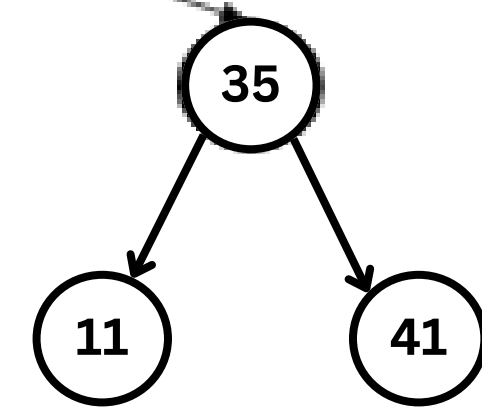
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

Árvore binária

Lista flexível (inserir no final)

Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, 8, 10, 20, 24, 48, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	
3	3
4	4
5	5

T2 - elem % 3

0
1
2

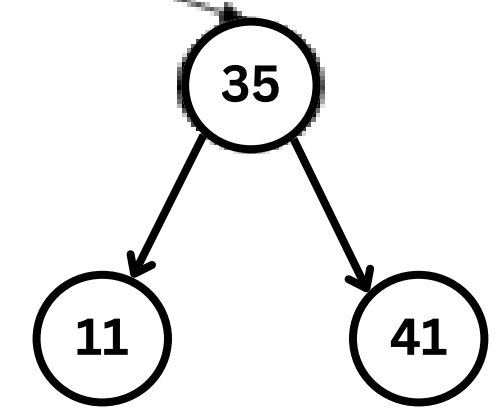
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

Árvore binária

Lista flexível (inserir no final)

Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, 10, 20, 24, 48, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	8
3	3
4	4
5	5

T2 - elem % 3

0
1
2

T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

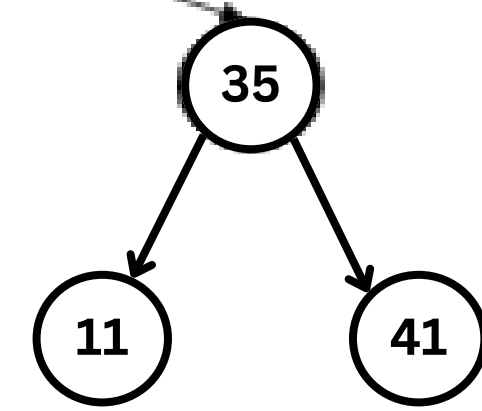
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, 20, 24, 48, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	8
3	3
4	4
5	5

T2 - elem % 3

0
1
2

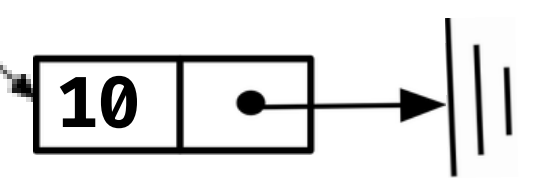
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

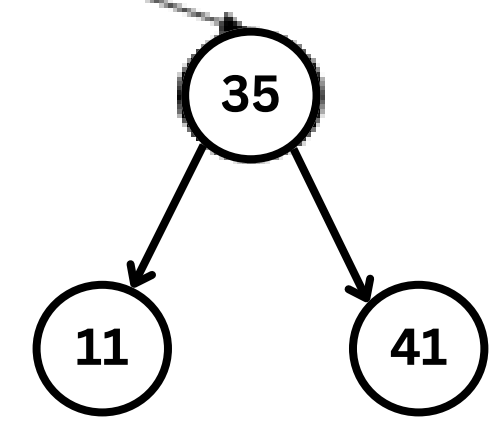
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~1~~, ~~8~~, ~~10~~, ~~20~~, 24, 48, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	8
3	3
4	4
5	5

T2 - elem % 3

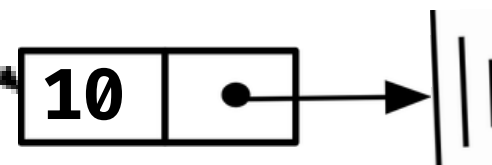
0
1
2

T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

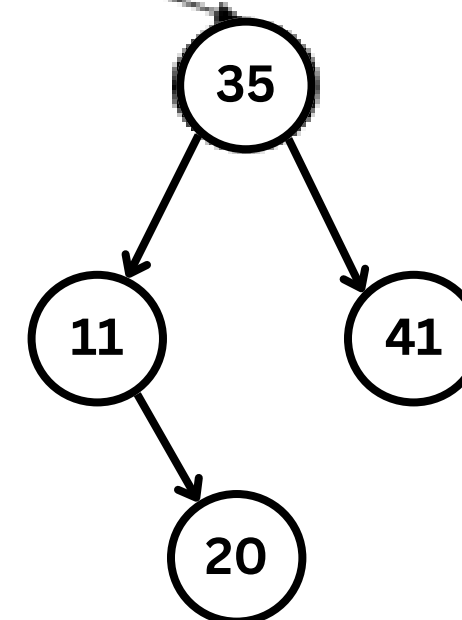
0	36
1	21
2	27
3	33
4	9

Árvore binária

Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, ~~20~~, ~~24~~, 48, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	8
3	3
4	4
5	5

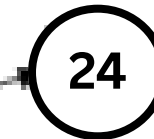
T2 - elem % 3

0
1
2

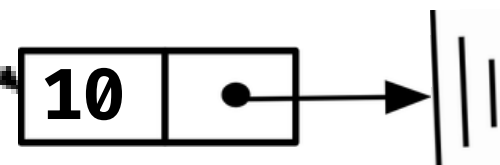
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

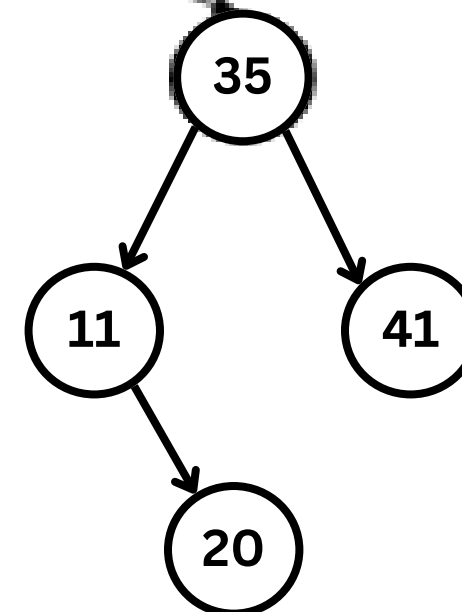
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, ~~20~~, ~~24~~, ~~48~~, 50, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	8
3	3
4	4
5	5

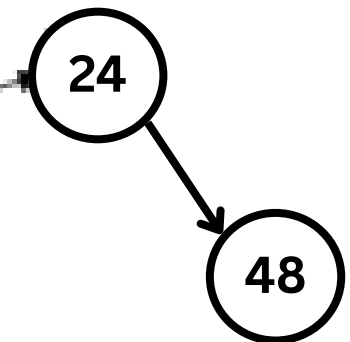
T2 - elem % 3

0
1
2

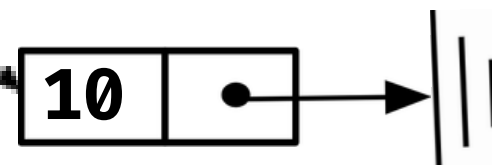
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

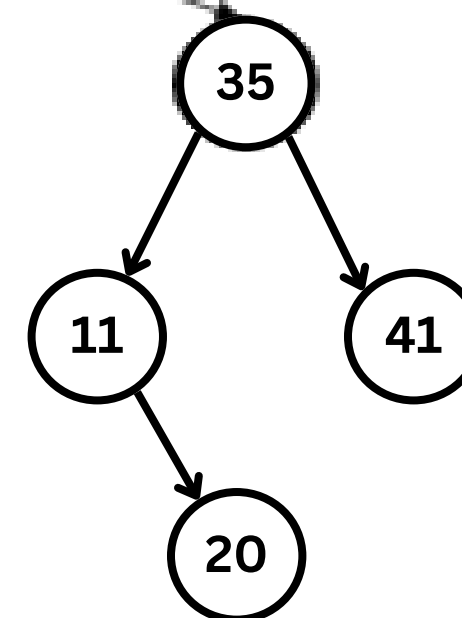
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, ~~20~~, ~~24~~, ~~48~~, ~~50~~, 25, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	
2	8
3	3
4	4
5	5

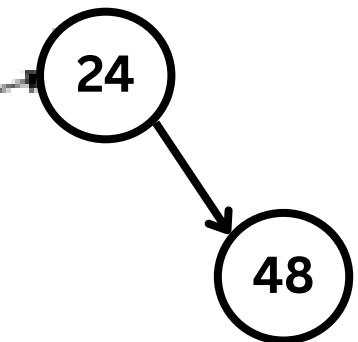
T2 - elem % 3

0
1
2

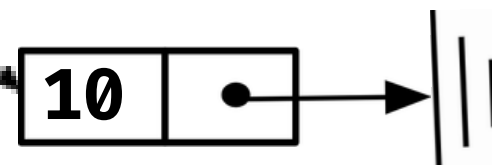
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

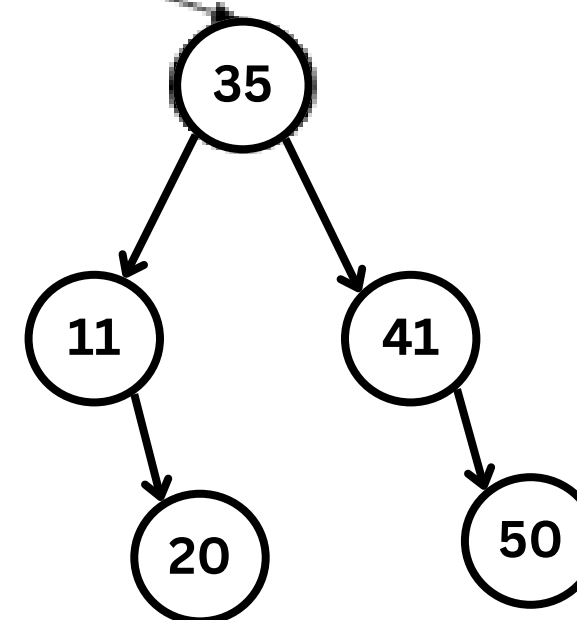
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, ~~20~~, ~~24~~, ~~48~~, ~~50~~, ~~25~~, 72, 19

T1 (Tabela hash) - elem % 6

0	18
1	25
2	8
3	3
4	4
5	5

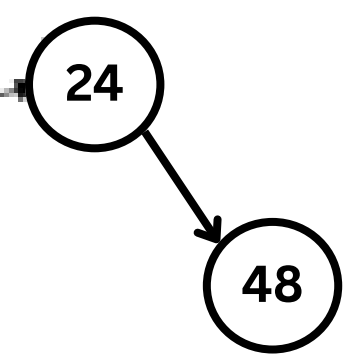
T2 - elem % 3

0
1
2

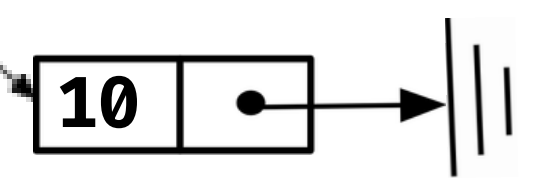
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

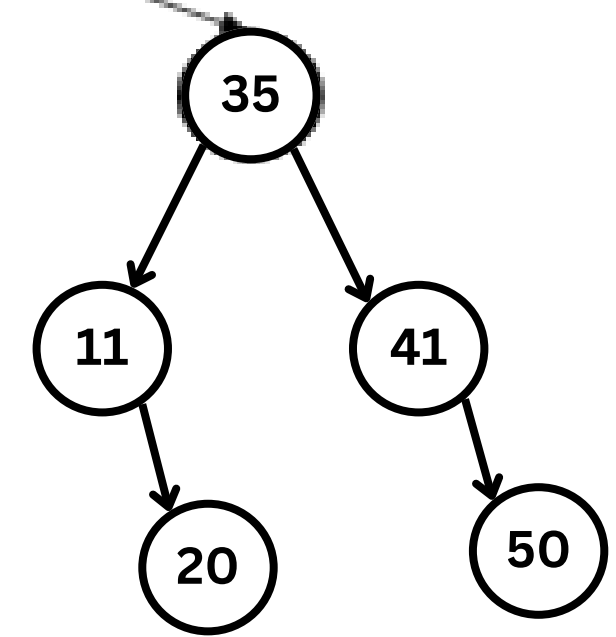
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, ~~20~~, ~~24~~, ~~48~~, ~~50~~, ~~25~~, ~~72~~, 19

T1 (Tabela hash) - elem % 6

0	18
1	25
2	8
3	3
4	4
5	5

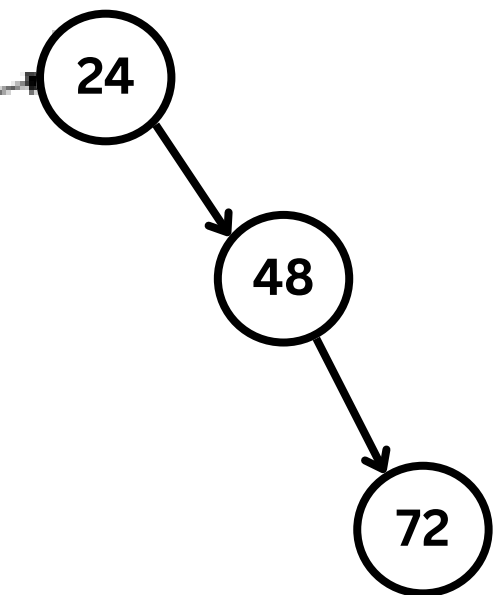
T2 - elem % 3

0
1
2

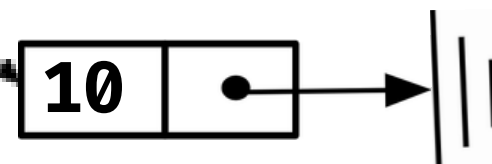
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

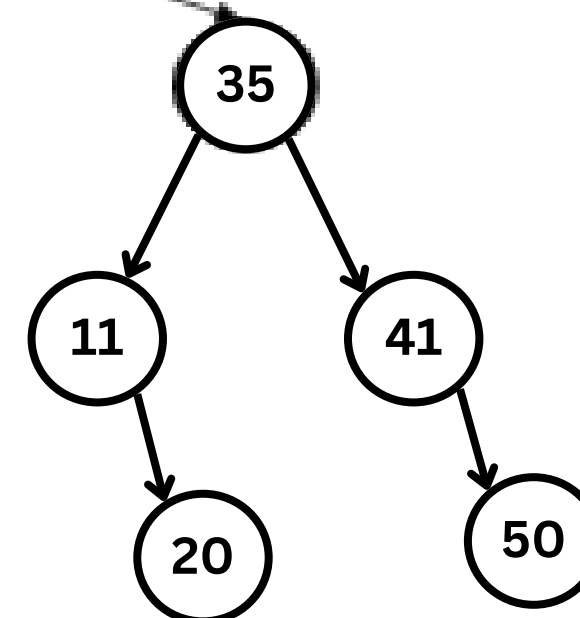
Árvore binária



Lista flexível (inserir no final)



Árvore AVL



~~5~~, ~~11~~, ~~3~~, ~~9~~, ~~18~~, ~~21~~, ~~33~~, ~~27~~, ~~35~~, ~~41~~, ~~36~~, ~~4~~, ~~8~~, ~~10~~, ~~20~~, ~~24~~, ~~48~~, ~~50~~, ~~25~~, ~~72~~, ~~19~~

T1 (Tabela hash) - elem % 6

0	18
1	25
2	8
3	3
4	4
5	5

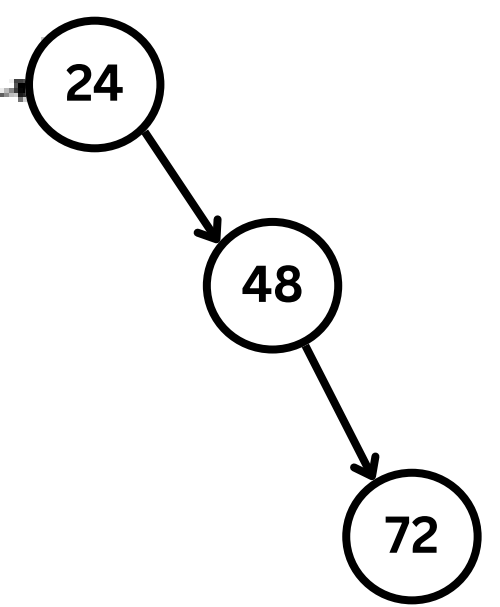
T2 - elem % 3

0
1
2

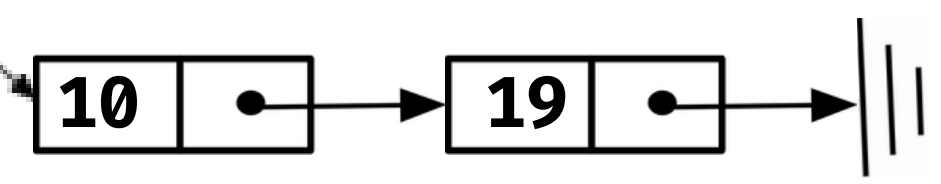
T3 (Tabela hash com rehash)
elem % 5 || elem - 1 % 5

0	36
1	21
2	27
3	33
4	9

Árvore binária



Lista flexível (inserir no final)



Árvore AVL

